Machine Learning for Systems and Control

5SC28

Lecture 2

dr. ir. Maarten Schoukens & prof. dr. ir. Roland Tóth

Control Systems Group

Department of Electrical Engineering

Eindhoven University of Technology

Academic Year: 2025-2026 (version 21/04/2026)

TU/e EINDHOVEN UNIVERSITY OF TECHNOLOGY

Learning Outcomes

What is an artificial neural network?

Why are artificial neural networks interesting for function approximation?

How to train a neural network? / What is backpropagation?

Artificial Neural Networks

What is an artificial neural network?

Simple feedforward networks

Approximation properties

Training an artificial neural network

Artificial Neural Networks

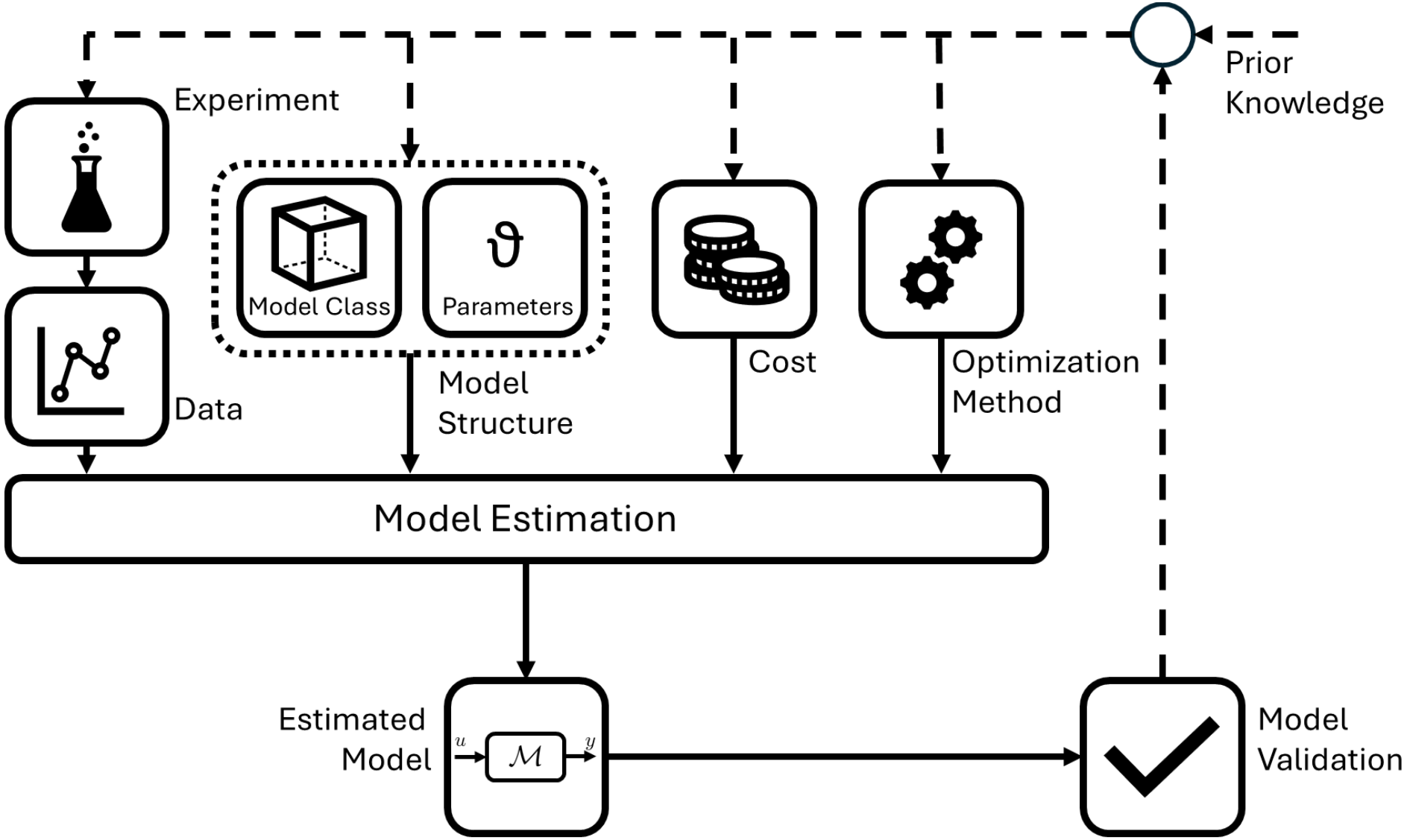
What is an artificial neural network?

Simple feedforward networks

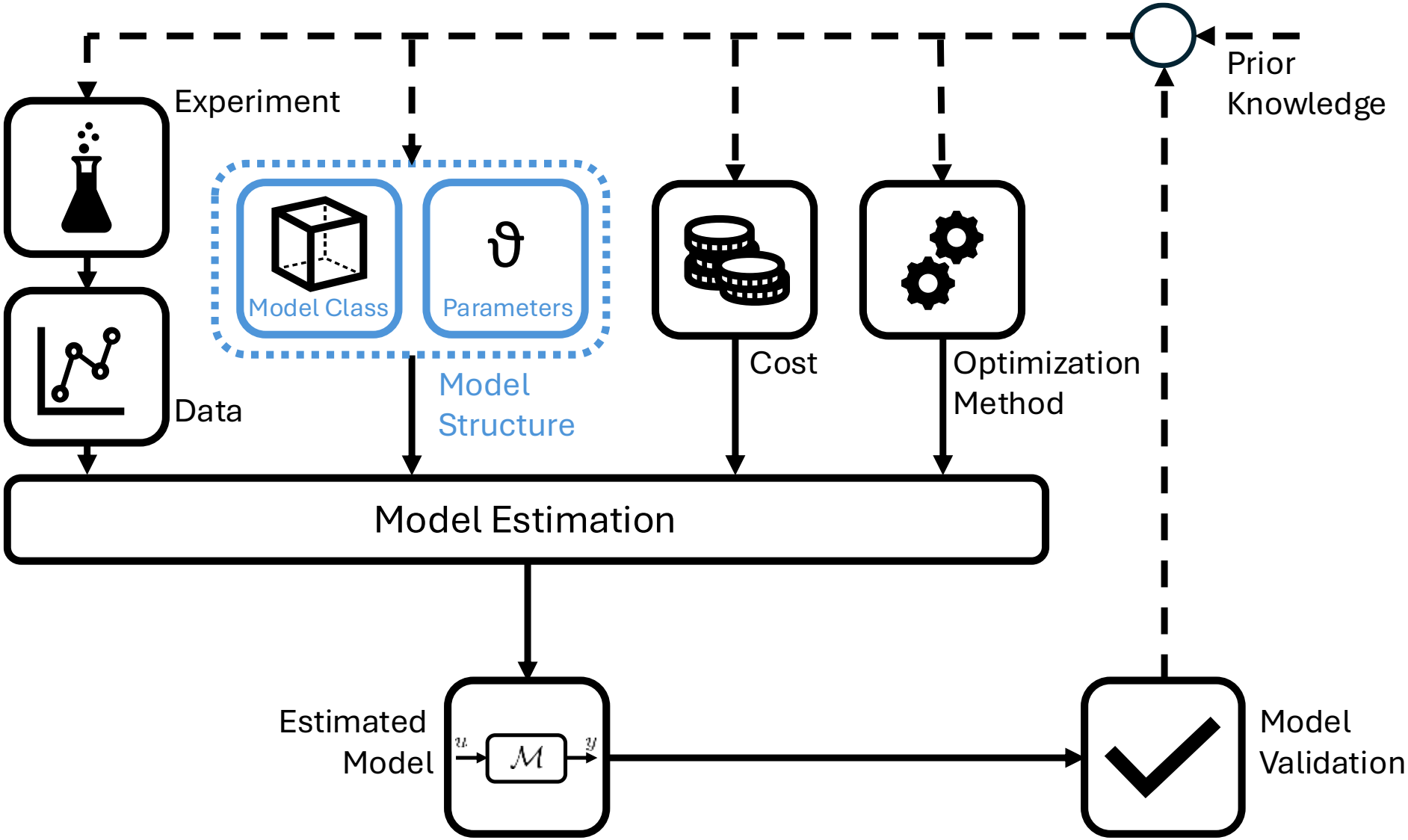
Approximation properties

Training an artificial neural network

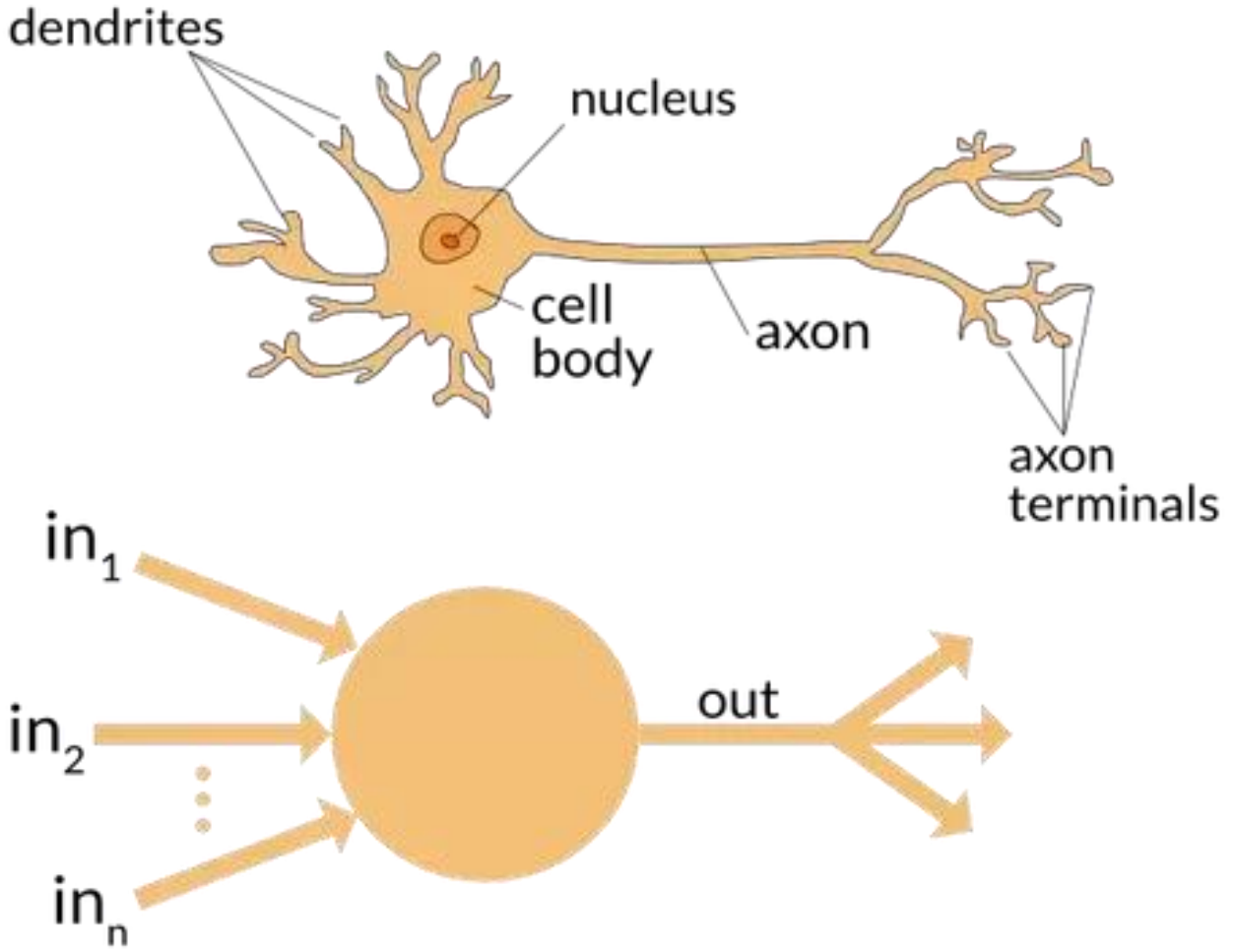
Data-Driven Modelling Process



Data-Driven Modelling Process

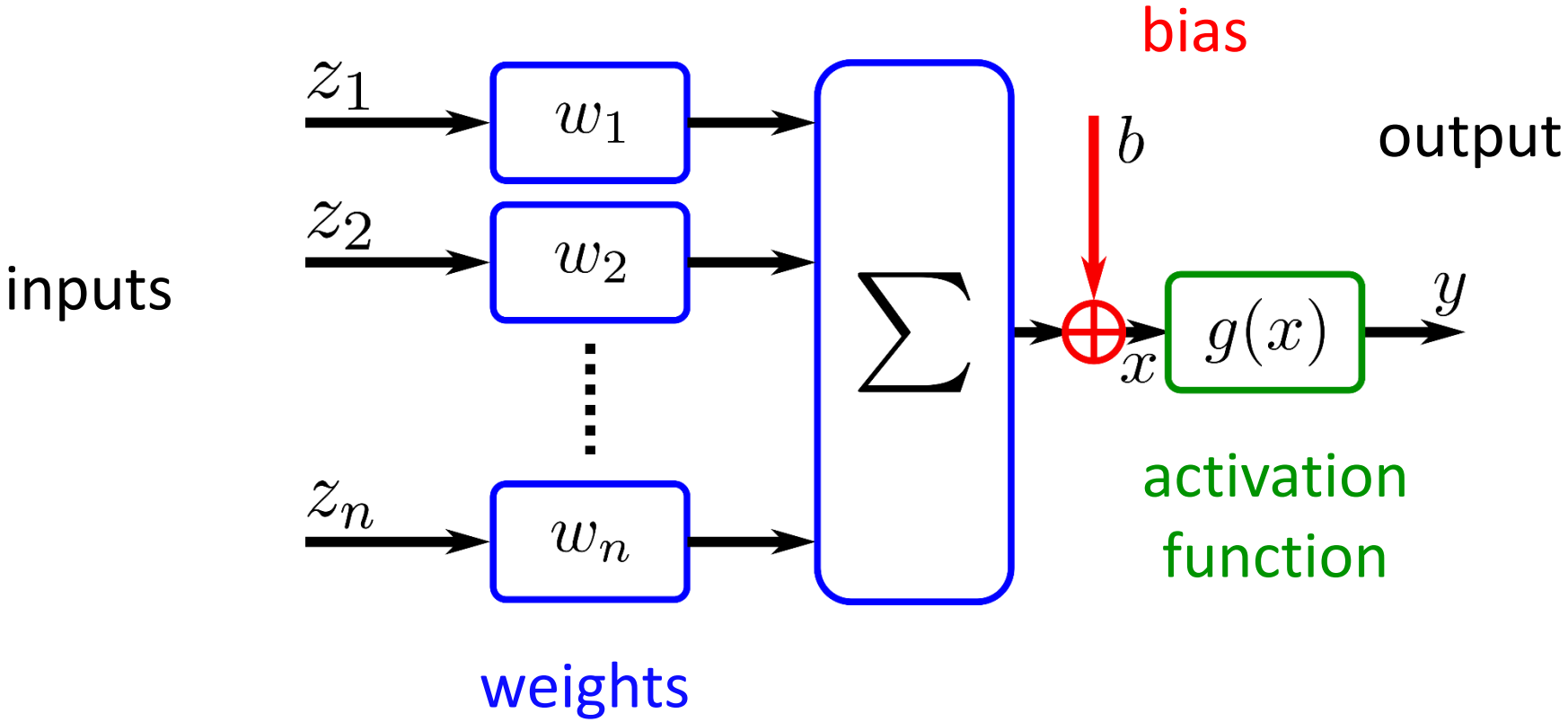


Biology Inspired



Source: https://www.quora.com/

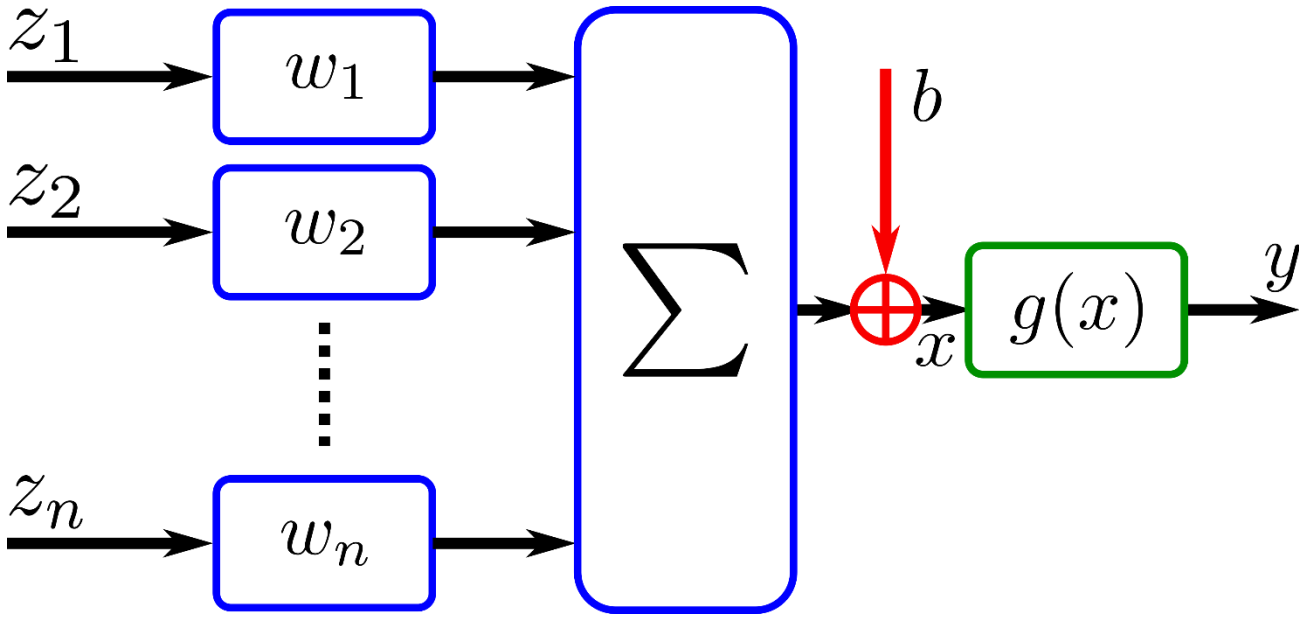
The Artificial Neuron



The Artificial Neuron

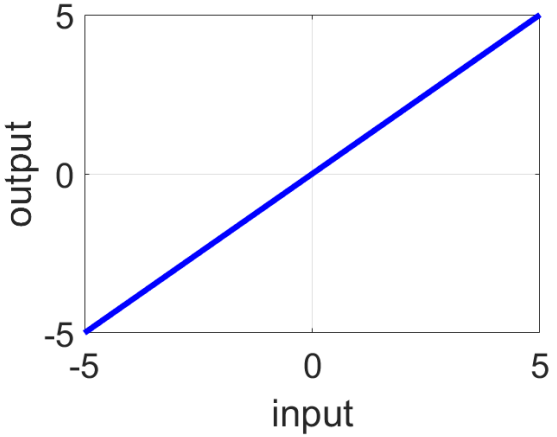
$$y = g(w^\top z + b)$$

$$w = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \end{bmatrix} \quad z = \begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_n \end{bmatrix}$$



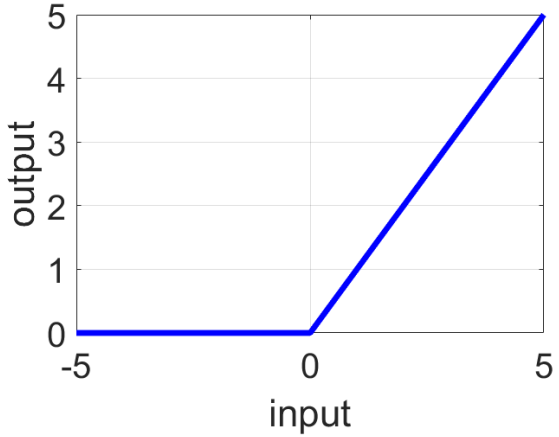
Activation Function

Linear



$$g(x) = x$$

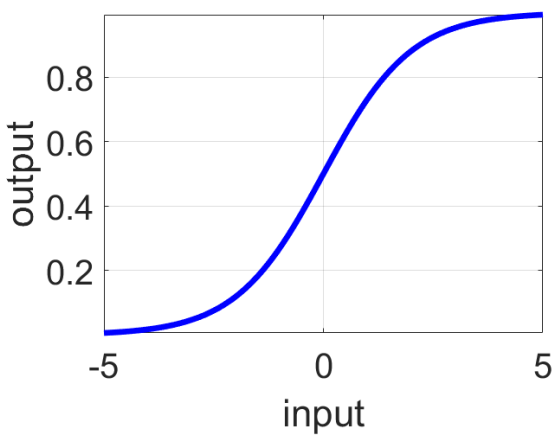
Rectified Linear Unit



$$g(x) = \begin{cases} 0 & \forall\, x < 0 \\ x & \forall\, x \geq 0 \end{cases}$$

ReLu

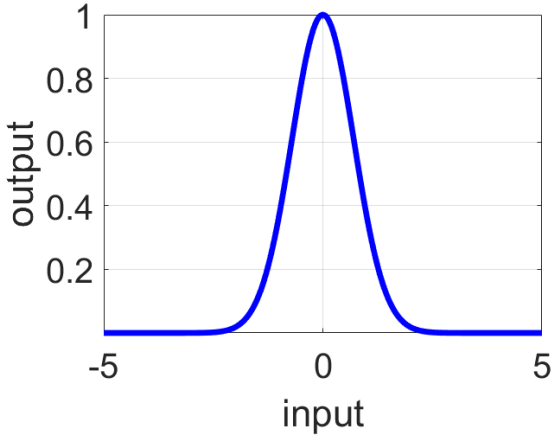
Sigmoid



$$g(x) = \frac{1}{1 + e^{-x}}$$

Logistic Function

Radial Basis



$$g(x) = e^{-x^2}$$

Artificial Neural Networks

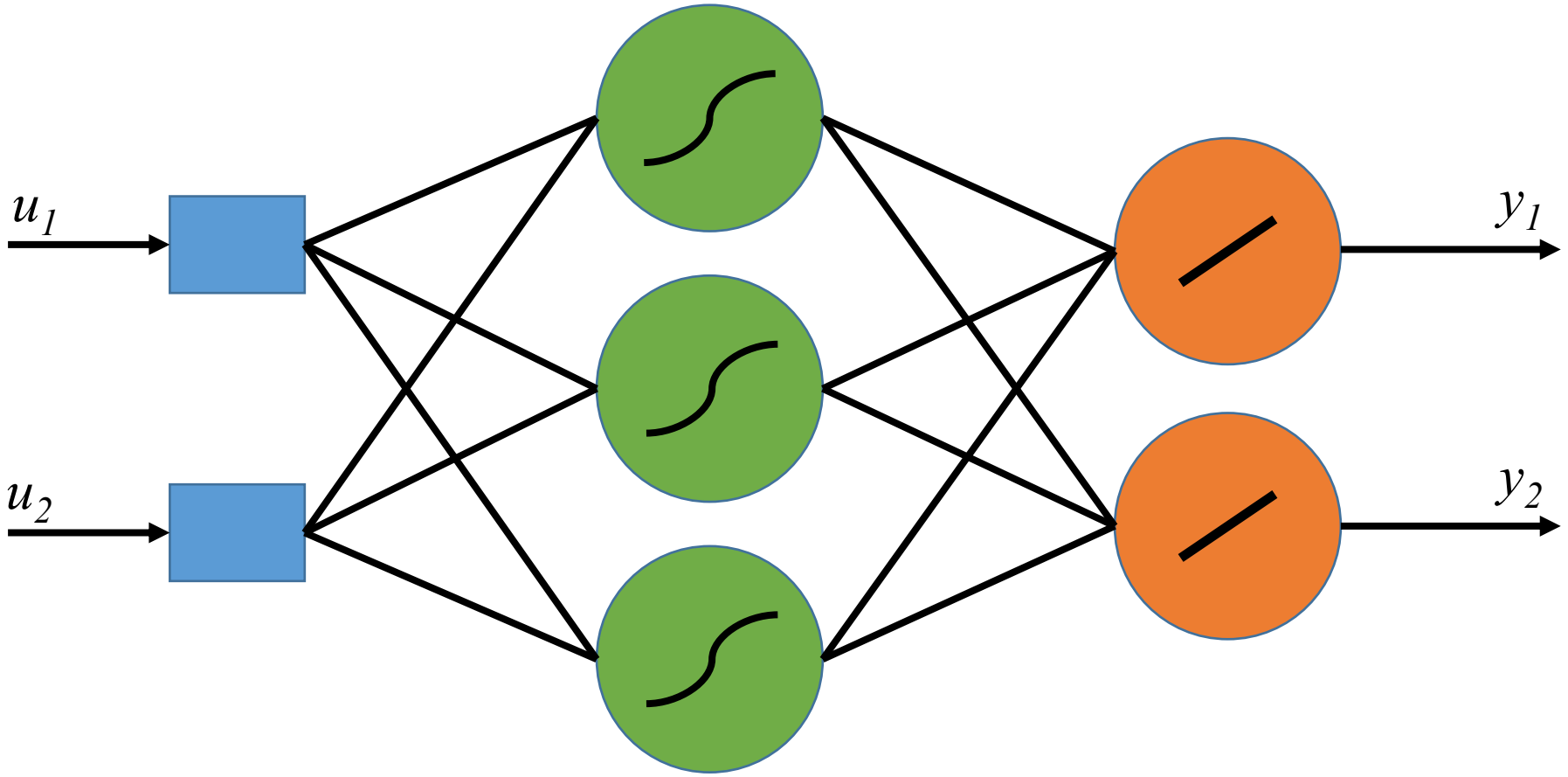
What is an artificial neural network?

Simple feedforward networks

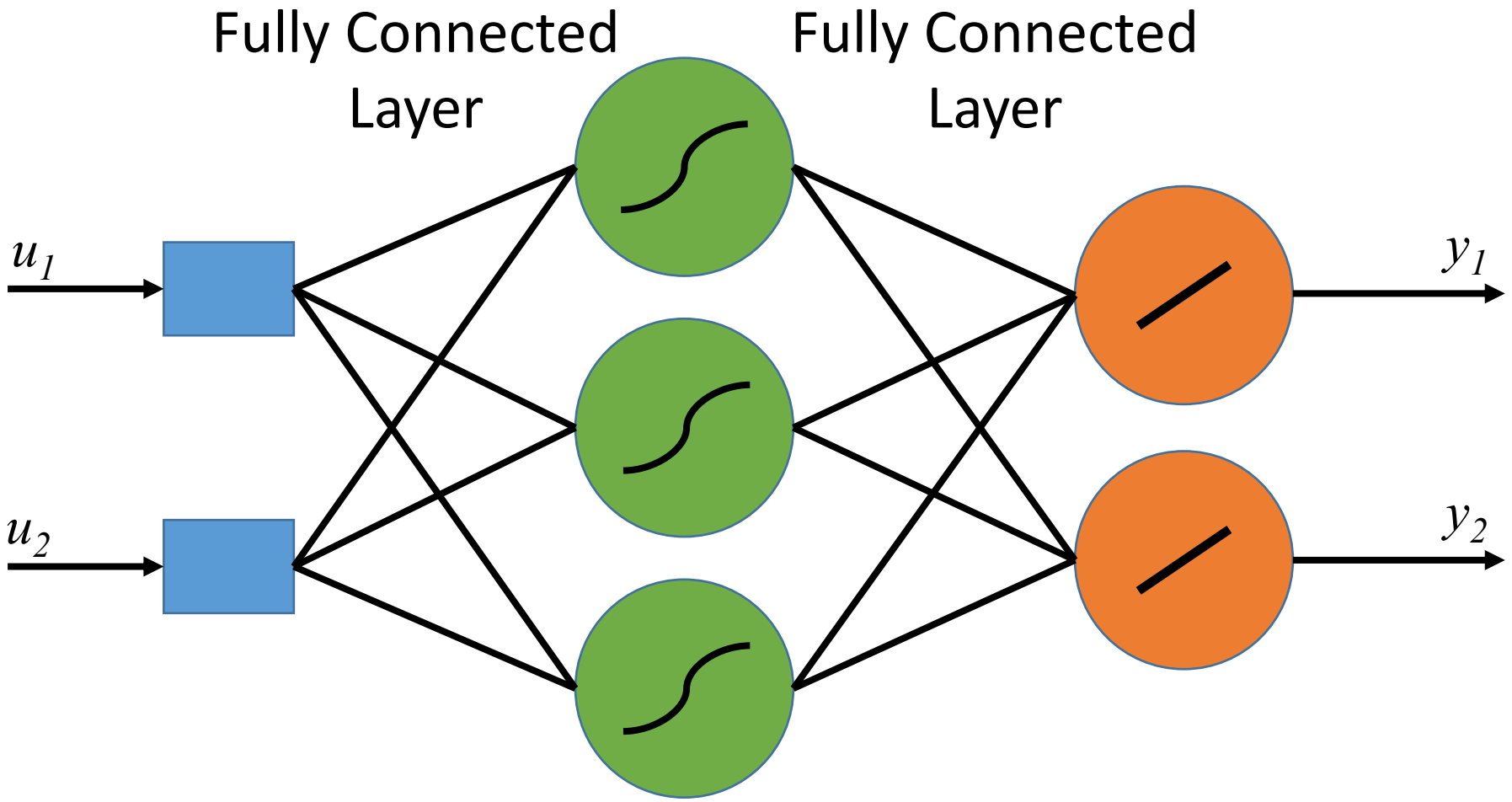
Approximation properties

Training an artificial neural network

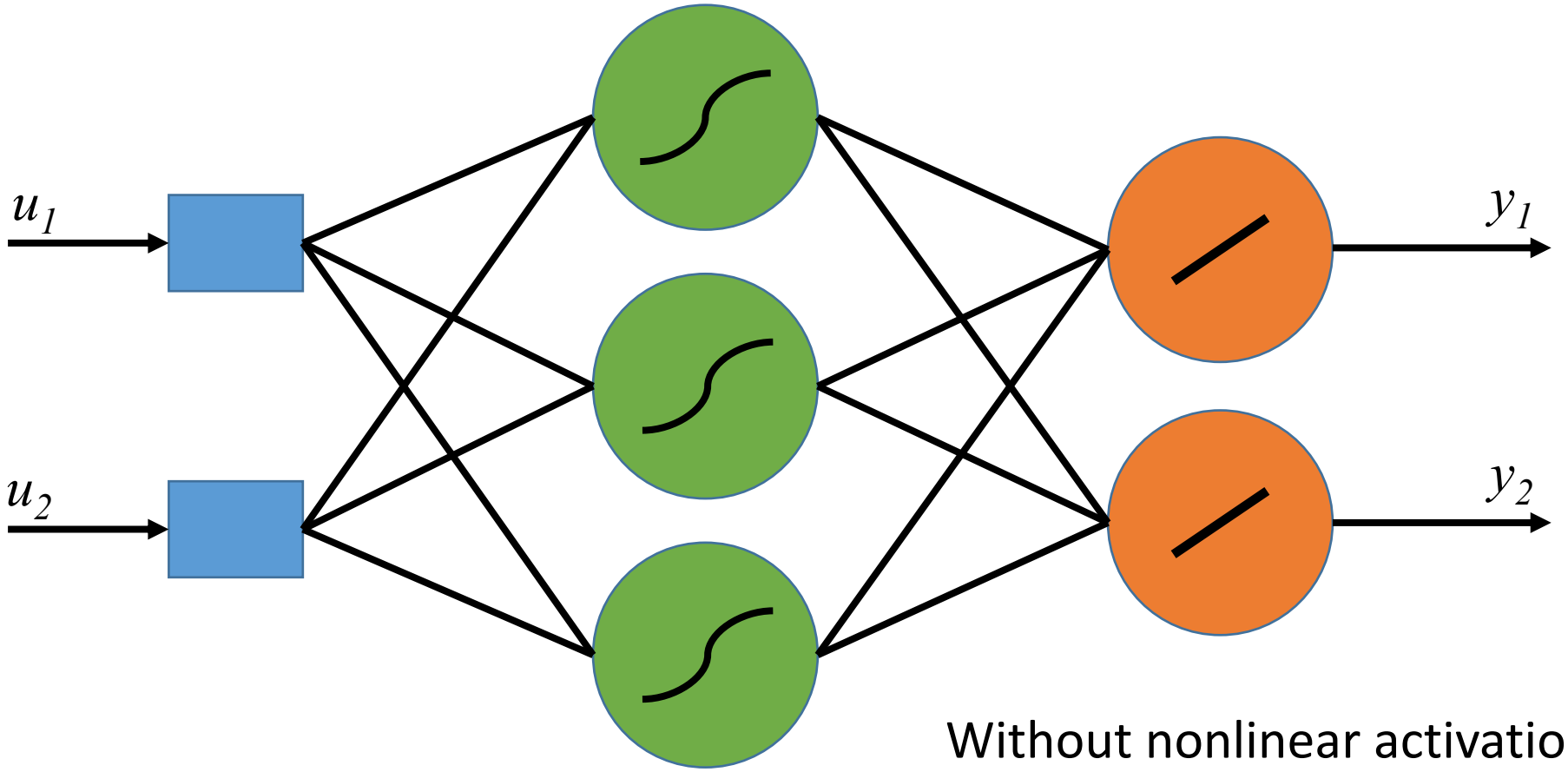
One Hidden Layer Network



One Hidden Layer Network



One Hidden Layer Network

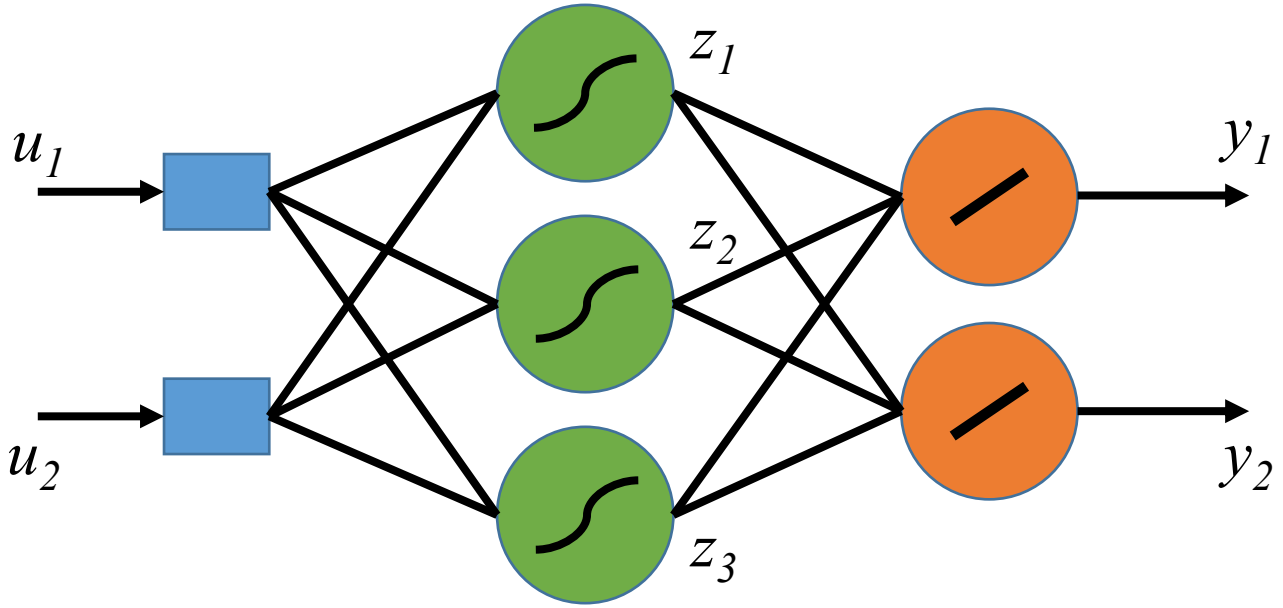


(Nonlinear) Activation

Without nonlinear activation layer this would be a simple linear/affine mapping

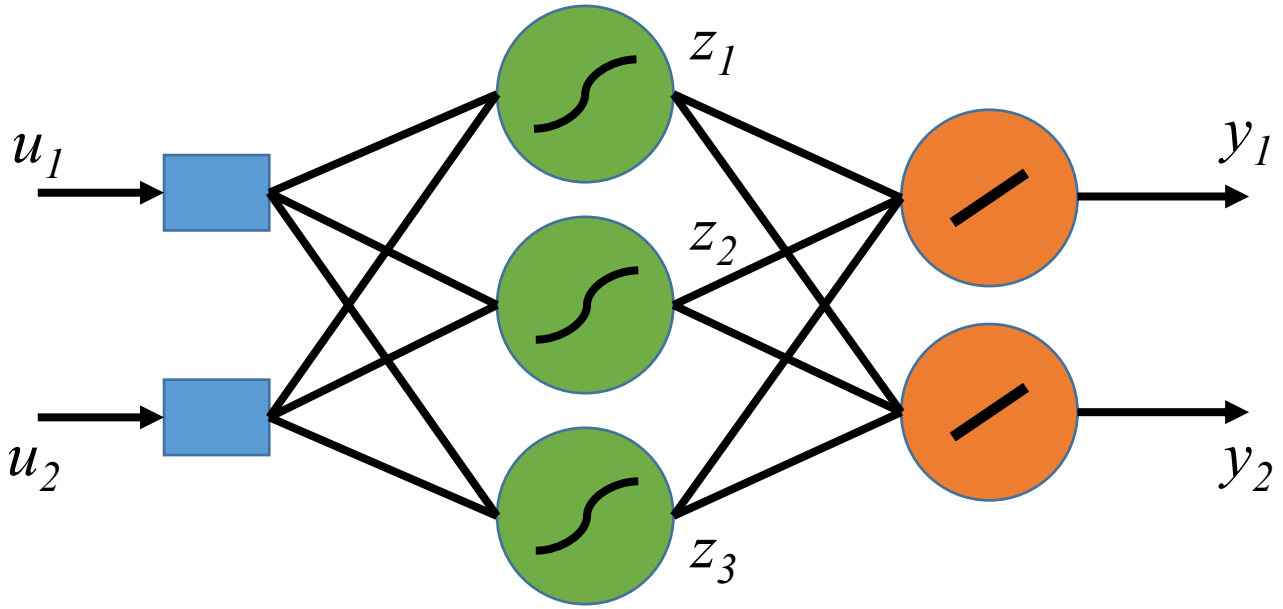
One Hidden Layer Network

Network output



One Hidden Layer Network

Network output



Output of **hidden layer** neuron j

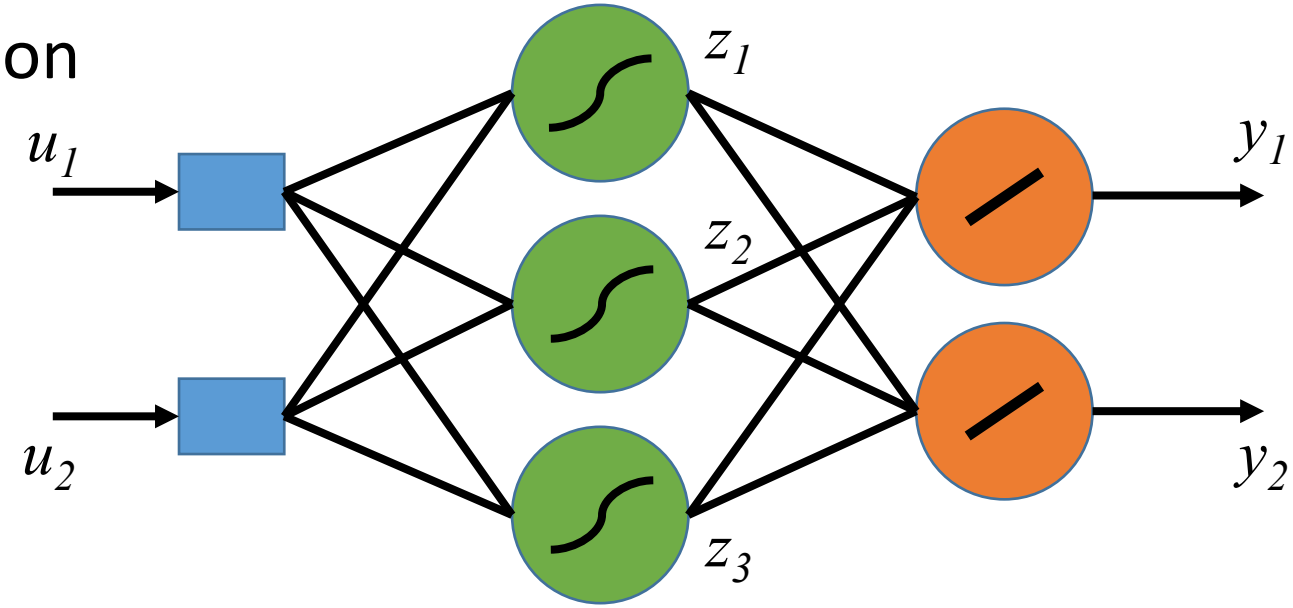
$$z_j = g\left(w_{1,j}^{\top} u + b_{1,j}\right)$$

Output of linear **output layer** neuron j

$$y_j = w_{2,j}^{\top} z + b_{2,j}$$

One Hidden Layer Network

Network output – matrix notation



Output of **hidden layer**

$$z = g\left(W_1 u + b_1\right)$$

Output of linear **output layer**

$$y = W_2 z + b_2$$

$$W_i = \begin{bmatrix} w_{i,1} & w_{i,2} & \dots & w_{i,n_i} \end{bmatrix}^\top$$

$$b_i = \begin{bmatrix} b_{i,1} & b_{i,2} & \dots & b_{i,n_i} \end{bmatrix}^\top$$

n_i = number of neurons in layer i

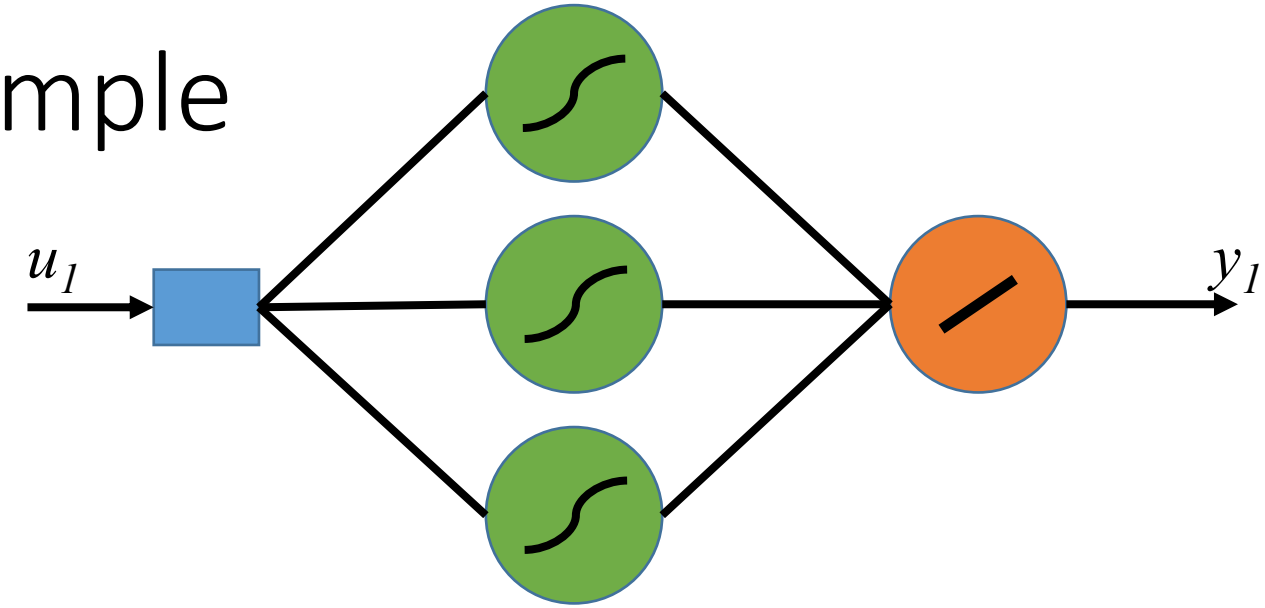
Network Output - Example

Output of hidden layer neuron j

$$z_j = g\left(w_{1,j}^{\top} u + b_{1,j}\right)$$

Output of linear output layer neuron j

$$y_j = w_{2,j}^{\top} z + b_{2,j}$$

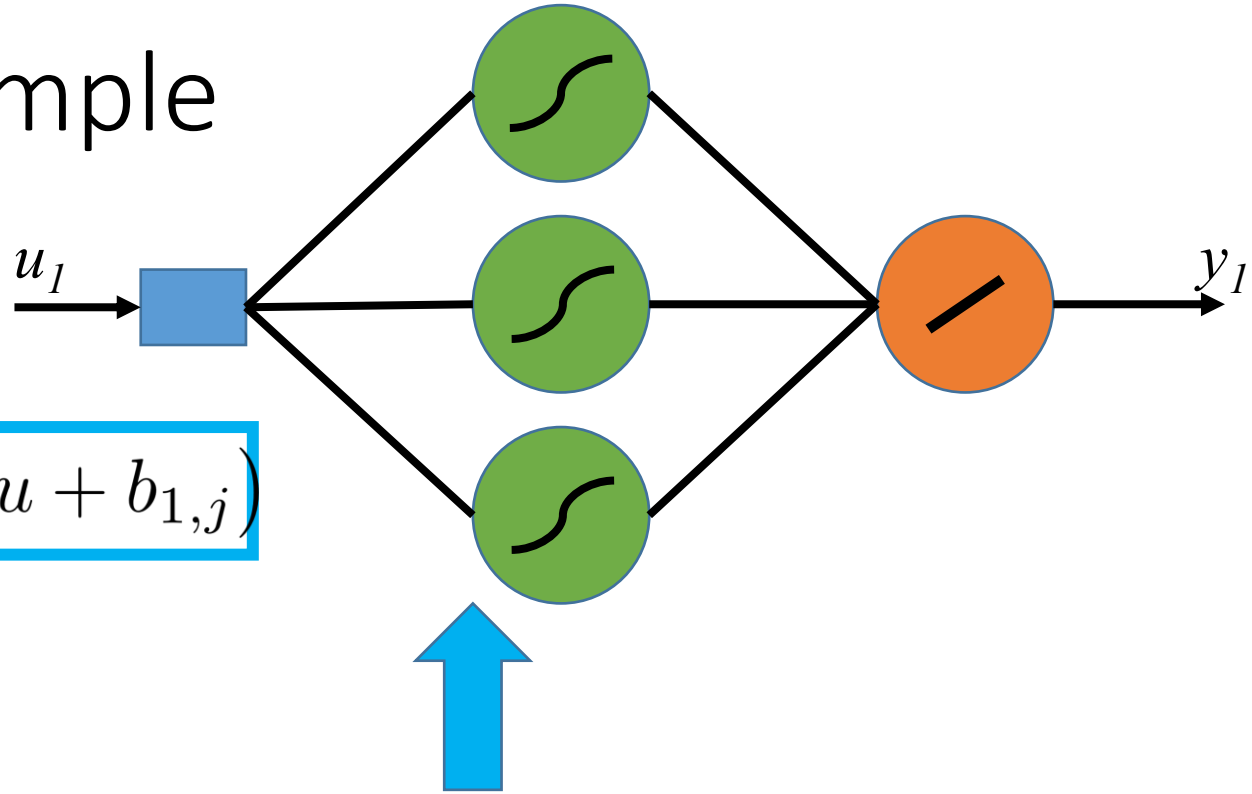


$\boldsymbol{w}_1$ = [1 -3 0.2] $\boldsymbol{w}_2$ = [1 2 0.5]

$\boldsymbol{b}_1$ = [0 -1 0.4] $\boldsymbol{b}_2$ = -1.5

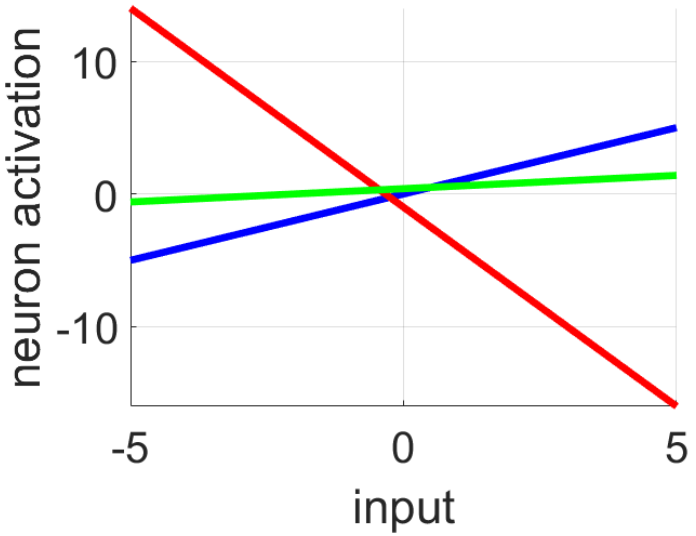
Network Output - Example

w_1 = [1 -3 0.2]
b_1 = [0 -1 0.4]
w_2 = [1 2 0.5]
b_2 = -1.5



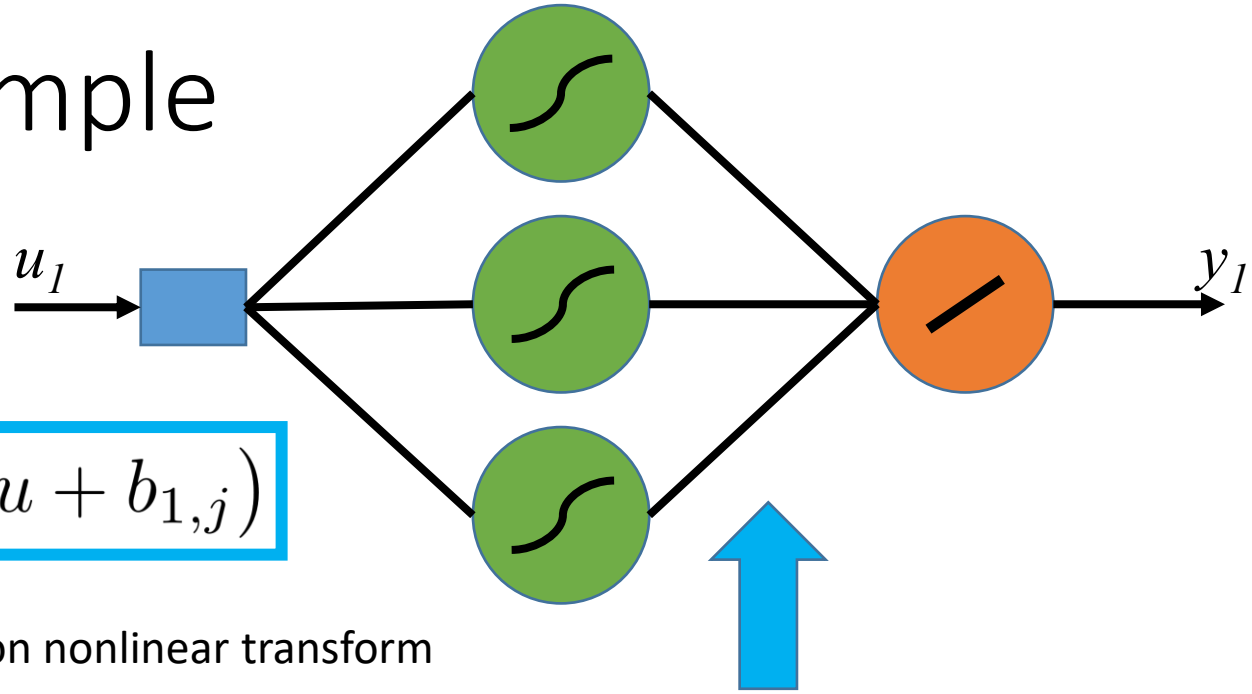
$$z_j = g\left(w_{1,j}^{\top} u + b_{1,j}\right)$$

Activation input weighting



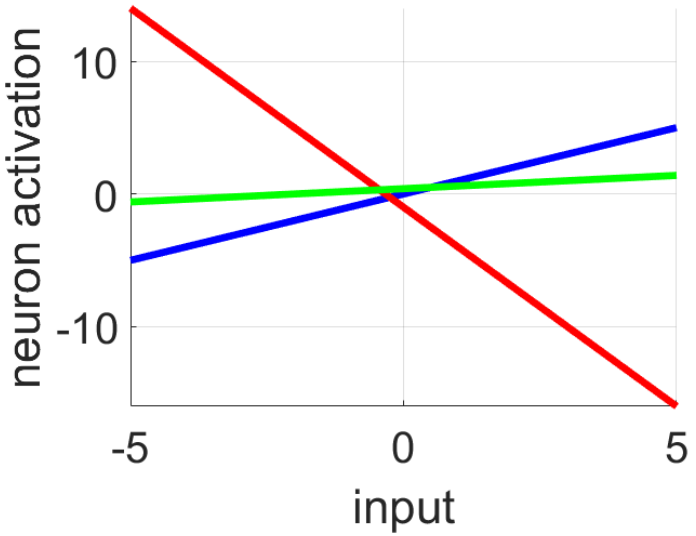
Network Output - Example

w_1 = [1 -3 0.2]
b_1 = [0 -1 0.4]
w_2 = [1 2 0.5]
b_2 = -1.5

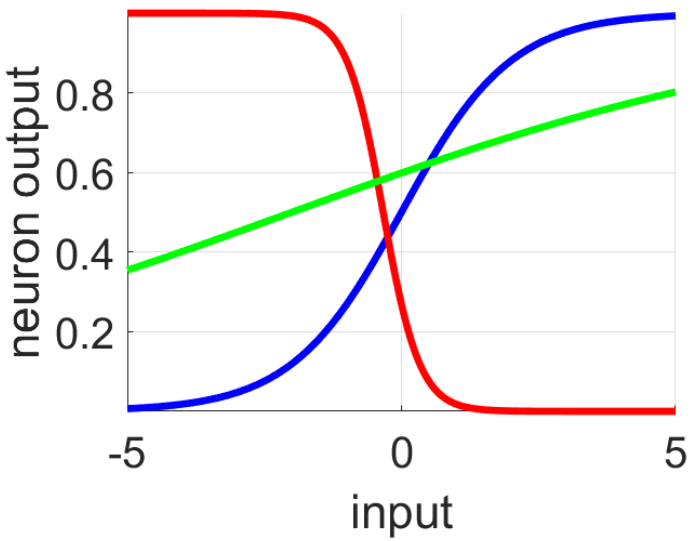


$$z_j = g\left(w_{1,j}^{\top} u + b_{1,j}\right)$$

Activation input weighting

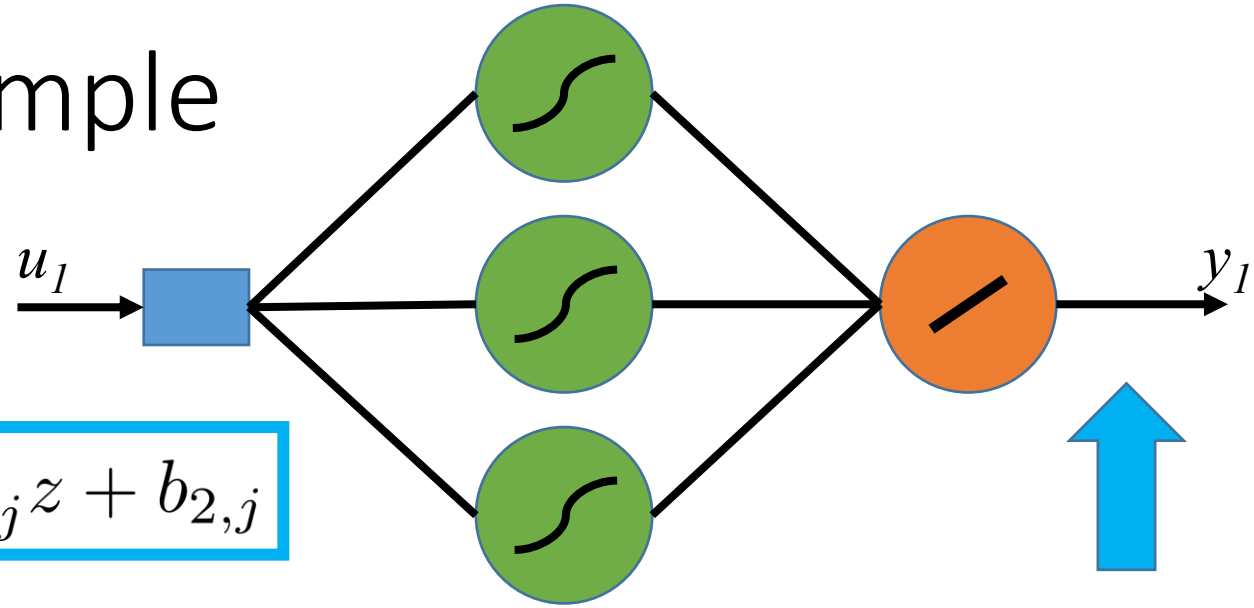


Activation nonlinear transform



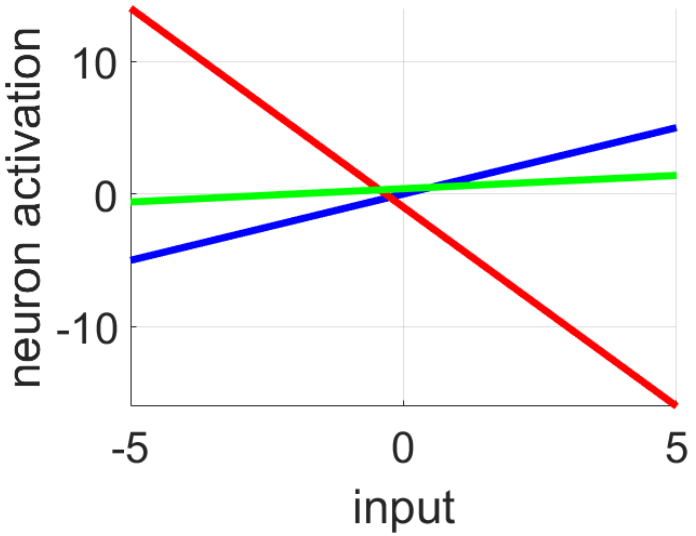
Network Output - Example

w_1 = [1 -3 0.2]
b_1 = [0 -1 0.4]
w_2 = [1 2 0.5]
b_2 = -1.5

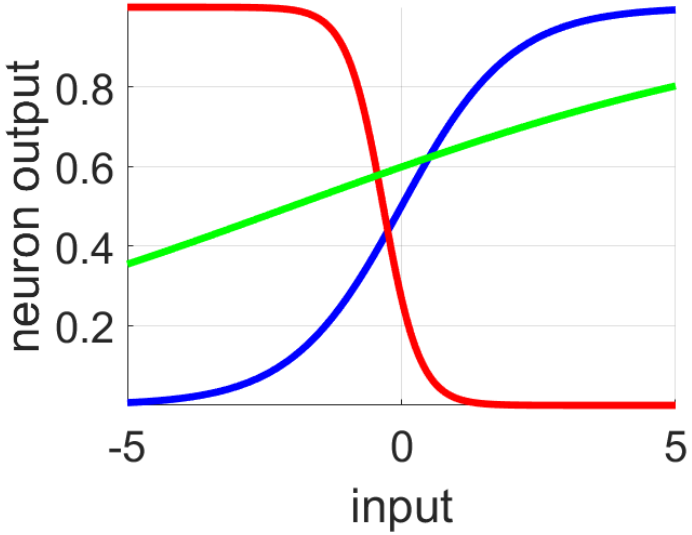


$$y_j = w_{2,j}^{\top} z + b_{2,j}$$

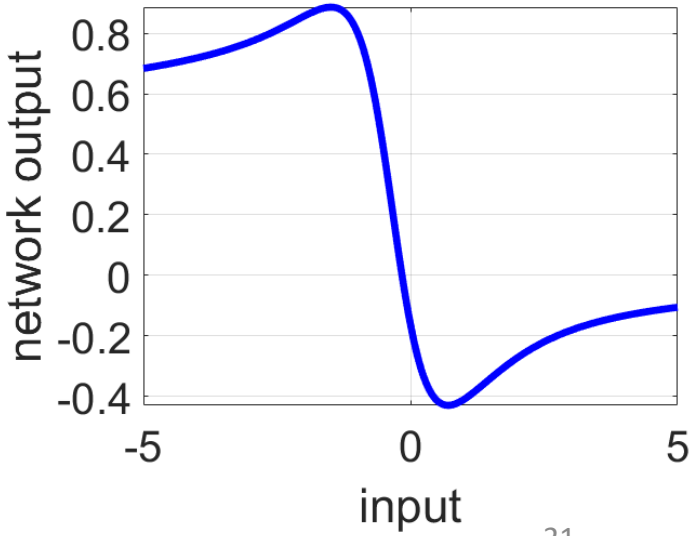
Activation input weighting



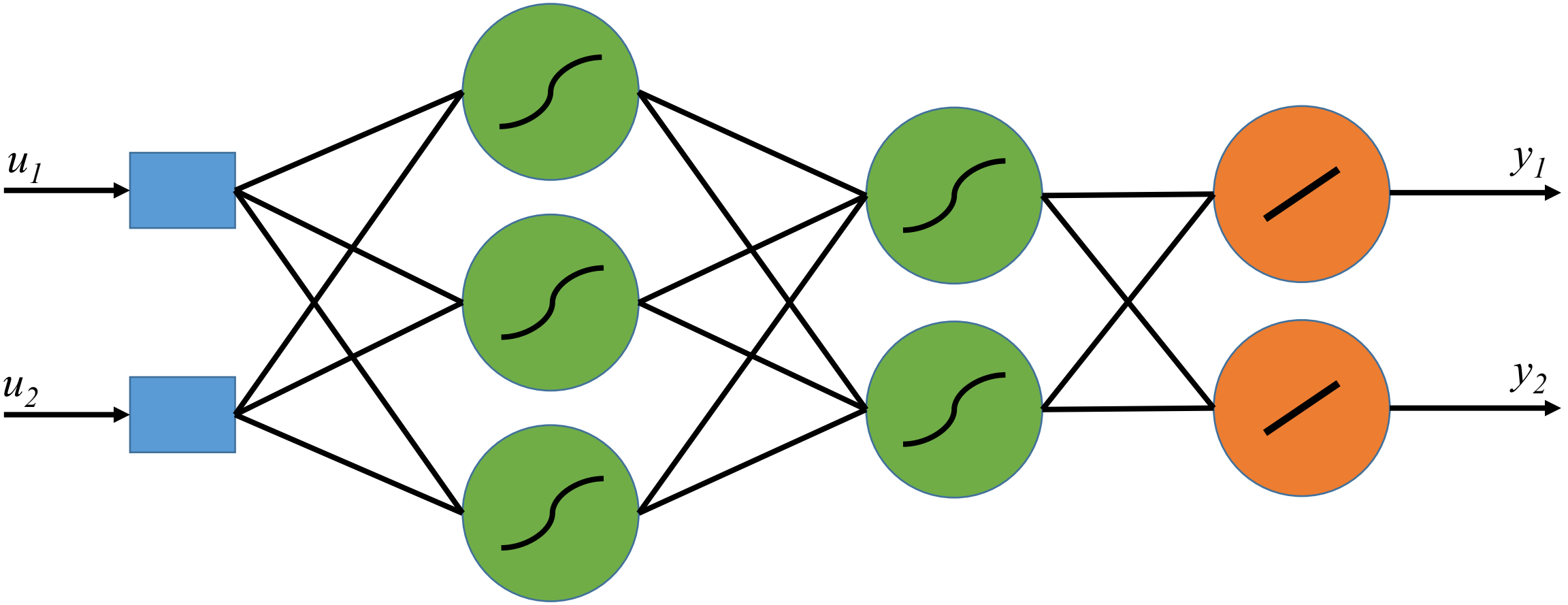
Activation nonlinear transform



Linear output layer



Two Hidden Layer Network



Input Layer

Hidden Layers

Output Layer

Artificial Neural Networks

What is an artificial neural network?

Simple feedforward networks

Approximation properties

Training an artificial neural network

Approximation Properties

[Cybenko, 1989]: A **feedforward sigmoidal neural net** with at least one hidden layer can approximate any continuous nonlinear function $\mathbb{R}^{n_x} \to \mathbb{R}^{n_y}$ arbitrarily well, provided that sufficient number of hidden neurons are available.

Polynomial or Neural Network?

Which one approximates best a function $f : \mathbb{R} \to \mathbb{R}$?

A: Neural Net

B: Polynomial basis functions

Polynomial or Neural Network?

Which one approximates best a function $f : \mathbb{R} \to \mathbb{R}$?

A: Neural Net B: Polynomial basis functions

Which one approximates best a function $f : \mathbb{R}^2 \to \mathbb{R}$?

A: Neural Net B: Polynomial basis functions

Polynomial or Neural Network?

Which one approximates best a function $f : \mathbb{R} \to \mathbb{R}$?

A: Neural Net B: Polynomial basis functions

Which one approximates best a function $f : \mathbb{R}^2 \to \mathbb{R}$?

A: Neural Net B: Polynomial basis functions

Which one approximates best a function $f : \mathbb{R}^4 \to \mathbb{R}$?

A: Neural Net B: Polynomial basis functions

Polynomial or Neural Network?

Which one approximates best a function $f : \mathbb{R} \to \mathbb{R}$?

A: Neural Net B: Polynomial basis functions

B

Which one approximates best a function $f : \mathbb{R}^2 \to \mathbb{R}$?

A: Neural Net B: Polynomial basis functions

A & B

Which one approximates best a function $f : \mathbb{R}^4 \to \mathbb{R}$?

A: Neural Net B: Polynomial basis functions

A

Approximation Properties

[Barron, 1993]: A **feedforward sigmoidal neural net** with one hidden layer can achieve an integrated squared error of the order

$$V = \mathcal{O}\left(\frac{1}{n}\right)$$

independently of the dimension of the input space, where n denotes the number of hidden neurons.

For a **basis function expansion** (e.g. multivariate polynomial) with n terms, in which only the parameters of the linear combination are adjusted the integrated squared error is of the order

$$V = \mathcal{O}\left(\frac{1}{n^{2/n_x}}\right)$$

where n_x denotes the number of input variables.

Example

Function of 2 variables $(n_x = 2)$

ANN: $V = \mathcal{O}\left(\frac{1}{n}\right)$

POL: $V = \mathcal{O}\left(\frac{1}{n}\right)$

Same behavior for both

Function of 10 variables $(n_x = 10)$

ANN: $V = \mathcal{O}\left(\frac{1}{n}\right)$

POL: $V = \mathcal{O}\left(\frac{1}{n^{2/10}}\right)$

Approximation error decreases much more rapidly for neural networks

Example

Approximation error decreases much more rapidly for neural networks

Function of 2 variables $(n_x = 2)$

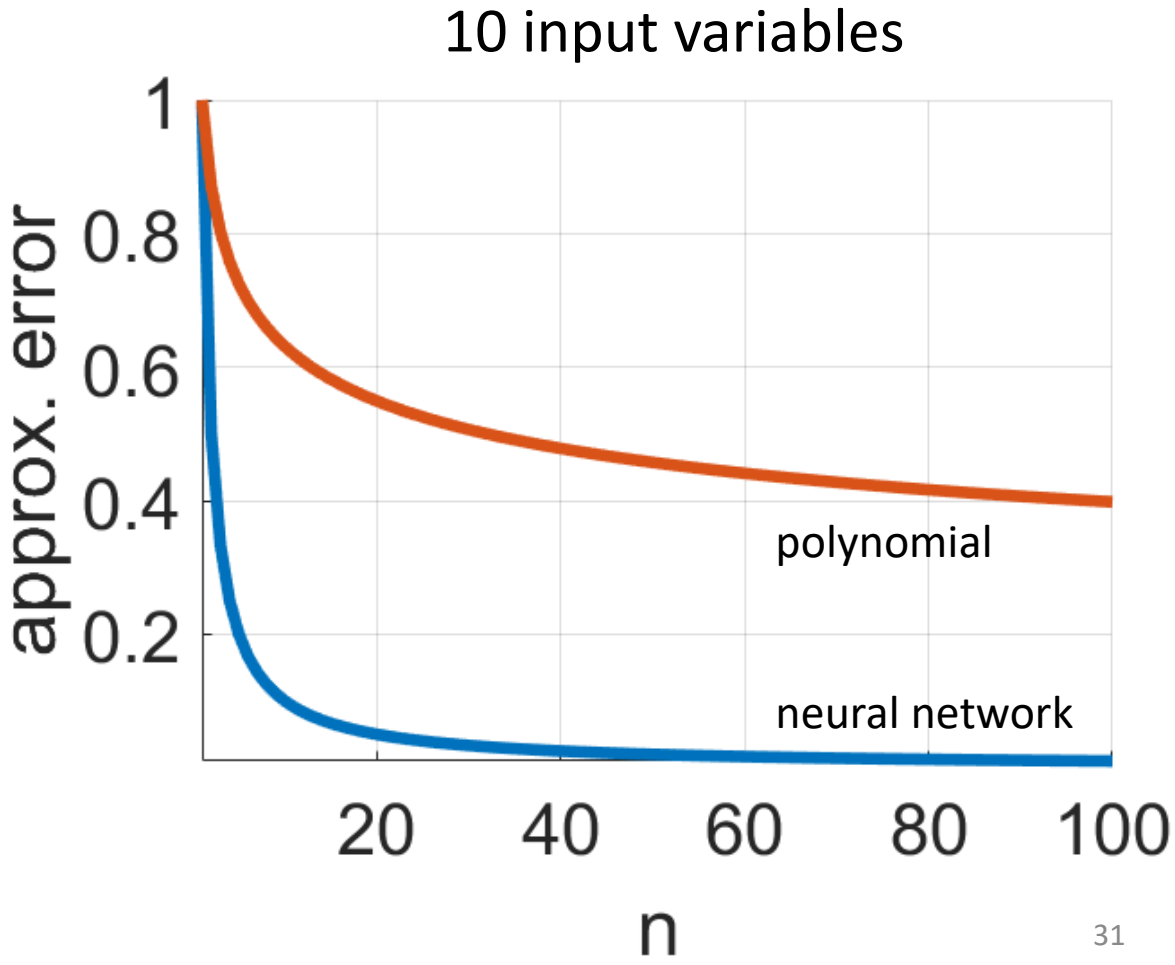
ANN: $V = \mathcal{O}\left(\frac{1}{n}\right)$

POL: $V = \mathcal{O}\left(\frac{1}{n}\right)$

Function of 10 variables $(n_x = 10)$

ANN: $V = \mathcal{O}\left(\frac{1}{n}\right)$

POL: $V = \mathcal{O}\left(\frac{1}{n^{2/10}}\right)$



Interpretation

Neural networks are well-suited for high-dimensional problems

Does not mean 1-hidden layer is always the best choice

Does not mean the error will always improve by adding more neurons

Does not mean a neural network is always better than basis function expansion

Artificial Neural Networks

What is an artificial neural network?

Simple feedforward networks

Approximation properties

Training an artificial neural network

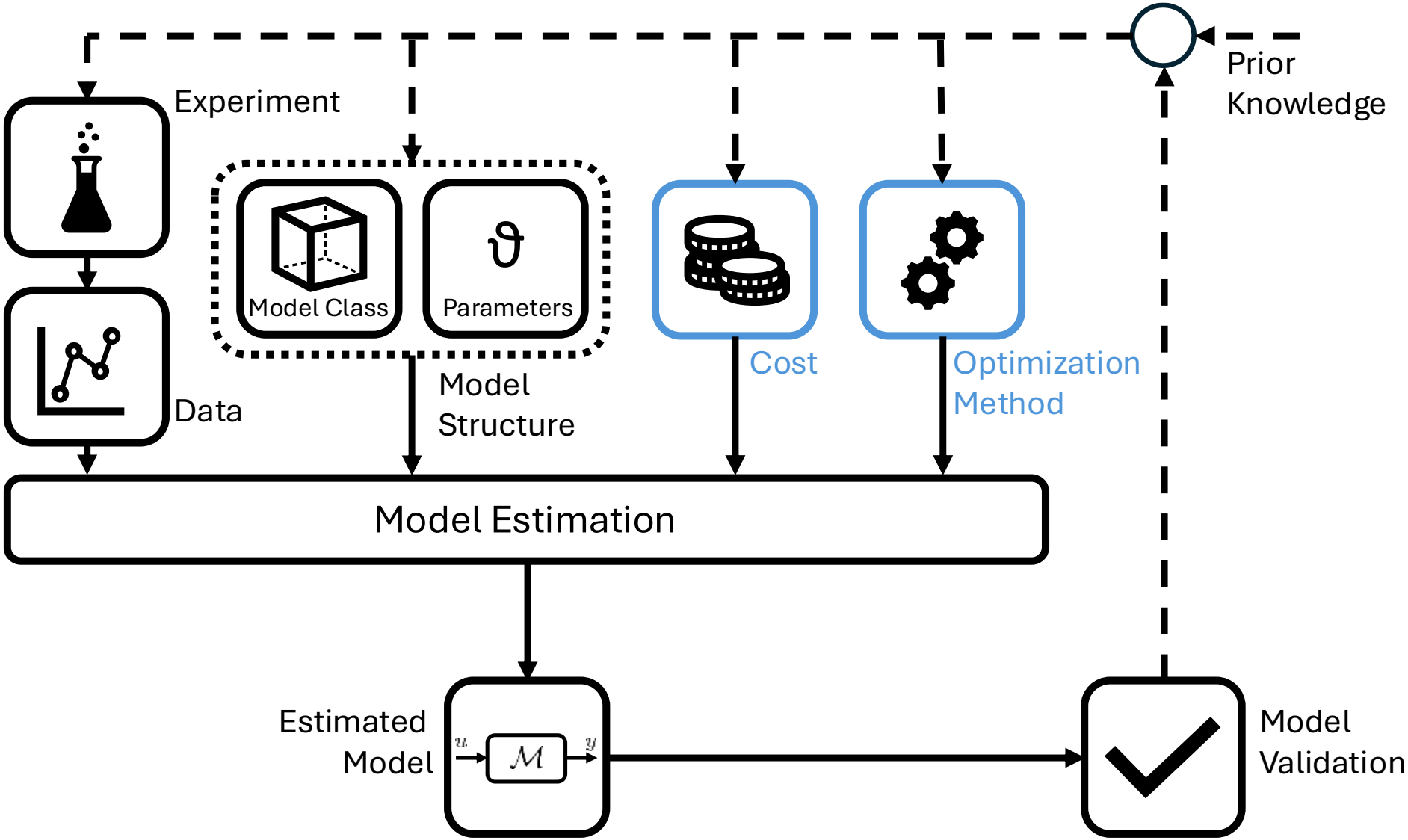
Training A Neural Network

Cost Function

Gradient Based Methods

Backpropagation

Data-Driven Modelling Process



Model Structure

Decisions to make:

network structure

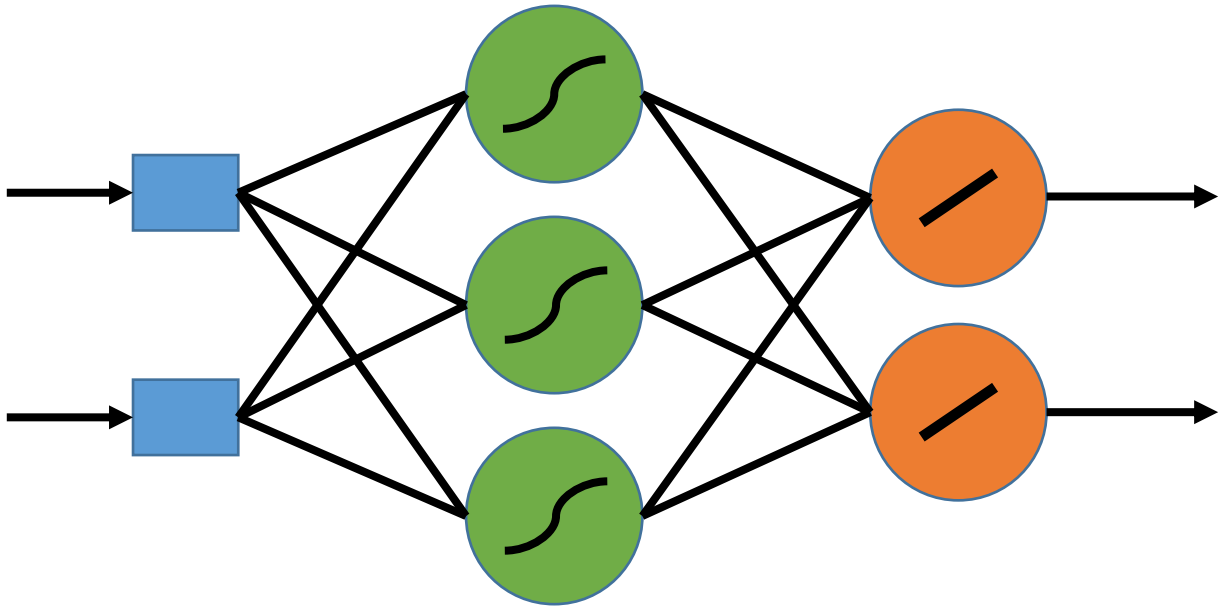
hidden layers

neurons

activation function

Parameters to estimate:

weights and biases

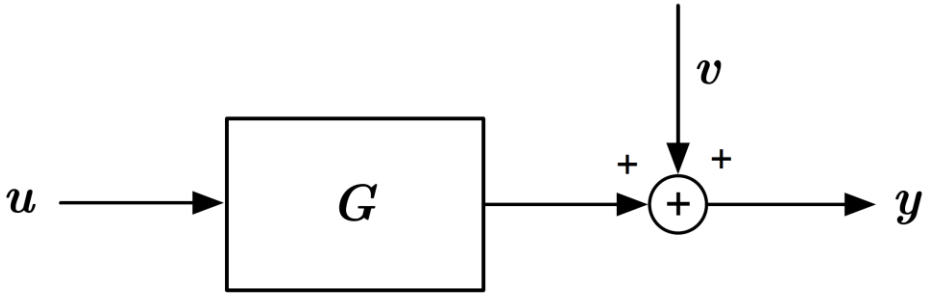


Cost Function

The cost function describes the objective we want to minimize or maximize

Describes how well the model matches the data

Can be extended with extra penalty terms (regularization)

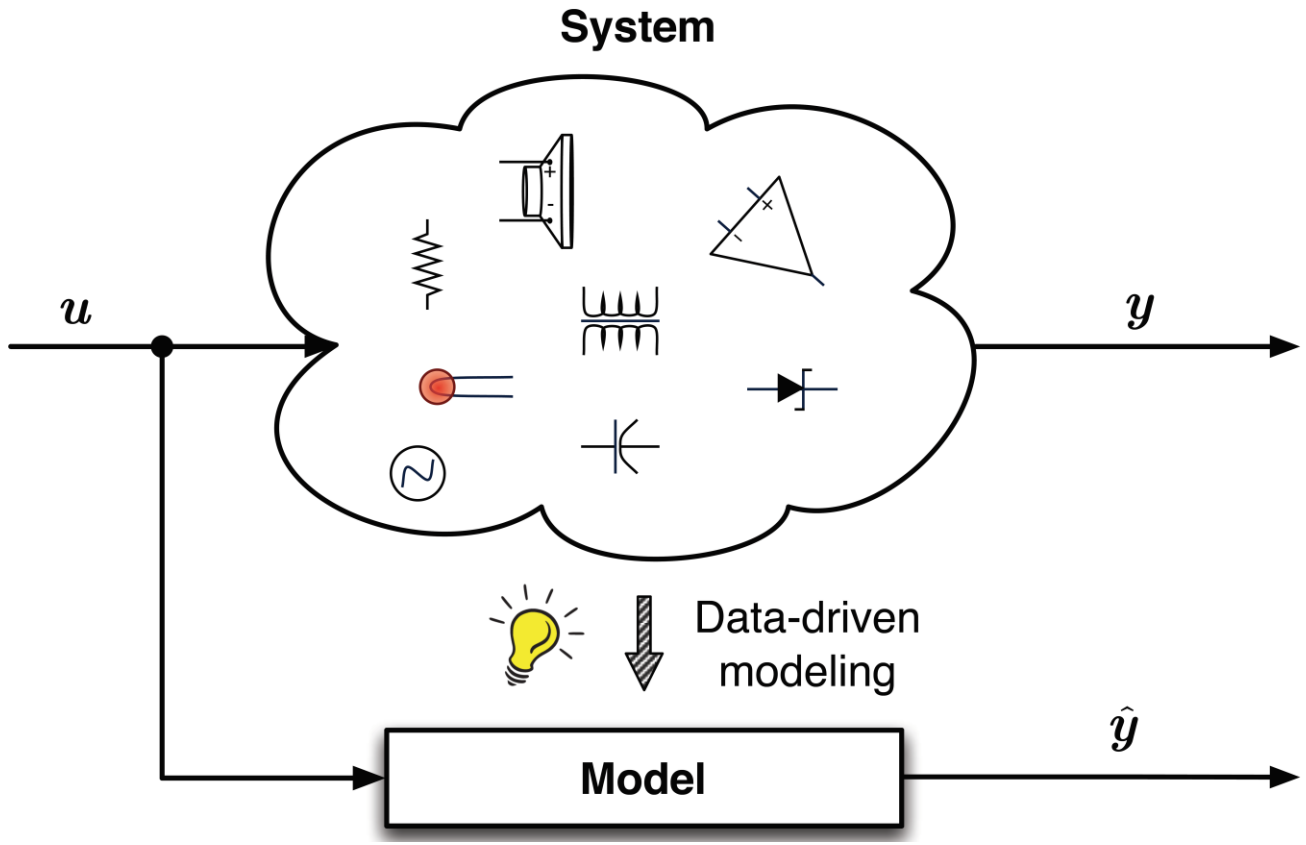


Cost Function
Objective Function
Loss Function

Cost Function

Squared l^2 Norm

$$V_N(\theta) = \frac{1}{N} \sum_{k=0}^{N-1} (y_k - \hat{y}_k)^2$$



Supervised learning is considered here (dataset with labeled input and output)

Estimator

$$\hat{\theta}_N = \arg\min_{\theta \in \Theta} \frac{1}{N} \sum_{k=0}^{N-1} (y_k - \hat{y}_k)^2$$

$$= \arg\min_{\theta \in \Theta} V_N(\theta)$$

$\hat{y}_k$ output of the artificial neural network

θ stacked parameter vector containing all the network weights and biases

How to minimize?

Nonlinear Optimization

$$\hat{\theta}_N = \arg\min_{\theta\in\Theta} \frac{1}{N} \sum_{k=0}^{N-1} (y_k - \hat{y}_k)^2$$
$$= \arg\min_{\theta\in\Theta} V_N(\theta)$$

Nonlinear in the parameters

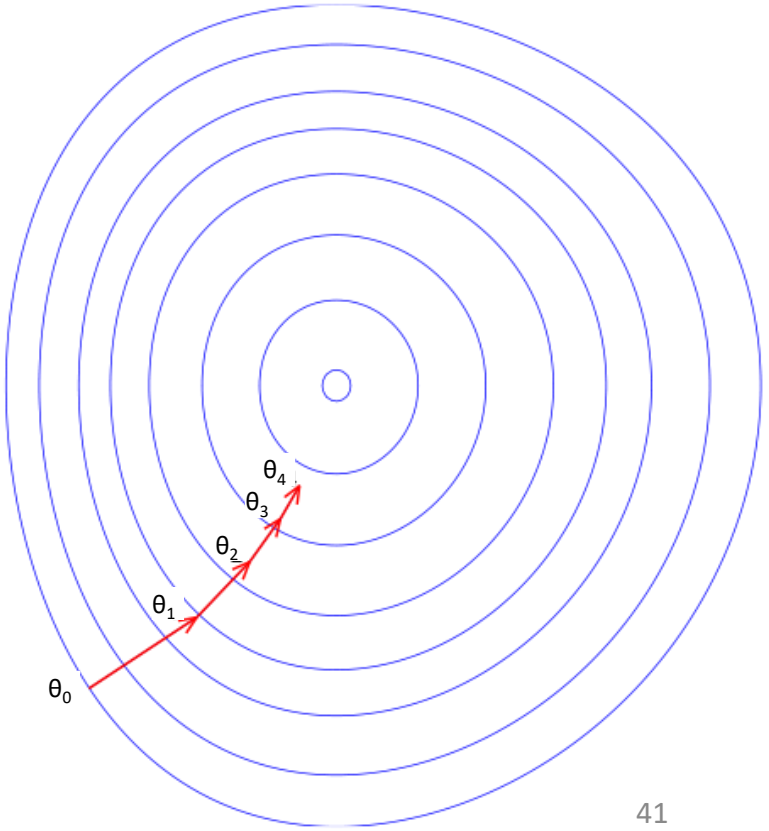
Differentiable

Steepest Descent

$$\hat{\theta}_N = \arg\min_{\theta\in\Theta} \frac{1}{N}\sum_{k=0}^{N-1}(y_k - \hat{y}_k)^2$$

$$= \arg\min_{\theta\in\Theta} V_N(\theta)$$

1. Make an initial guess
2. Compute the gradients $\nabla_{\theta_i} V_N(\theta)$
3. Parameter update $\theta_{i+1} = \theta_i - \epsilon\nabla_{\theta_i} V_N(\theta)$
4. Check stopping criterion
5. Stop



Steepest Descent

Initial guess?	(small random values)
Step size?	(line search, adaptive rules, second-order methods, …)
Stopping Criterium?	(gradient threshold, cost threshold, early stopping, …)
Gradient Calculation?	(backpropagation)

Gradient Calculation

Automatic gradient calculation for

wide range of model structures

large datasets

→ Flexible and efficient algorithm required

Automatic Differentiation

Functions are composed of

Elementary operations: + - / x …

Elementary functions: sin, log, exp, …

Automatic differentiation exploits the function structure and evaluates the derivative for a given set of function values and parameters

≠ symbolic differentiation

≠ numerical differentiation

Forward vs Reverse Automatic Differentiation

Forward Mode

1. Fix the free variable you want to know the derivative of
2. Perform chain rule
3. Repeat for all free variables

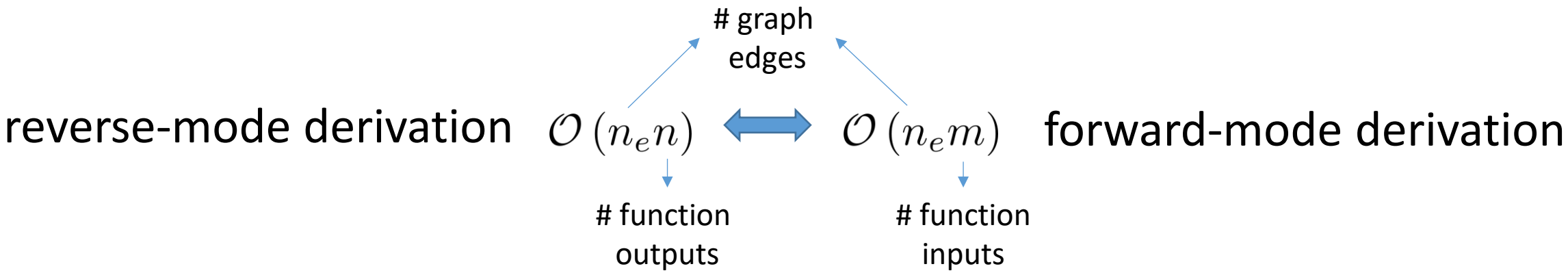
Good for functions with little free variables and many outputs

Reverse Mode

1. Select the function output wrt which you will calculate the derivative
2. Calculate all chain rule elements towards all free variables
3. Repeat for all function outputs

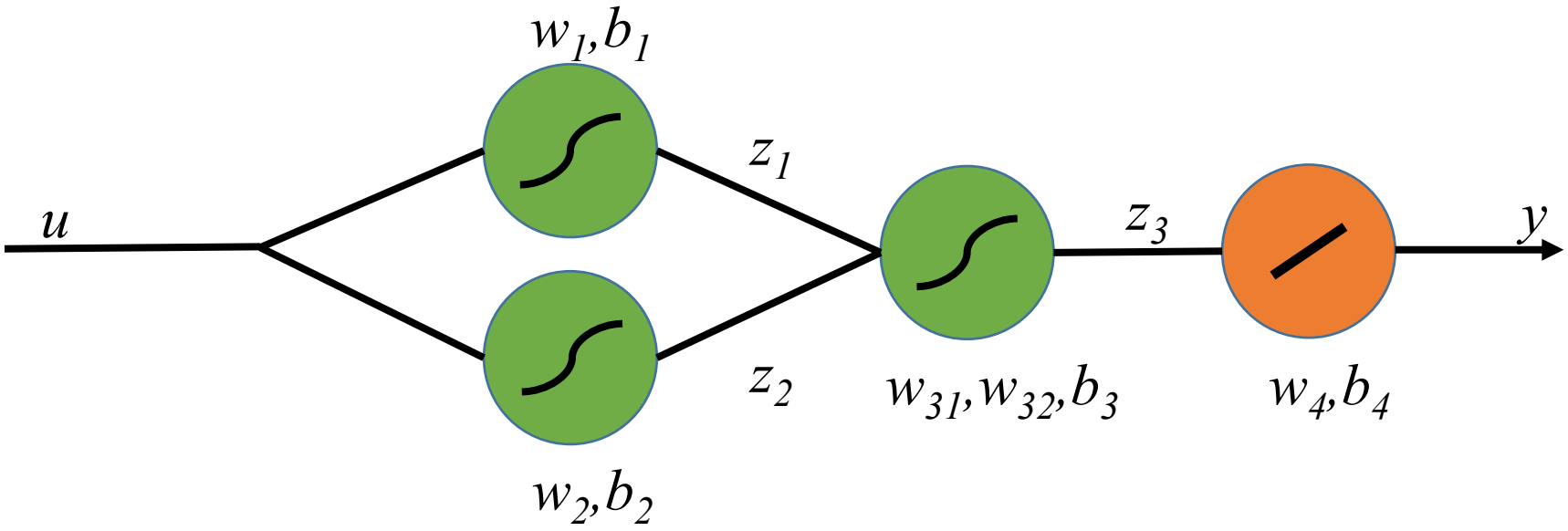
Good for functions with many free variables and little outputs

Backpropagation: Computational Efficiency



Function: cost function
Function inputs: parameters → Many!
Function outputs: cost → Few!
Graph edges: ≈ANN links

Example



2-Hidden Layer NN

1 input, 1 output

9 parameters

Sigmoid Activation

$$g(x) = \frac{1}{1 + e^{-x}}$$

Example

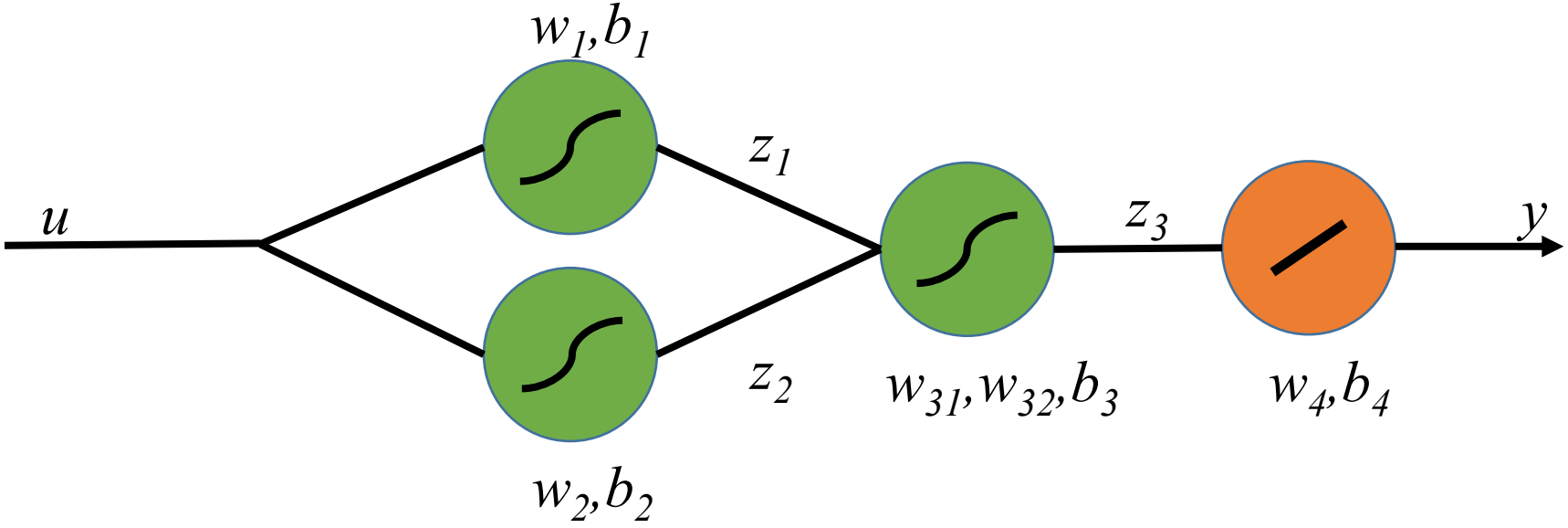
2-Hidden Layer NN

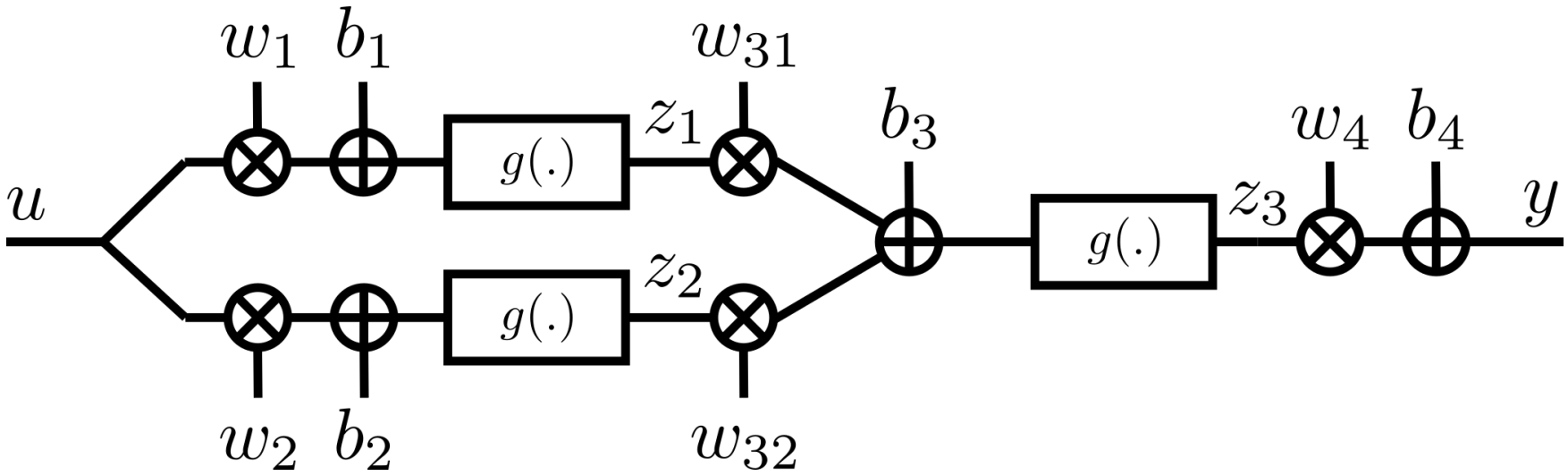
1 input, 1 output

9 parameters

Sigmoid Activation

$$g(x) = \frac{1}{1 + e^{-x}}$$





Backpropagation

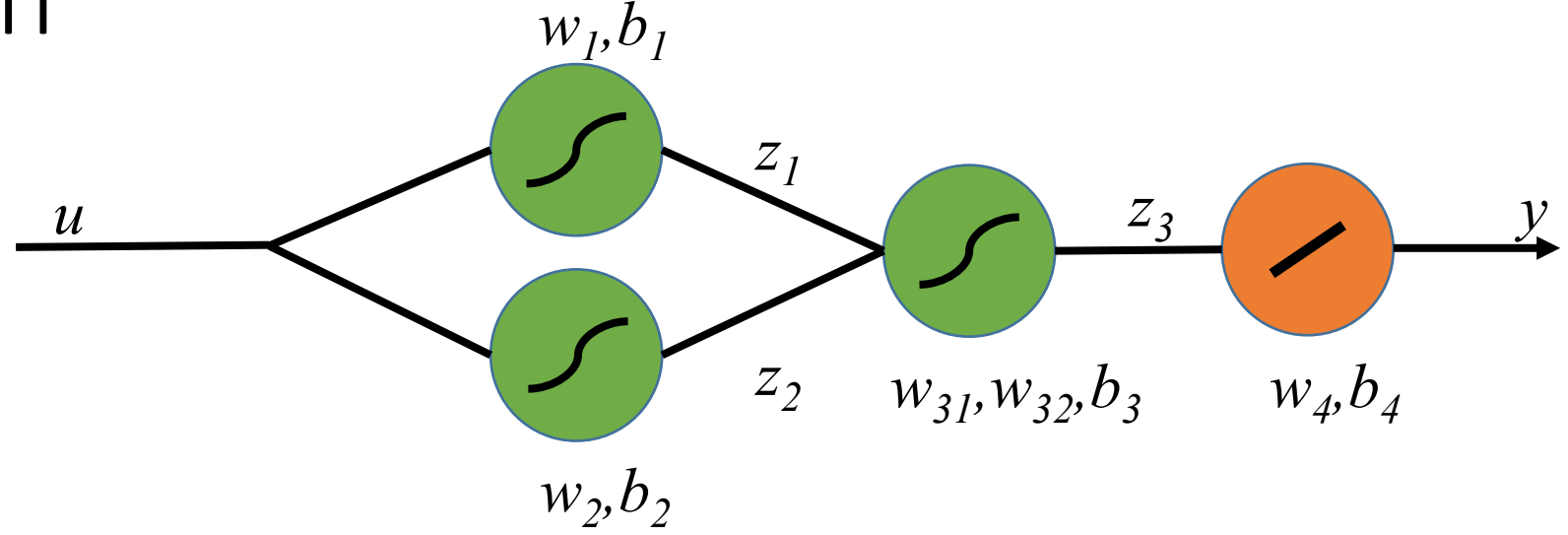
Automatic differentiation

Reverse mode algorithm

- Forward pass
- Backward pass
- Applying chain rule
- Reuse of computations

Backpropagation

$$\hat{\theta}_N = \arg\min_{\theta\in\Theta} \frac{1}{N}\sum_{k=0}^{N-1}(y_k - \hat{y}_k)^2$$
$$= \arg\min_{\theta\in\Theta} V_N(\theta)$$

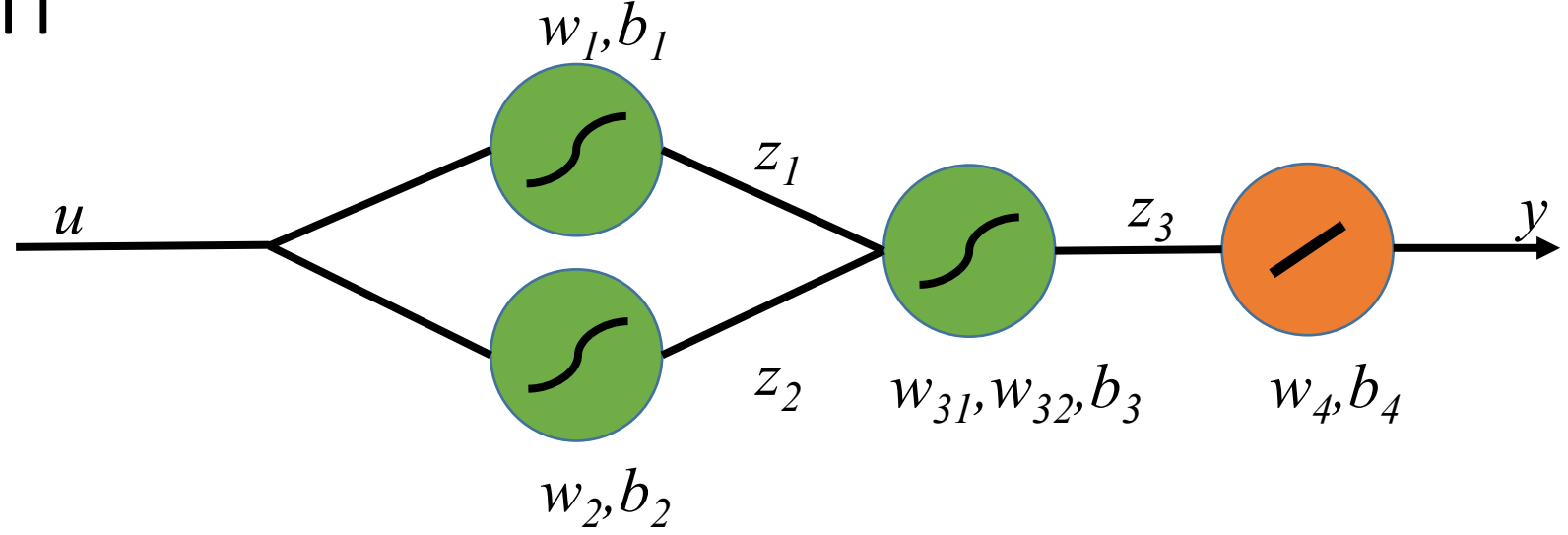


Gradients:

$$\nabla_{\theta_i} V_N(\theta) = \left(\frac{\partial V_N(\theta)}{\partial w_1}\Big|_{\theta_i}, \frac{\partial V_N(\theta)}{\partial w_2}\Big|_{\theta_i}, \frac{\partial V_N(\theta)}{\partial w_{31}}\Big|_{\theta_i}, \frac{\partial V_N(\theta)}{\partial w_{32}}\Big|_{\theta_i}, \frac{\partial V_N(\theta)}{\partial w_4}\Big|_{\theta_i}, \frac{\partial V_N(\theta)}{\partial b_1}\Big|_{\theta_i}, \frac{\partial V_N(\theta)}{\partial b_2}\Big|_{\theta_i}, \frac{\partial V_N(\theta)}{\partial b_3}\Big|_{\theta_i}, \frac{\partial V_N(\theta)}{\partial b_4}\Big|_{\theta_i} \right)$$

Backpropagation

$$\hat{\theta}_N = \arg\min_{\theta\in\Theta} \frac{1}{N}\sum_{k=0}^{N-1}(y_k - \hat{y}_k)^2$$
$$= \arg\min_{\theta\in\Theta} V_N(\theta)$$



$$\left.\frac{\partial V_N(\theta)}{\partial\theta}\right|_{\theta_i} = -\frac{2}{N}\sum_{k=1}^{N}(y_k - \hat{y}_k)\left.\frac{\partial\hat{y}_k}{\partial\theta}\right|_{\theta_i}$$

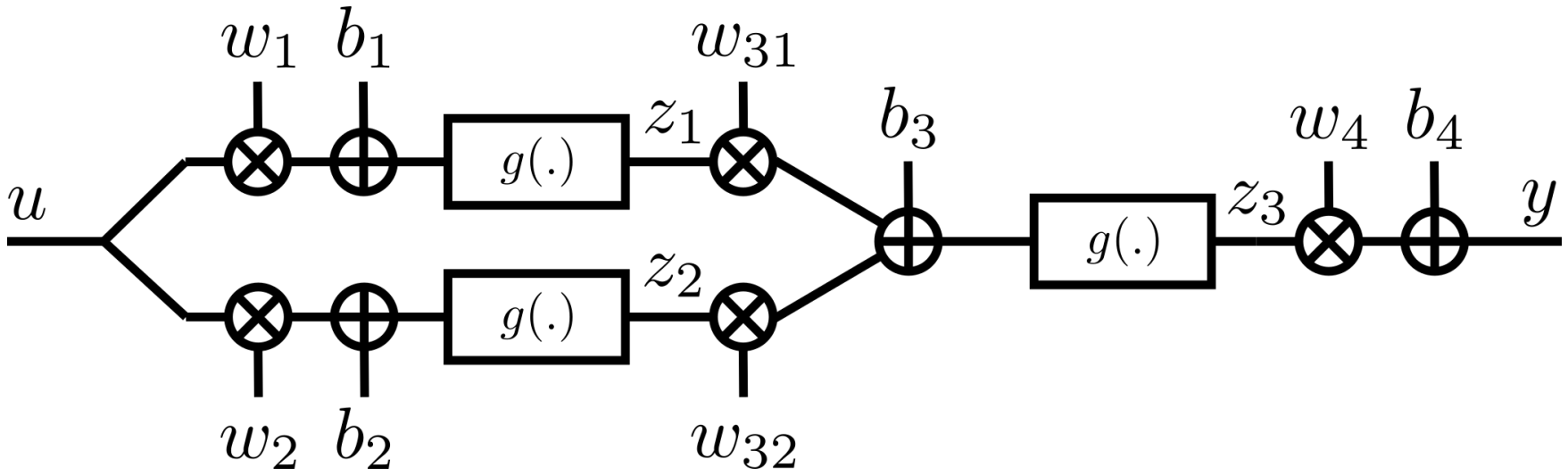
Focus on the partial derivatives of the outputs for simplicity

The notation for evaluating the partial derivative in the i-th iteration of the parameters is dropped from now on
The notation for time-indexing is also dropped

Backpropagation – Fill in Values

Parameter Values for Iteration i

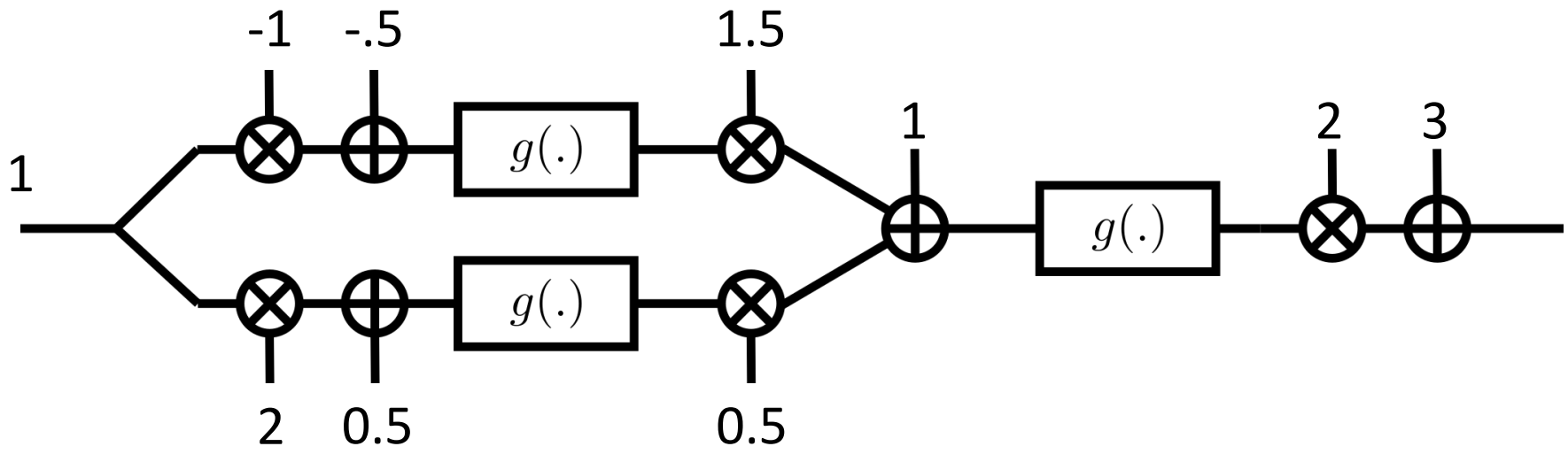
Input-Output Values for time k



Backpropagation – Fill in Values

Parameter Values for Iteration i

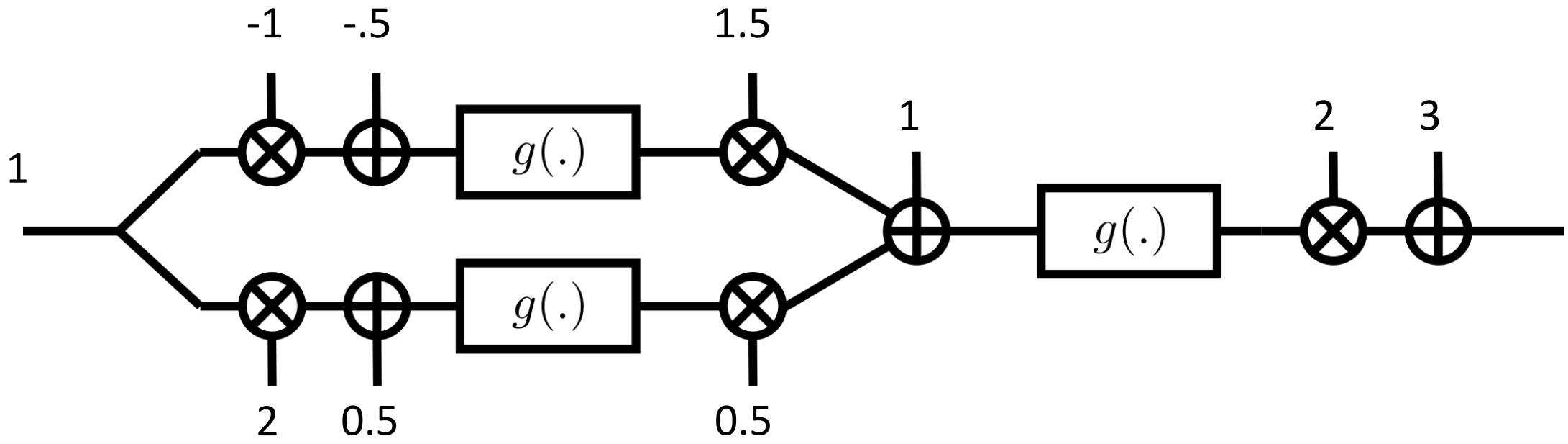
Input-Output Values for time k



Backpropagation – Forward Pass

Propagate the values forward through the network

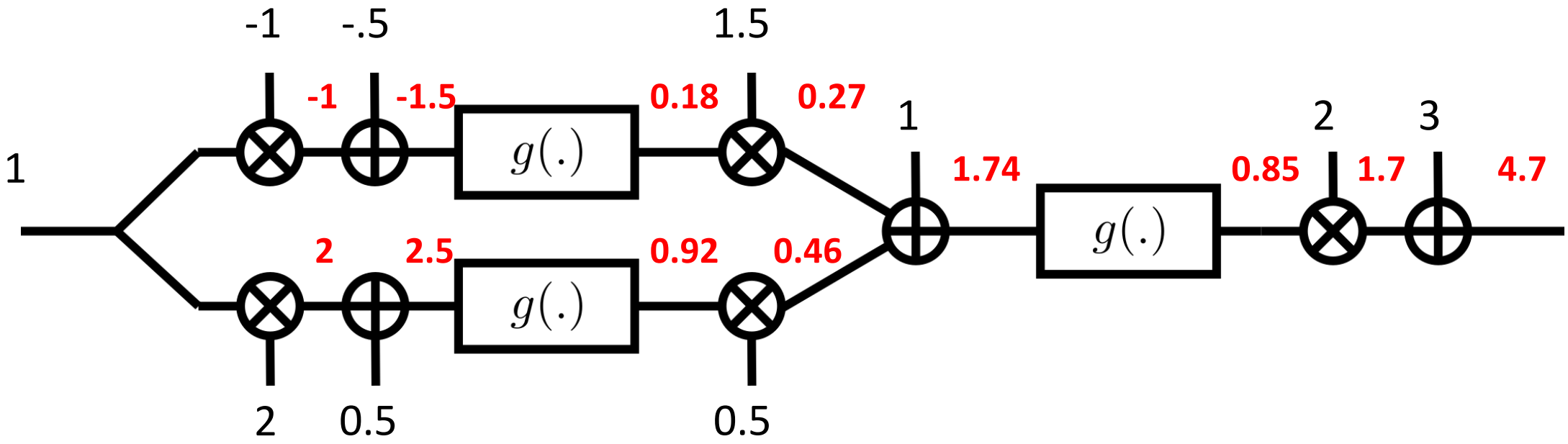
$$g(x) = \frac{1}{1 + e^{-x}}$$



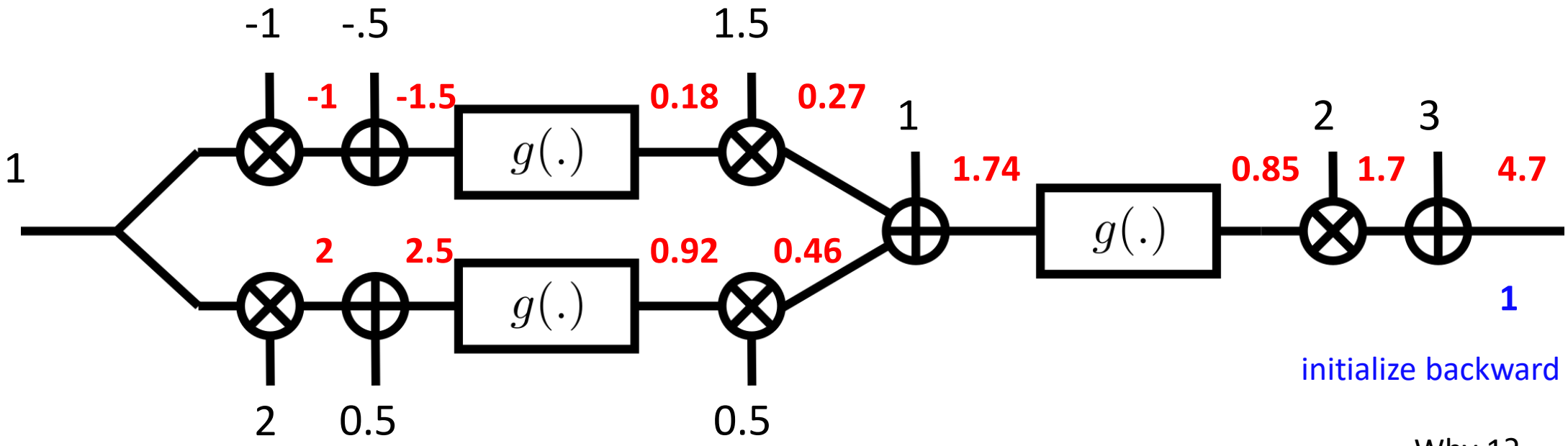
Backpropagation – Forward Pass

Propagate the values forward through the network

$$g(x) = \frac{1}{1 + e^{-x}}$$



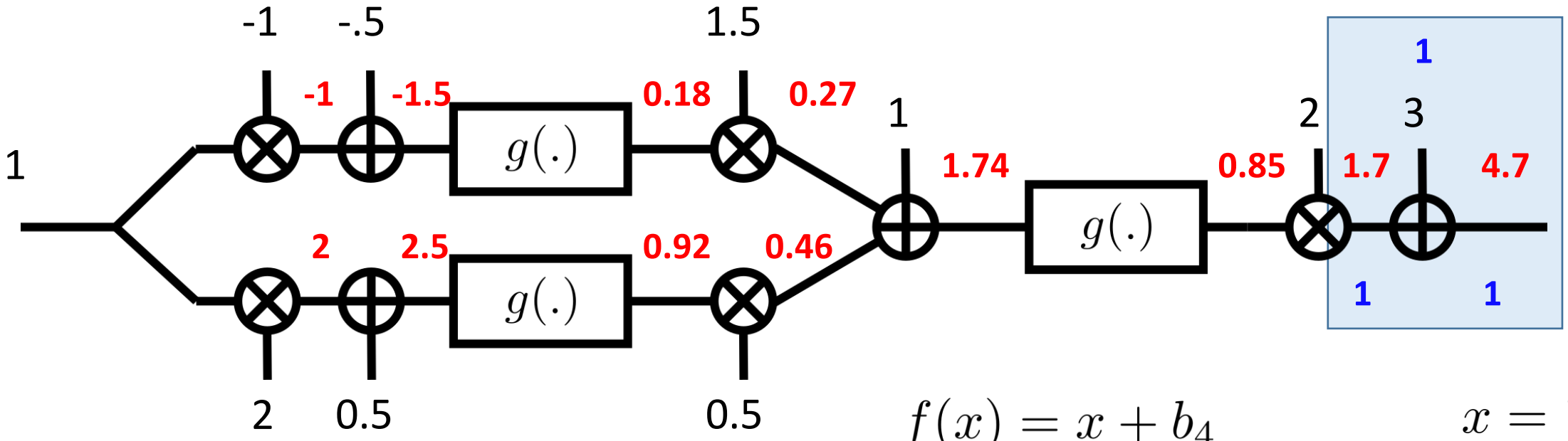
Backpropagation – Backward Pass



initialize backward pass

Why 1?

Backpropagation – Backward Pass



$$\frac{\partial y}{\partial x} = \frac{\partial y}{\partial f(x)} \frac{\partial f(x)}{\partial x}$$

↳ Downstream partial derivative (previously computed)

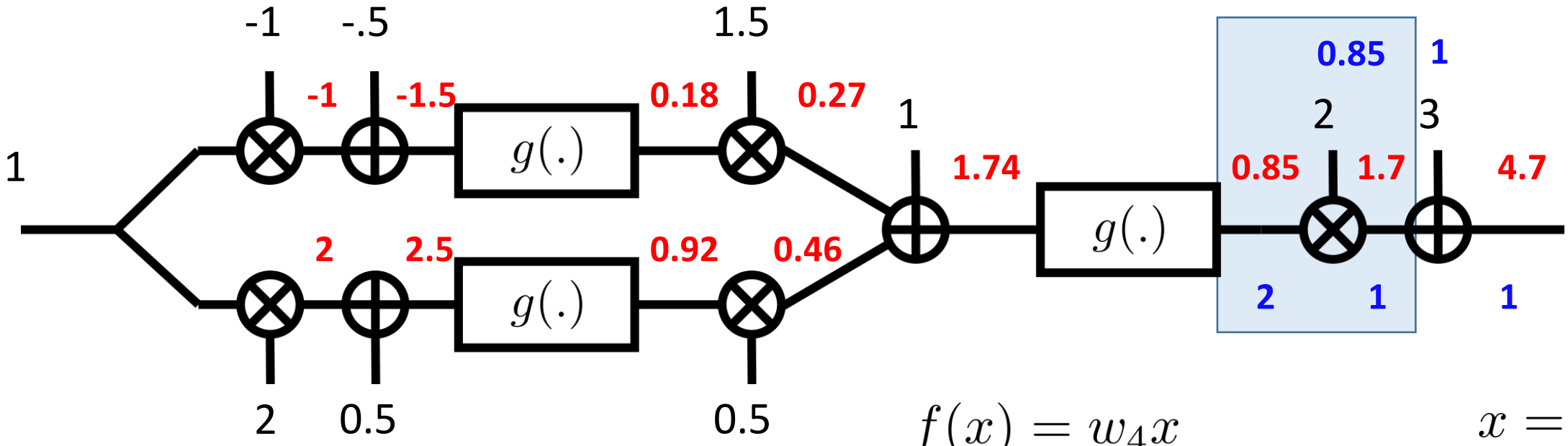
$$f(x) = x + b_4$$

$$\frac{\partial f}{\partial x} = 1, \quad \frac{\partial f}{\partial b_4} = 1$$

$$x = 1.7$$

$$b_4 = 3$$

Backpropagation – Backward Pass



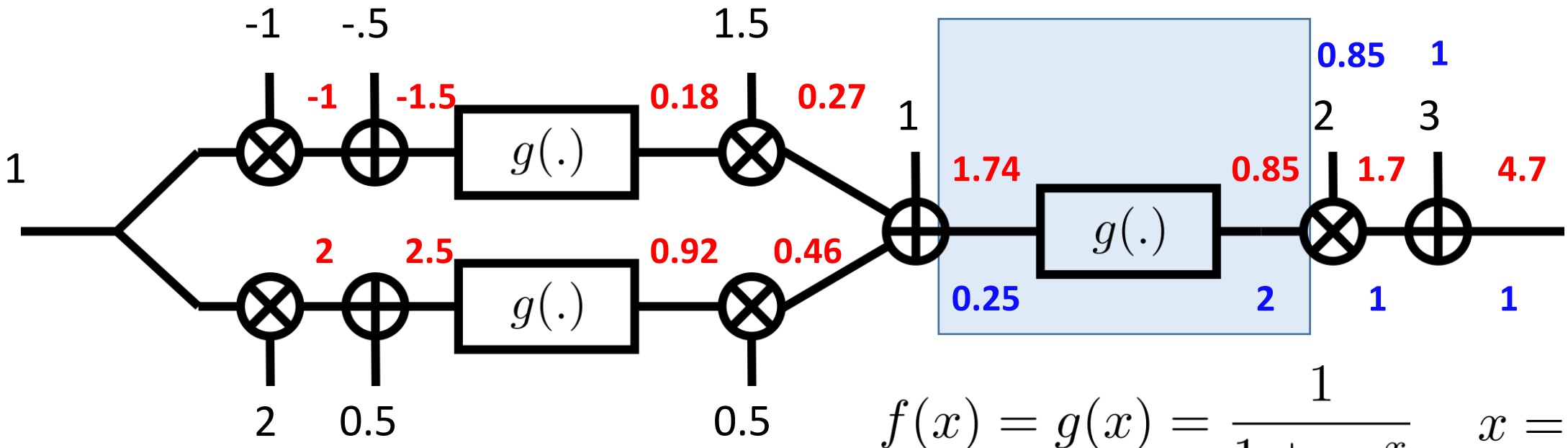
$$\frac{\partial y}{\partial x} = \frac{\partial y}{\partial f(x)} \frac{\partial f(x)}{\partial x}$$

↳ Downstream partial derivative (previously computed)

$$f(x) = w_4 x \qquad x = 0.85$$

$$\frac{\partial f}{\partial x} = w_4, \quad \frac{\partial f}{\partial w_4} = x \qquad w_4 = 2$$

Backpropagation – Backward Pass



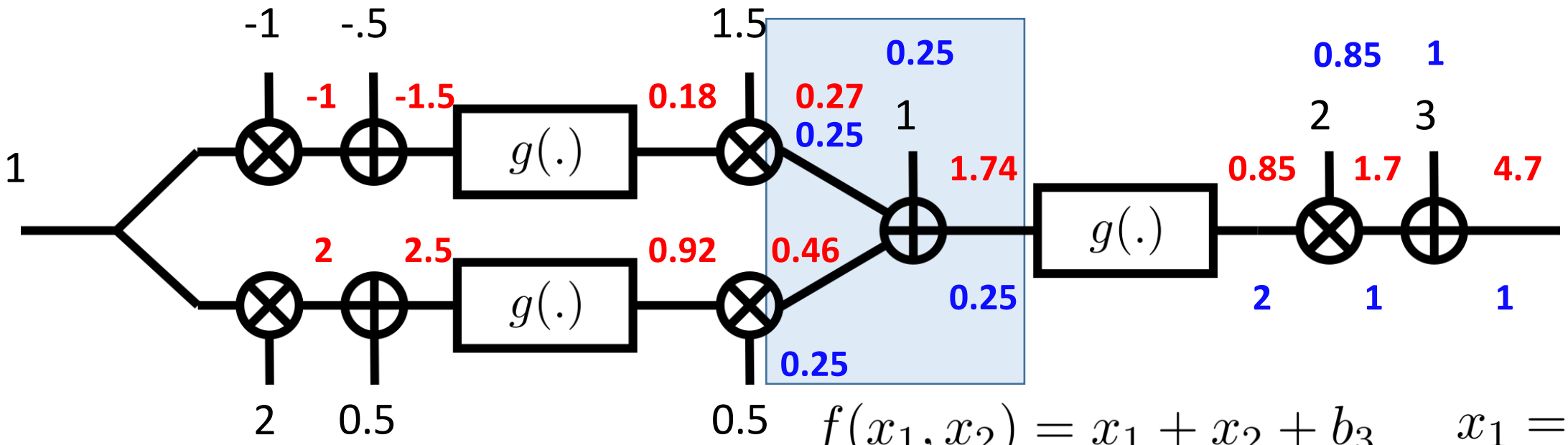
$$f(x) = g(x) = \frac{1}{1+e^{-x}} \qquad x = 1.74$$

$$\frac{\partial y}{\partial x} = \frac{\partial y}{\partial f(x)} \frac{\partial f(x)}{\partial x}$$

$$\frac{\partial f(x)}{\partial x} = \frac{e^x}{(1+e^x)^2} = 0.127$$

↳ Downstream partial derivative (previously computed)

Backpropagation – Backward Pass



$$f(x_1, x_2) = x_1 + x_2 + b_3$$

$$x_1 = 0.27$$

$$x_2 = 0.46$$

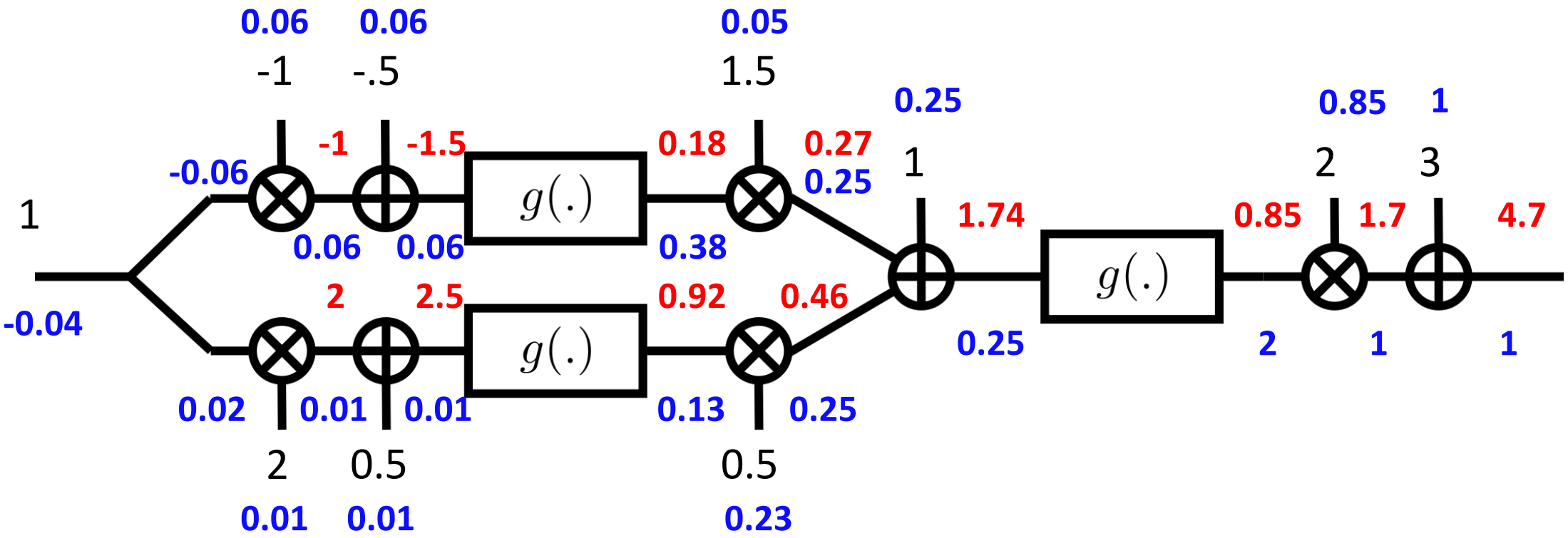
$$b_3 = 1$$

$$\frac{\partial y}{\partial x} = \frac{\partial y}{\partial f(x)} \frac{\partial f(x)}{\partial x}$$

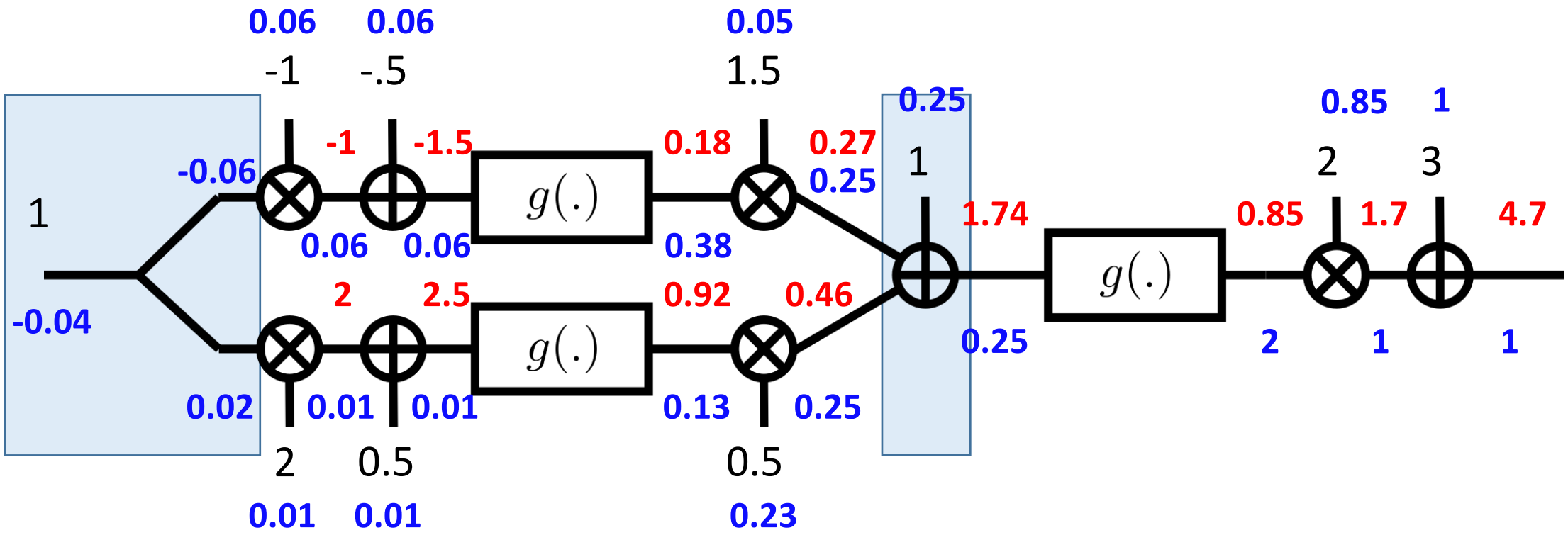
$$\frac{\partial f}{\partial x_1} = 1, \quad \frac{\partial f}{\partial x_2} = 1, \quad \frac{\partial f}{\partial b_3} = 1$$

↳ Downstream partial derivative (previously computed)

Backpropagation – Backward Pass

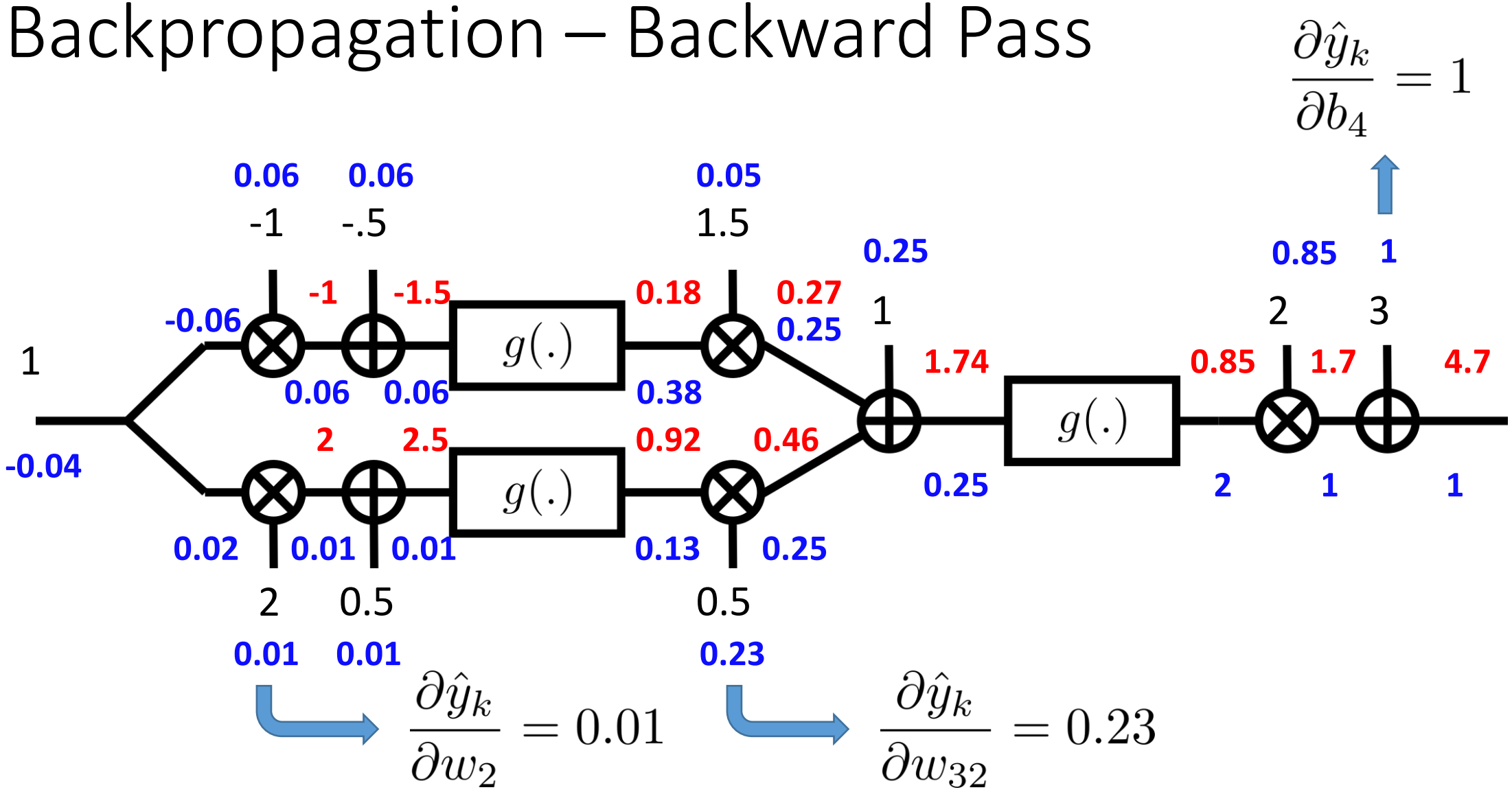


Backpropagation – Backward Pass



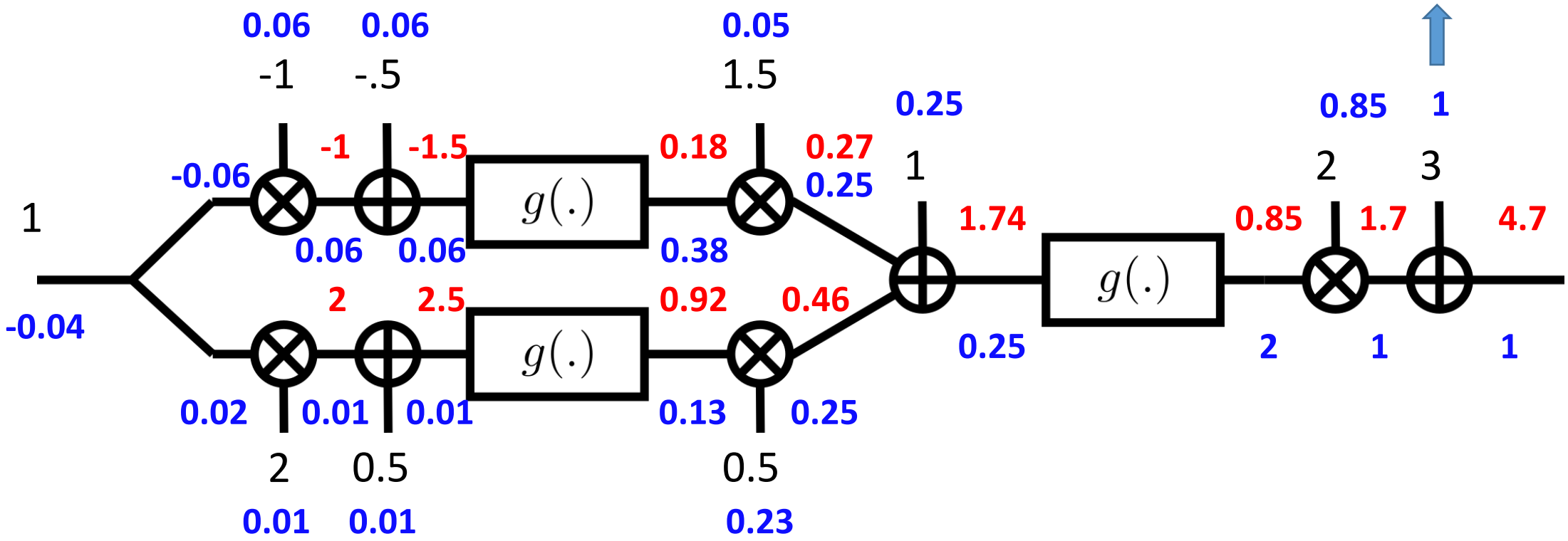
Derivative of a sum = sum of derivatives

Backpropagation – Backward Pass



Backpropagation – Backward Pass

$$\frac{\partial \hat{y}_k}{\partial b_4} = 1$$



$$\left.\frac{\partial V_N(\theta)}{\partial \theta}\right|_{\theta_i} = -\frac{2}{N}\sum_{k=1}^{N}(y_k - \hat{y}_k)\left.\frac{\partial \hat{y}_k}{\partial \theta}\right|_{\theta_i}$$

Do this for all time-instances k

The cost equation should also be included in the backpropagation

Backpropagation

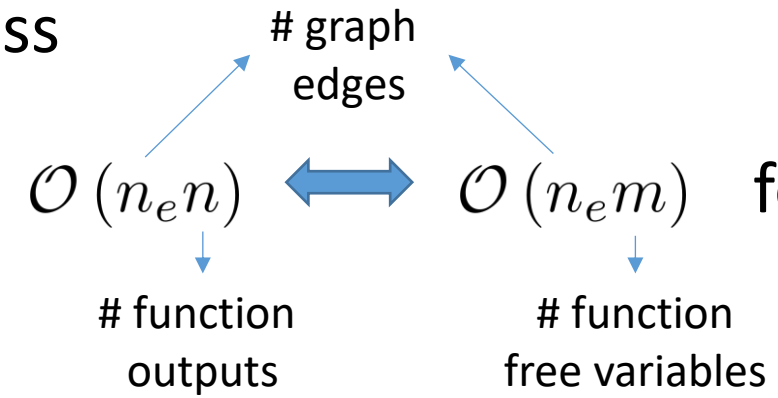
Automatic differentiation

Small sub-functions & Applying chain rule

Forward and backward pass

Computational efficiency

Requires much storage



forward-mode derivation

Learning Outcomes

What is an artificial neural network?

Why are artificial neural networks interesting for function approximation?

How to train a neural network? / What is backpropagation?

Artificial Neural Networks – What Else?

Training, Validation and Test (or Training a Neural Network Cont'd)

Simple Neural Networks for Modelling Dynamical Systems

Artificial Neural Networks and Gaussian Processes

Learning Outcomes

Overfitting and how to prevent it.

How to use artificial neural networks for dynamic models?

Artificial Neural Networks

Overfitting and Regularization (or Training a Neural Network Cont'd)

Simple Neural Networks to Model Dynamics

Artificial Neural Networks

Overfitting and Regularization (or Training a Neural Network Cont'd)

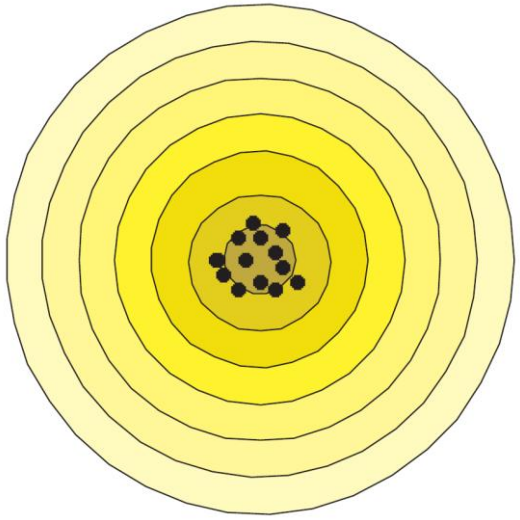
Simple Neural Networks to Model Dynamics

Bias – Variance Tradeoff

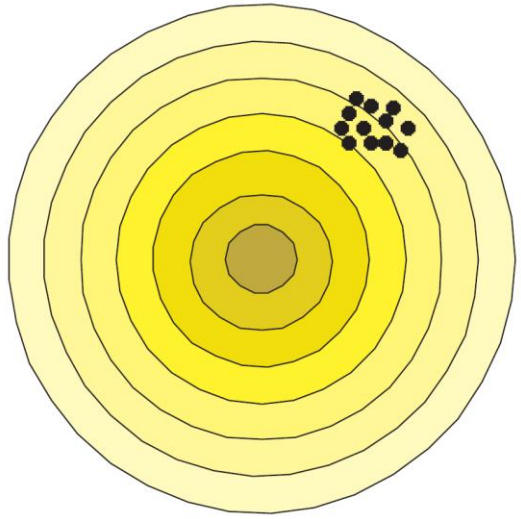
An **estimator** $\hat{\theta}_N$ of θ_o is a mapping from the (measured) data $\mathcal{D}_N$ to $\hat{\theta}_N$. If the data contains random variables, then $\hat{\theta}_N$ is a random variable.

Bias – Variance Tradeoff

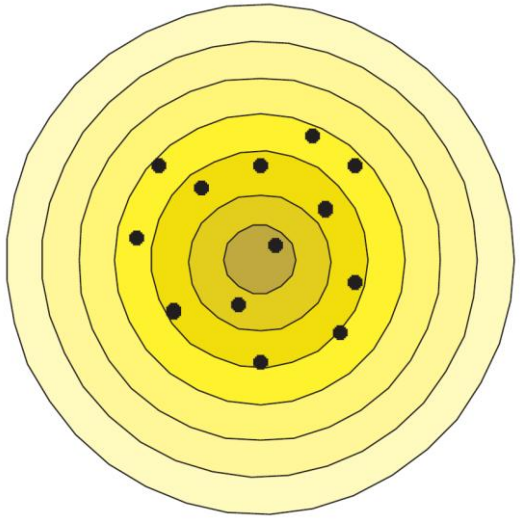
An **estimator** $\hat{\theta}_N$ of θ_o is a mapping from the (measured) data $\mathcal{D}_N$ to $\hat{\theta}_N$. If the data contains random variables, then $\hat{\theta}_N$ is a random variable.



Unbiased
Small variance



Biased
Small variance

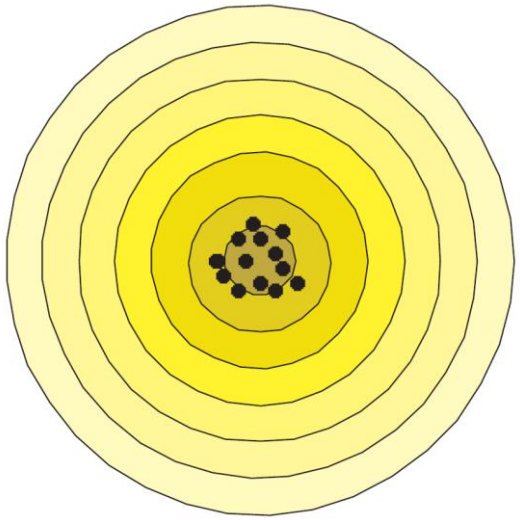


Unbiased
Large variance

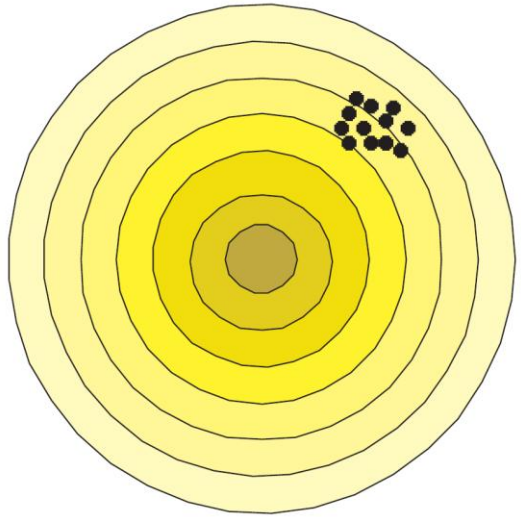
Bias – Variance Tradeoff

$$V_N(\theta) = \frac{1}{N}\sum_{k=0}^{N-1}(y_k - \hat{y}_k)^2$$

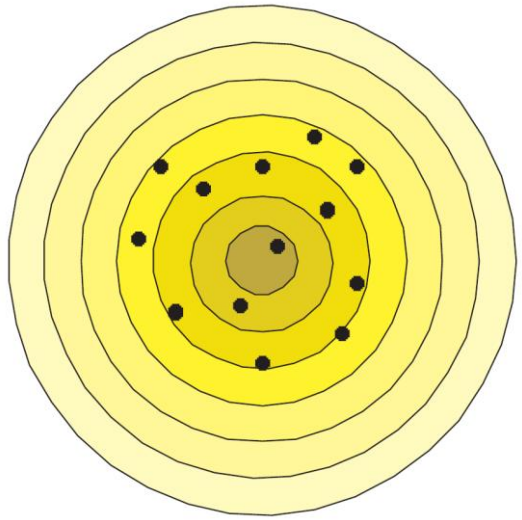
Can be split in a contribution due to parameter bias and parameter variance



Unbiased
Small variance



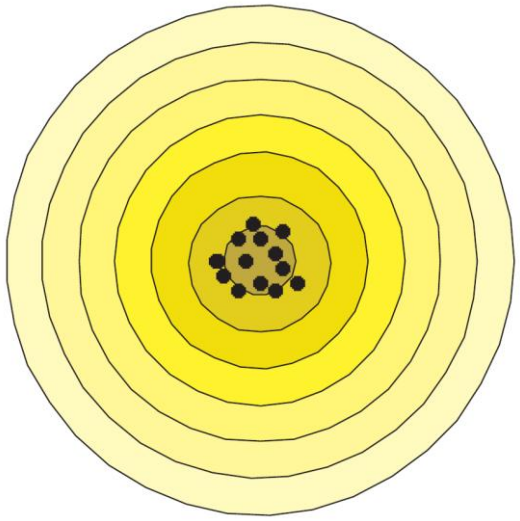
Biased
Small variance



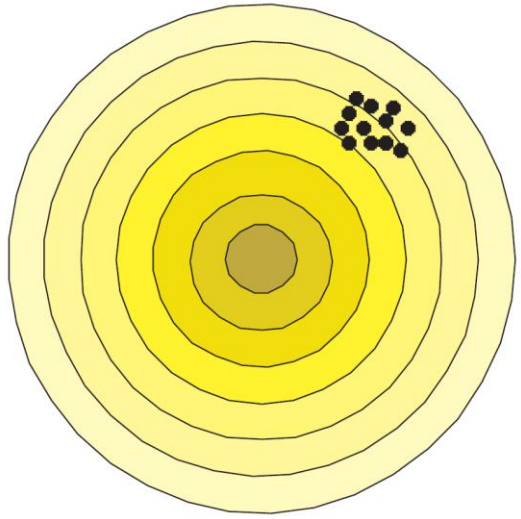
Unbiased
Large variance

Bias – Variance Tradeoff

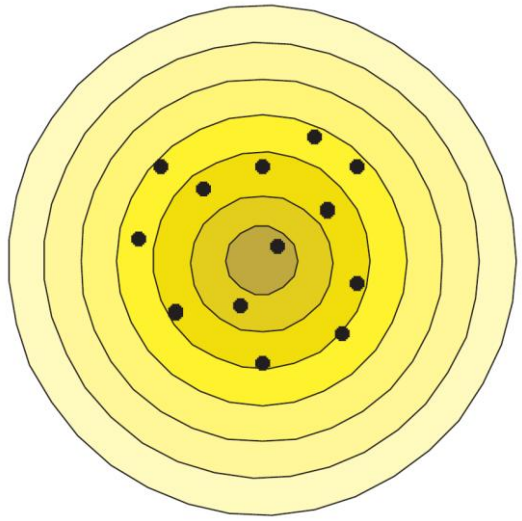
What combination of parameter bias and variance leads to the smallest cost?



Unbiased
Small variance



Biased
Small variance



Unbiased
Large variance

Estimation – Validation – Test

Estimation / Training dataset:

dataset used to estimate the parameters

Validation dataset:

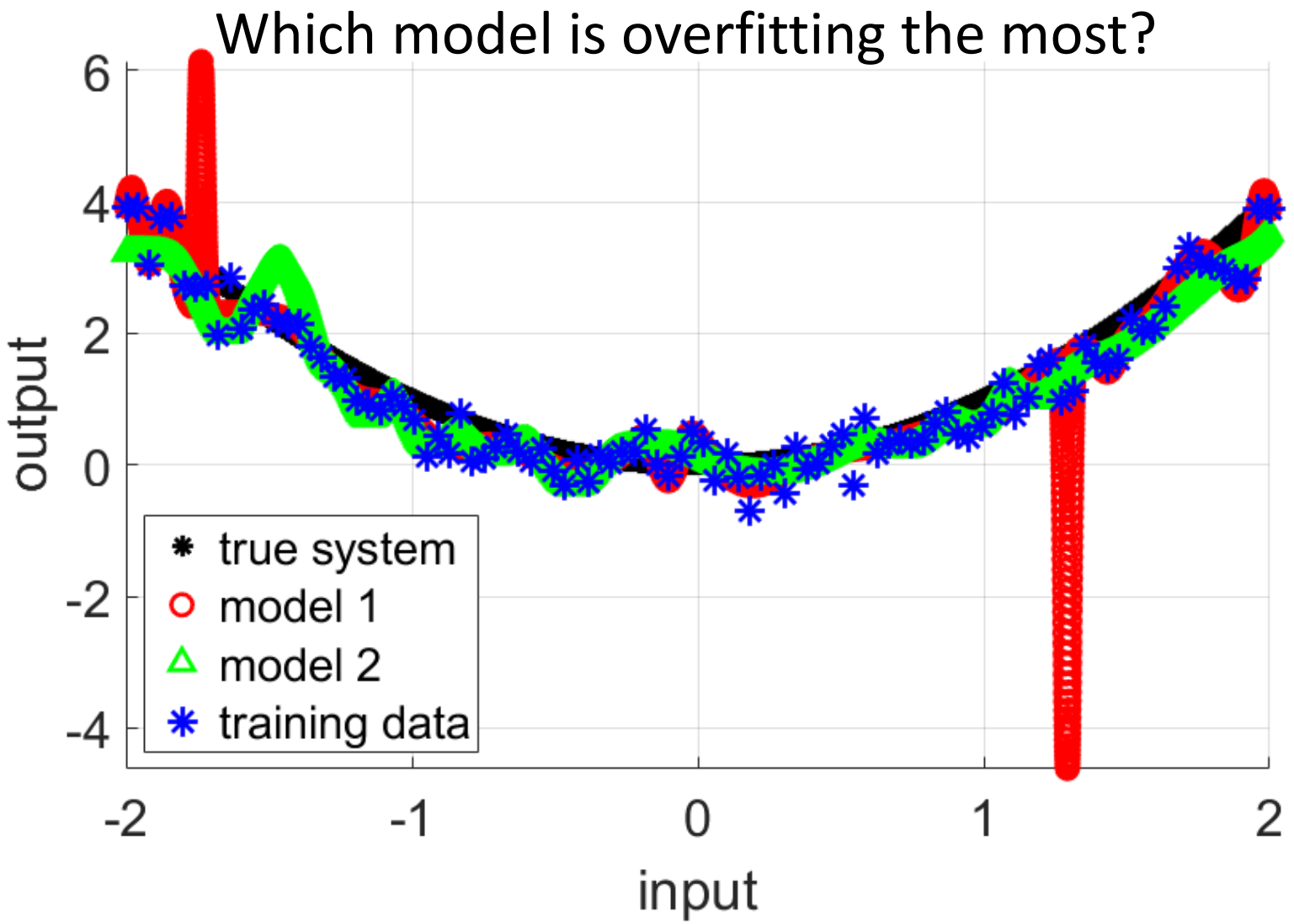
dataset used to validate the model structure, tune the hyperparameters (e.g. # neurons, # training iterations, network structure, …). The dataset is independent from the estimation data.

Test dataset:

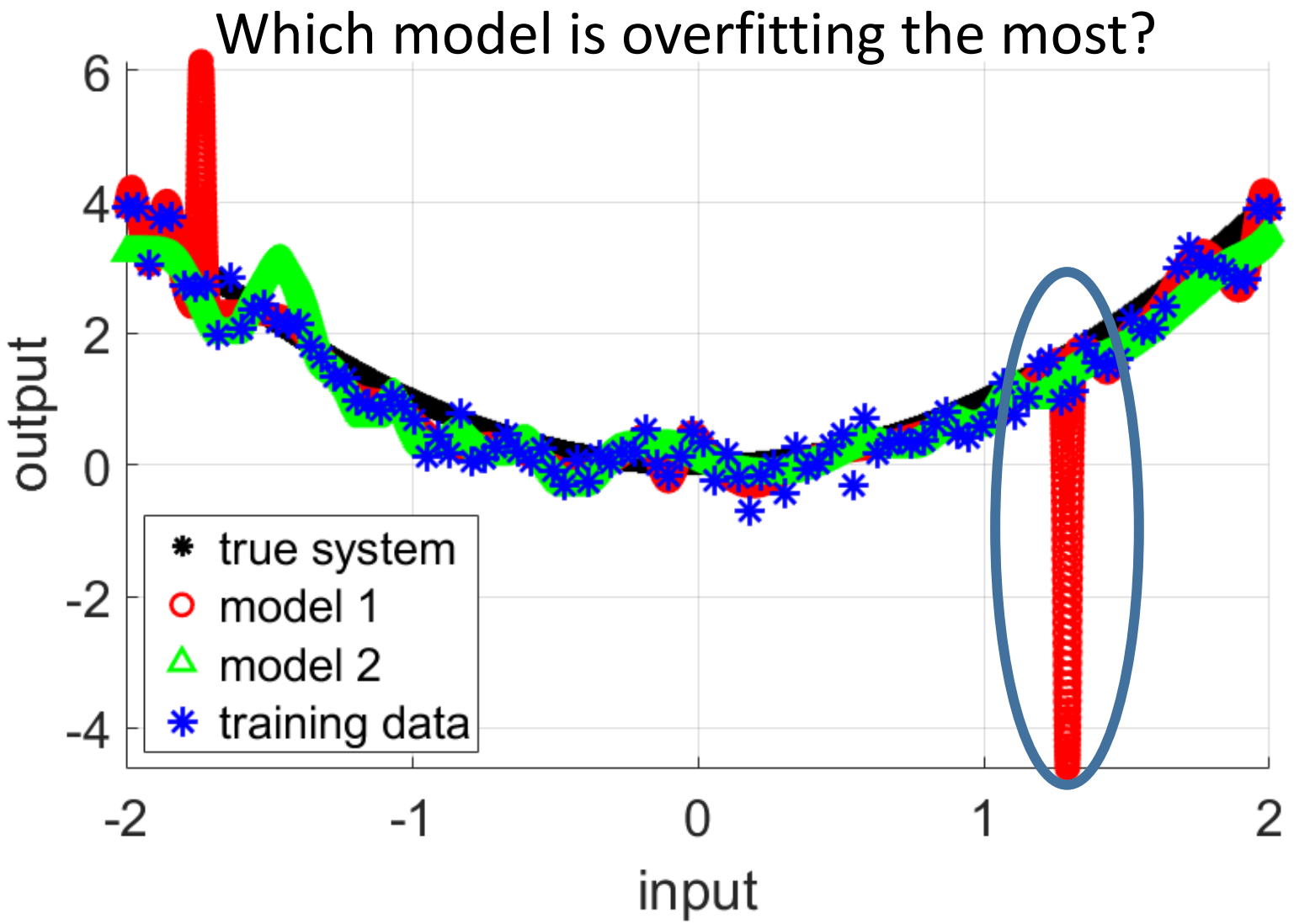
dataset to test the quality of the final model. The dataset is independent from the estimation and validation dataset.

Typically all datasets are chosen such that they follow the same probability density function and other statistics (e.g. power spectral density for time series).

Overfitting

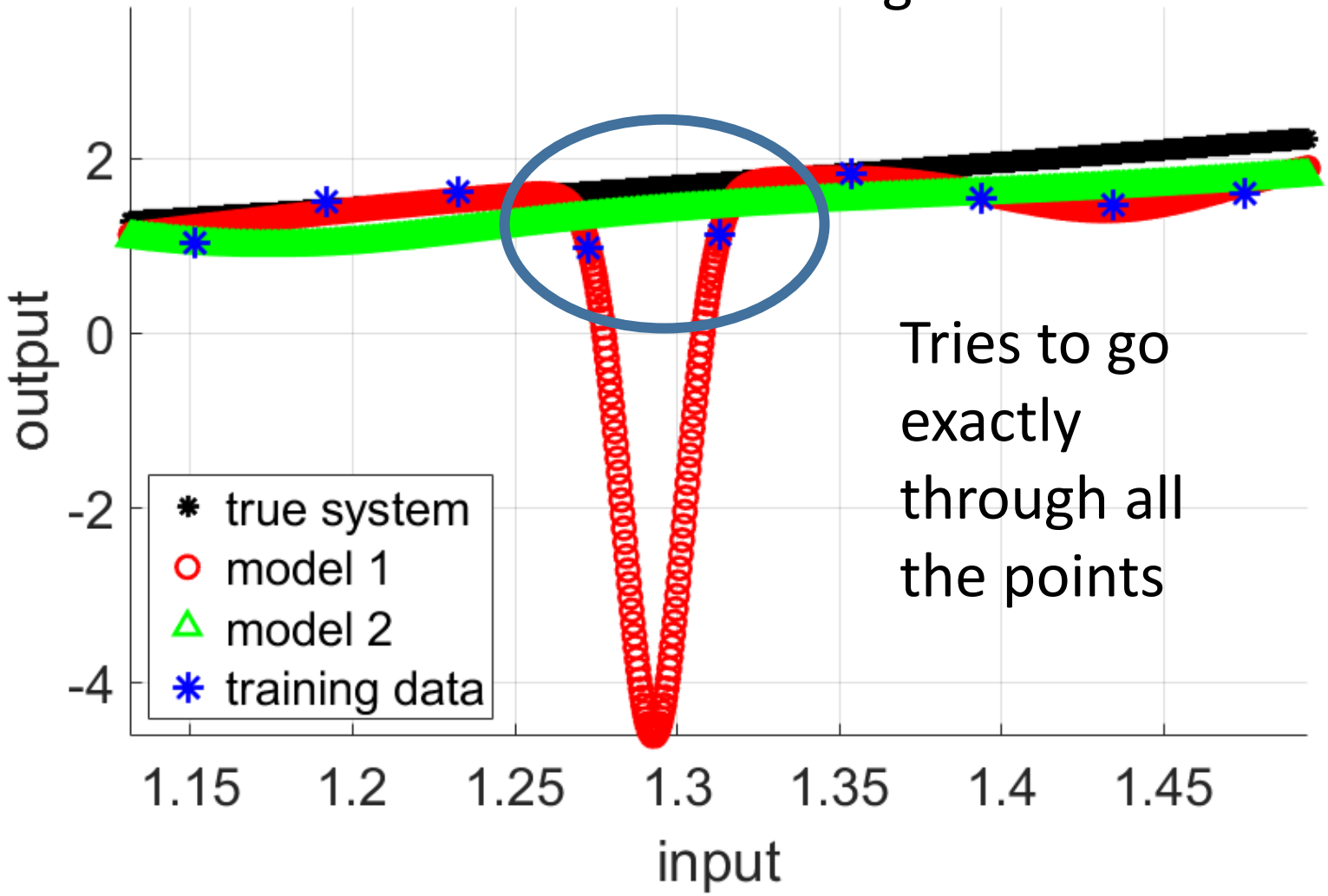


Overfitting



Overfitting

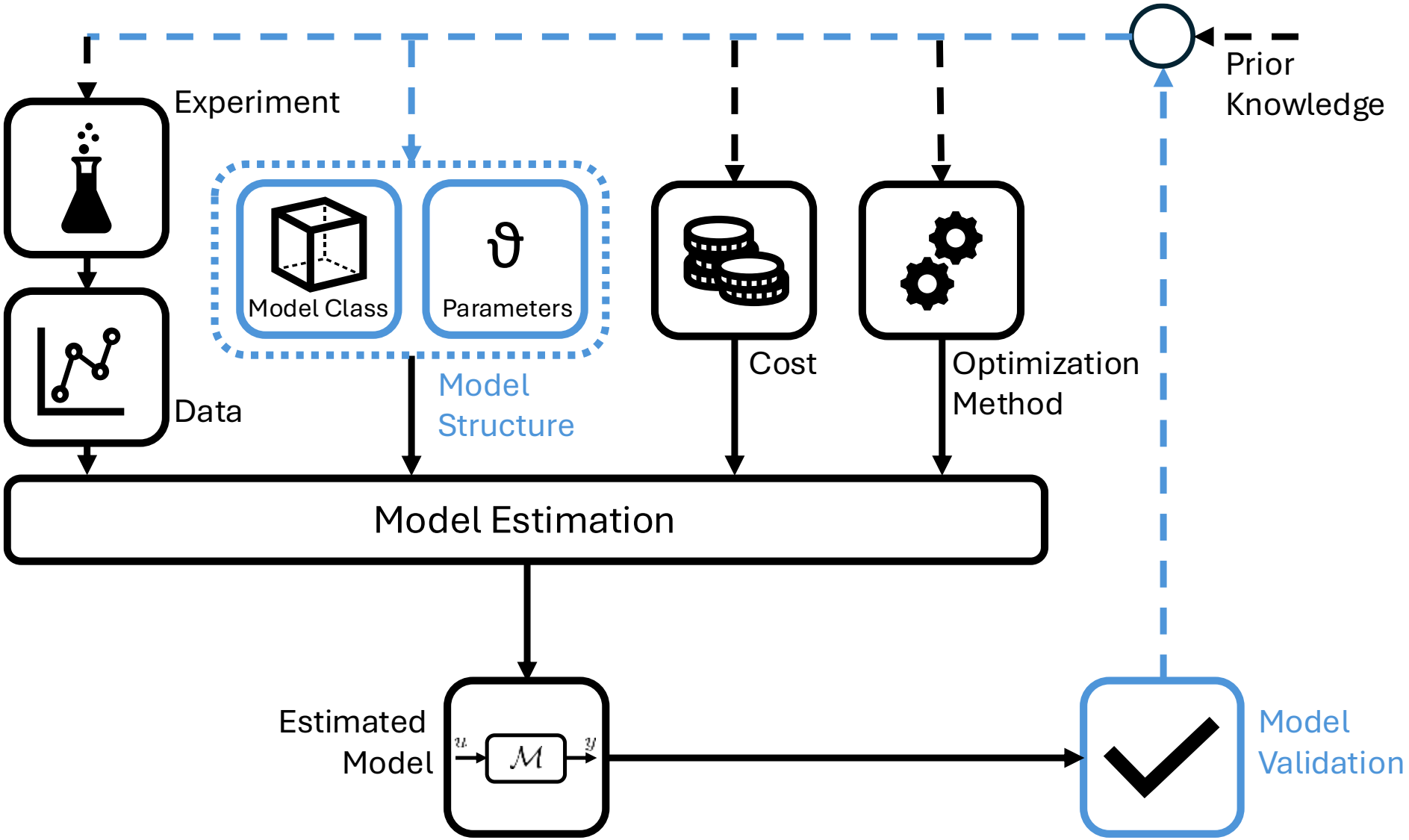
Which model is overfitting the most?



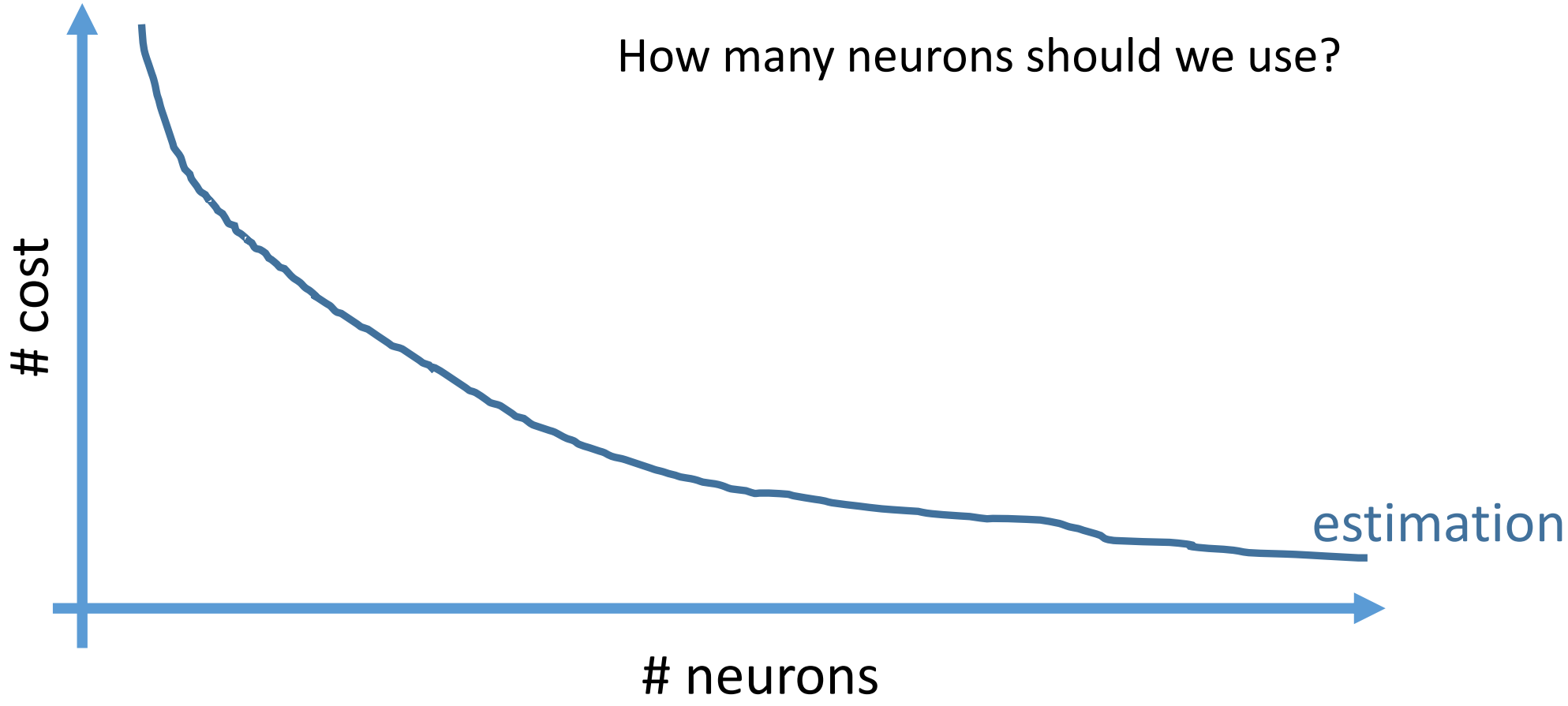
Overfitting

The model tries to reproduce too exactly the dataset used for estimation, due to which the model fails to predict or simulate additional data.

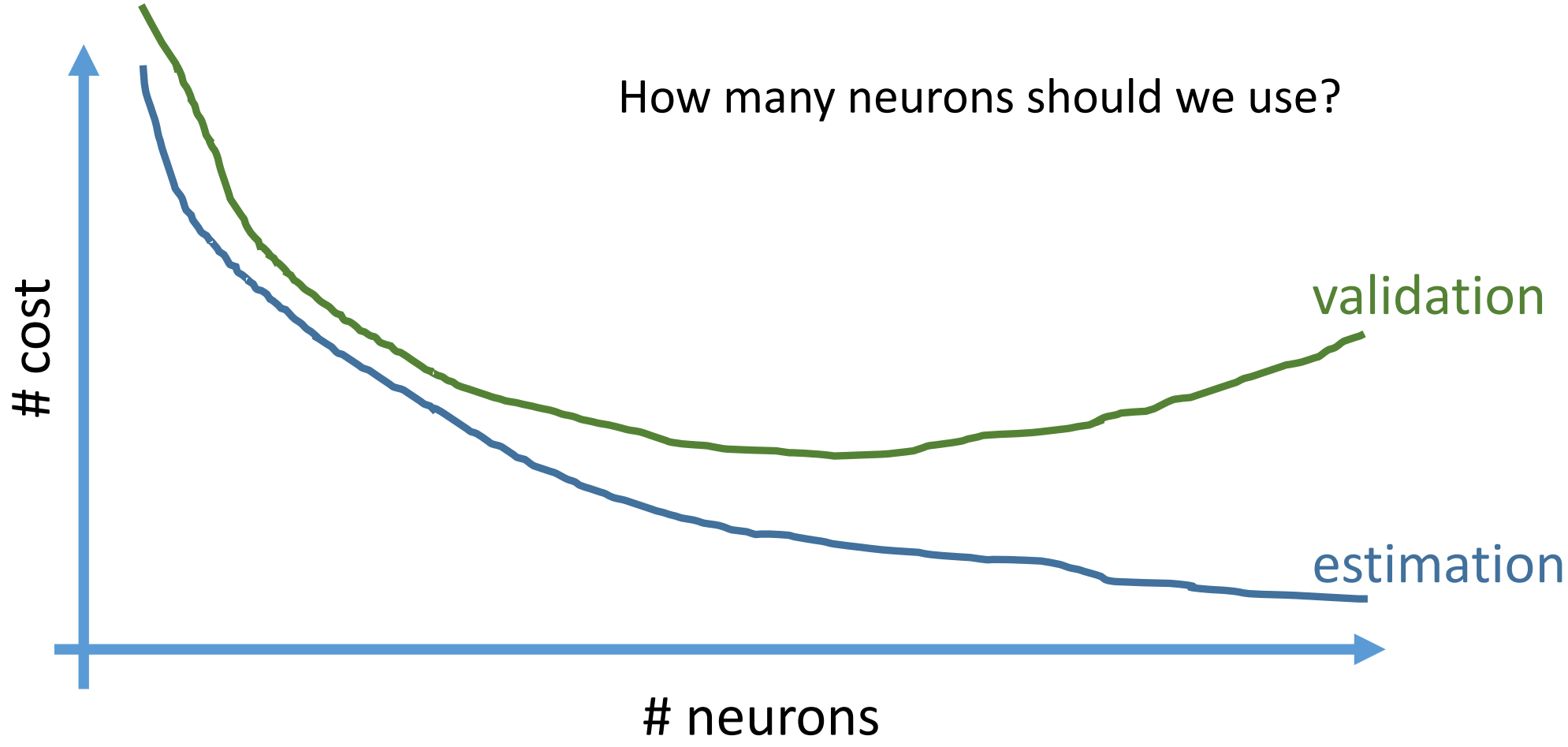
Model Order Selection



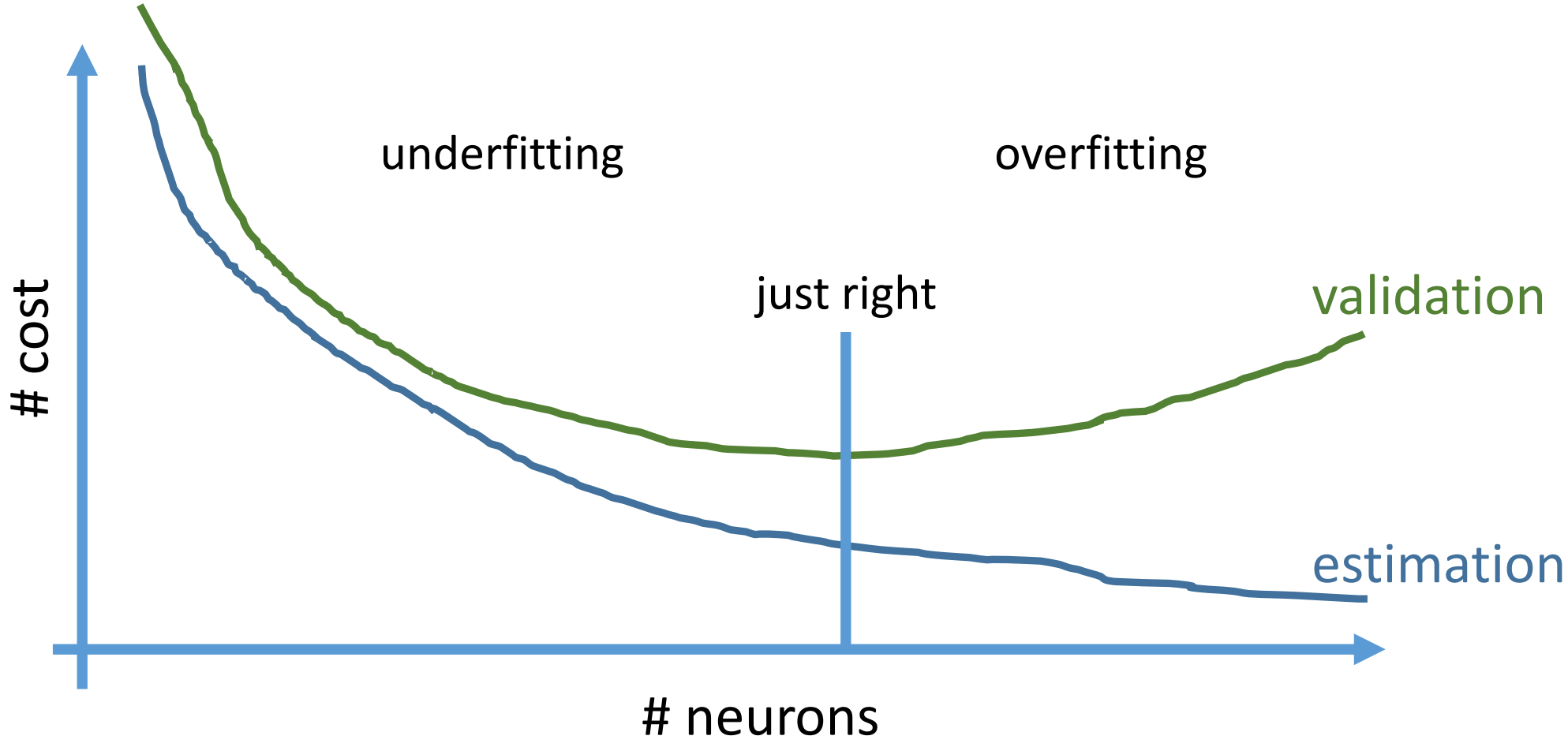
Overfitting – Model Order Selection



Overfitting – Model Order Selection



Overfitting – Model Order Selection



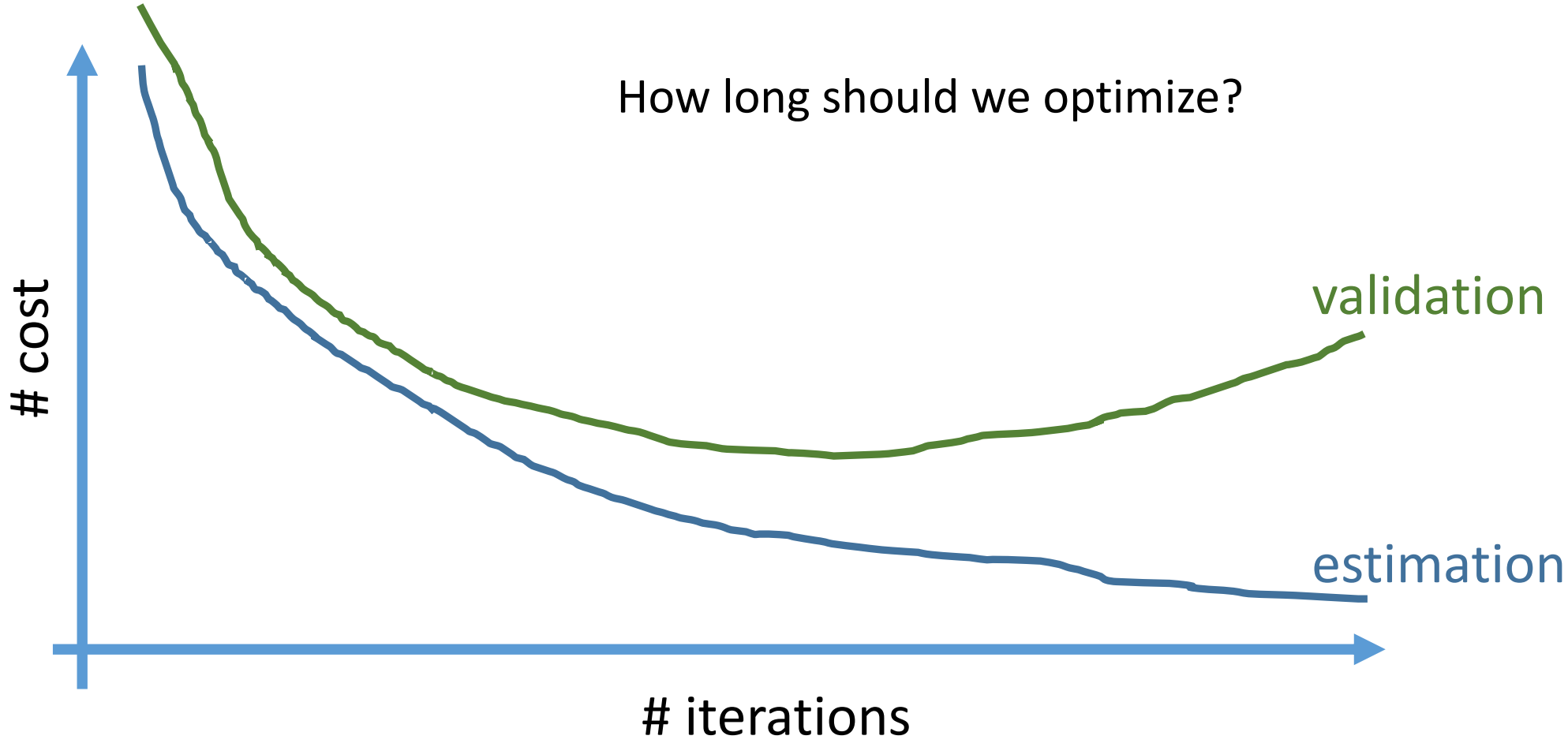
Overfitting - Early Stopping

At every iteration of the nonlinear optimization problem, evaluate the cost on the validation dataset.

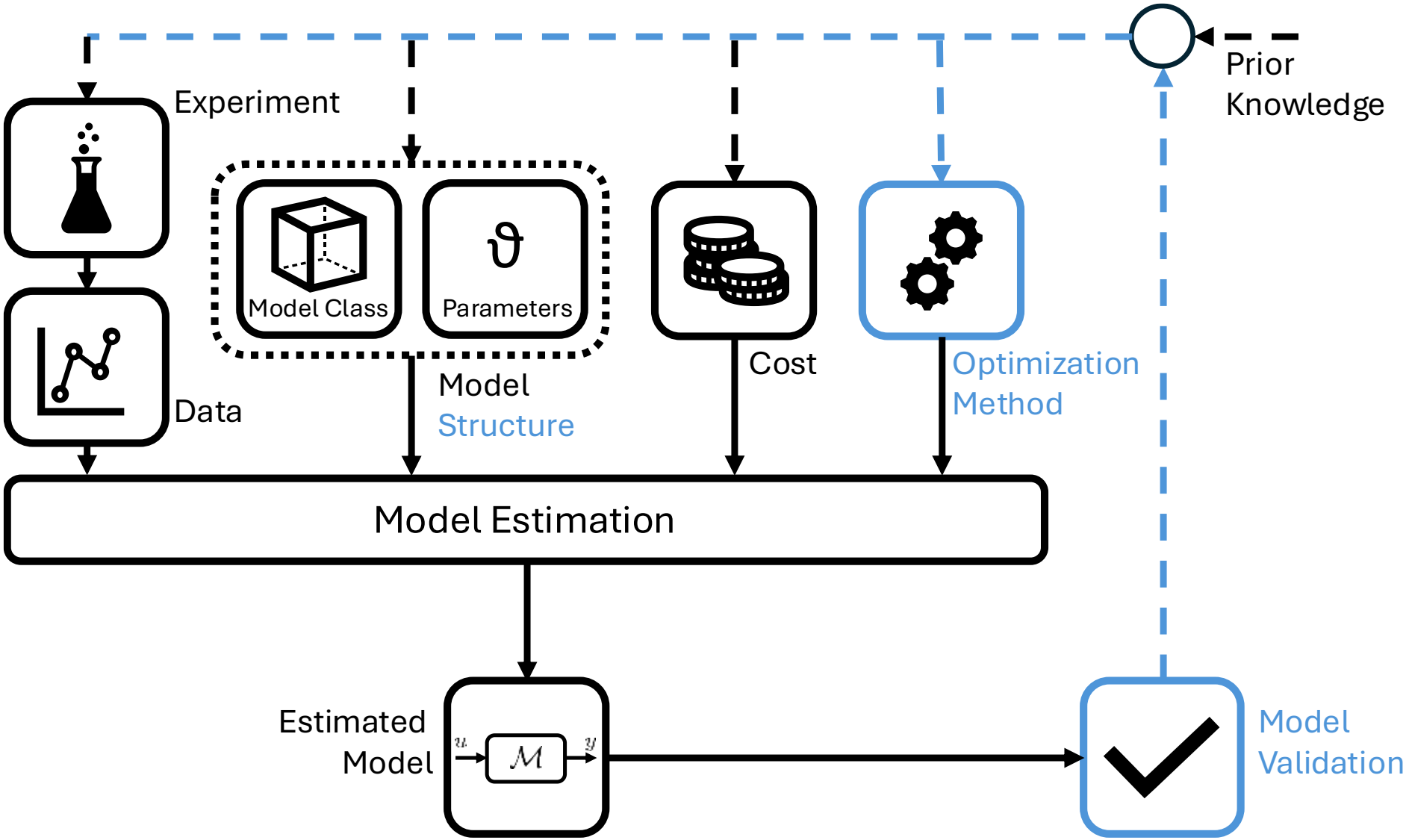
Two general use cases:

1. If validation cost does not decrease for m consecutive iterations: stop the optimization algorithm (shorter optimization time)
2. After optimization, select the iteration for which the lowest validation cost is obtained (longer optimization time)

Overfitting – Model Order Selection



Early Stopping



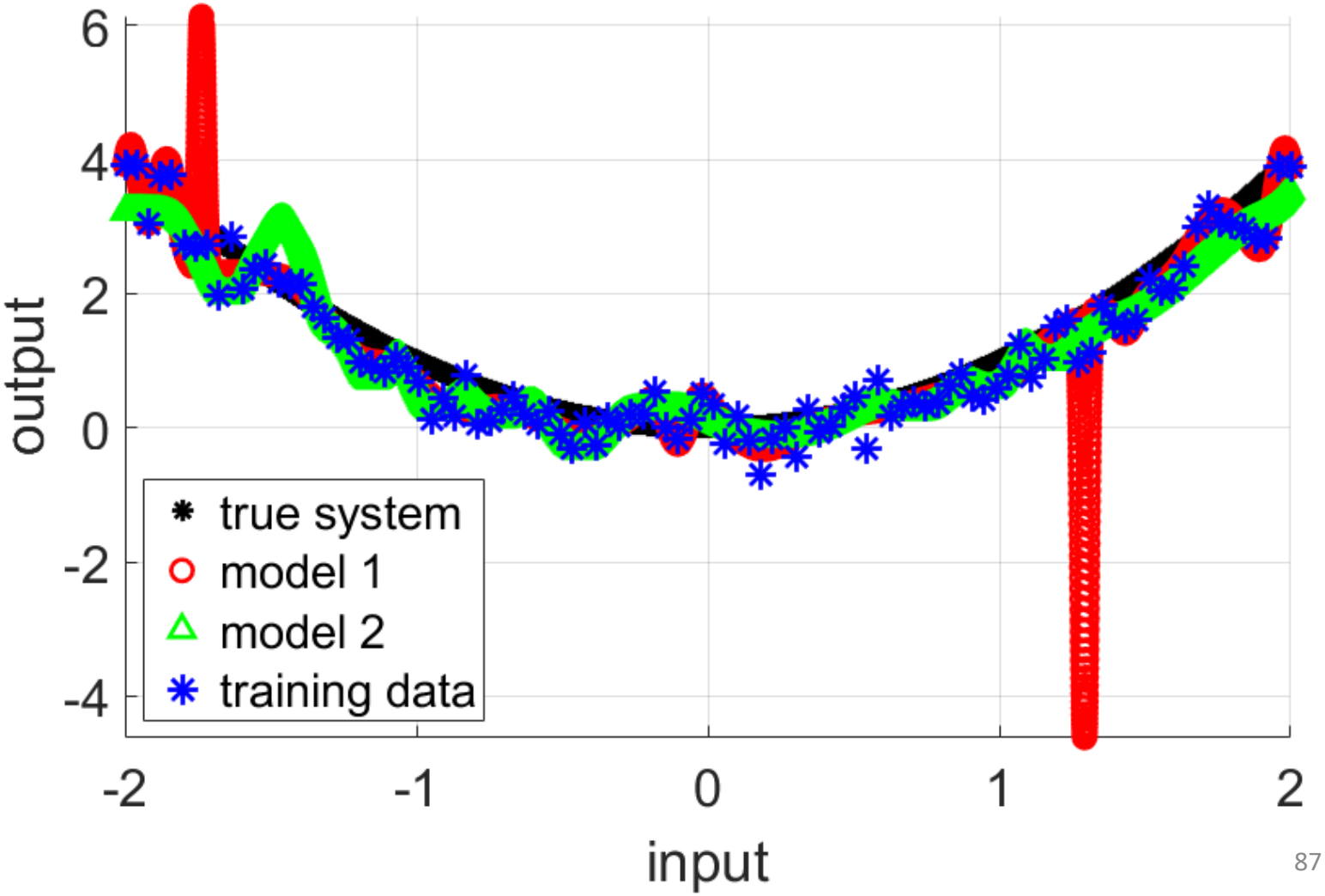
Early Stopping

Red:

No validation dataset is used.

Green:

20% of the total available data is used as validation dataset.



Splitting a Dataset

Standard approach:

Random assignment of datapoint to estimation / validation / test

Underlying assumption:

Neighboring datapoints are independent

Splitting a Dataset: Time-Series

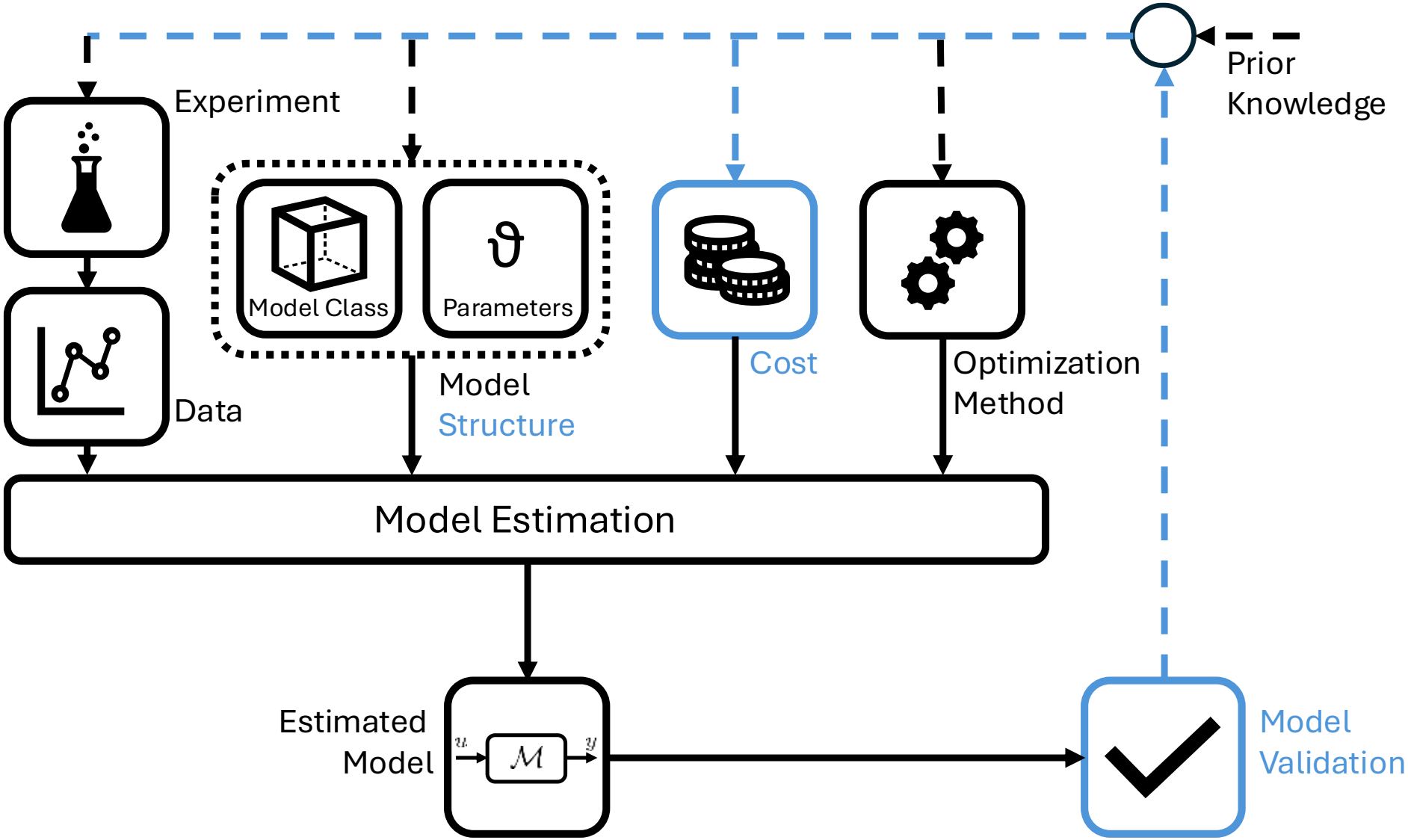
Neighboring datapoints are often NOT independent

Correlated input-output data

Correlated disturbances

➔ Work with 3 blocks of data or 3 separate time-series

Overfitting - Parameter Norm Penalization



Parameter Norm Penalization

l^2 Regularization, ridge regression or Thikonov regularization:

$$V_N(\theta) = \frac{1}{N}\sum_{k=0}^{N-1}(y_k - \hat{y}_k)^2 = \frac{1}{N}e^\top e$$

$$V_N(\theta) = (1-\lambda)\frac{1}{N}e^\top e + \lambda\theta^\top\theta$$

Parameter vector

Hyperparameter: trade-off between regularization and data fit

Typically only the network weights are penalized, not the biases
Since the network weights tell how multiple neuron inputs are combined, more data is required to estimate them well

Parameter Norm Penalization

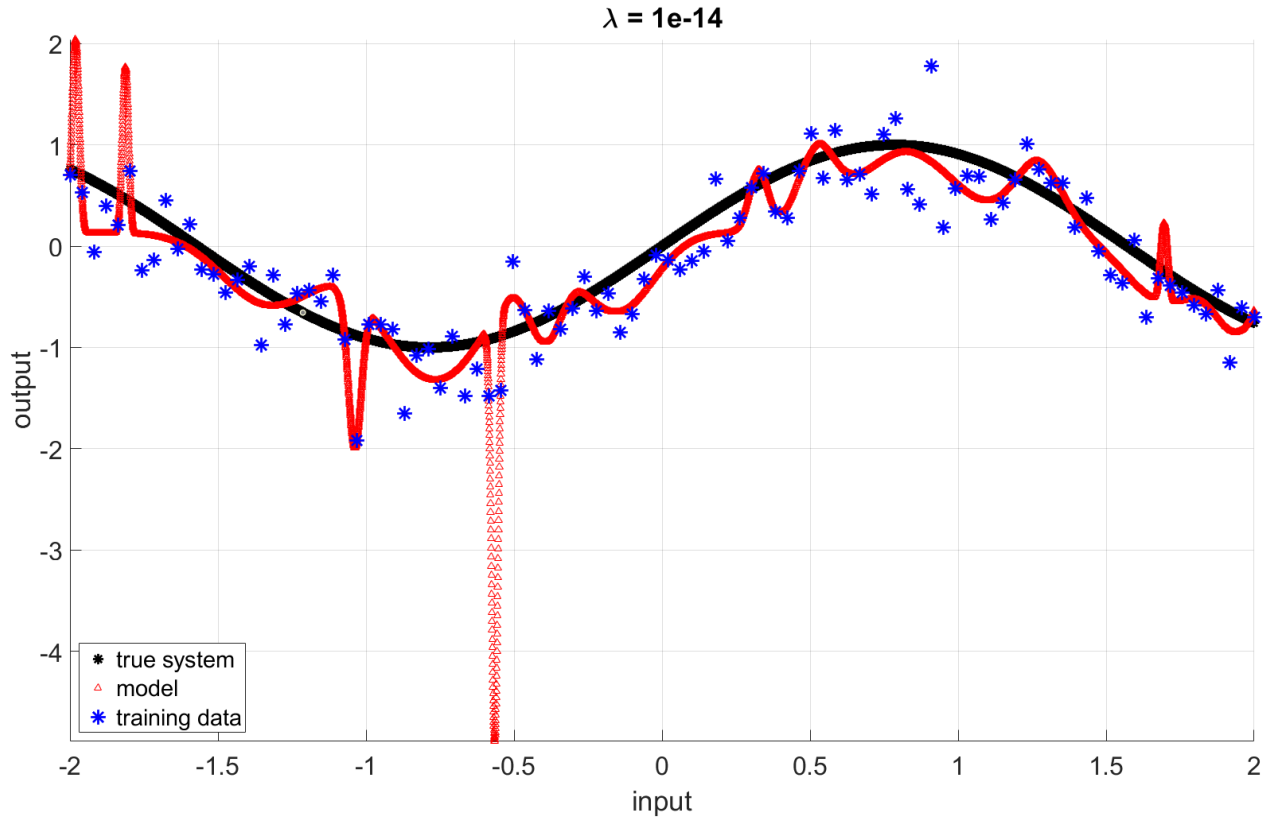
l^2 Regularization, ridge regression or Thikonov regularization:

Forces changes in the parameter vector that do not contribute significantly in reducing the cost to decay away during training
(more info in: I. Goodfellow et al., *Deep Learning*, The MIT Press, pp. 223-227)

Also used for RKHS estimation: see lecture 2

Parameter Norm Penalization

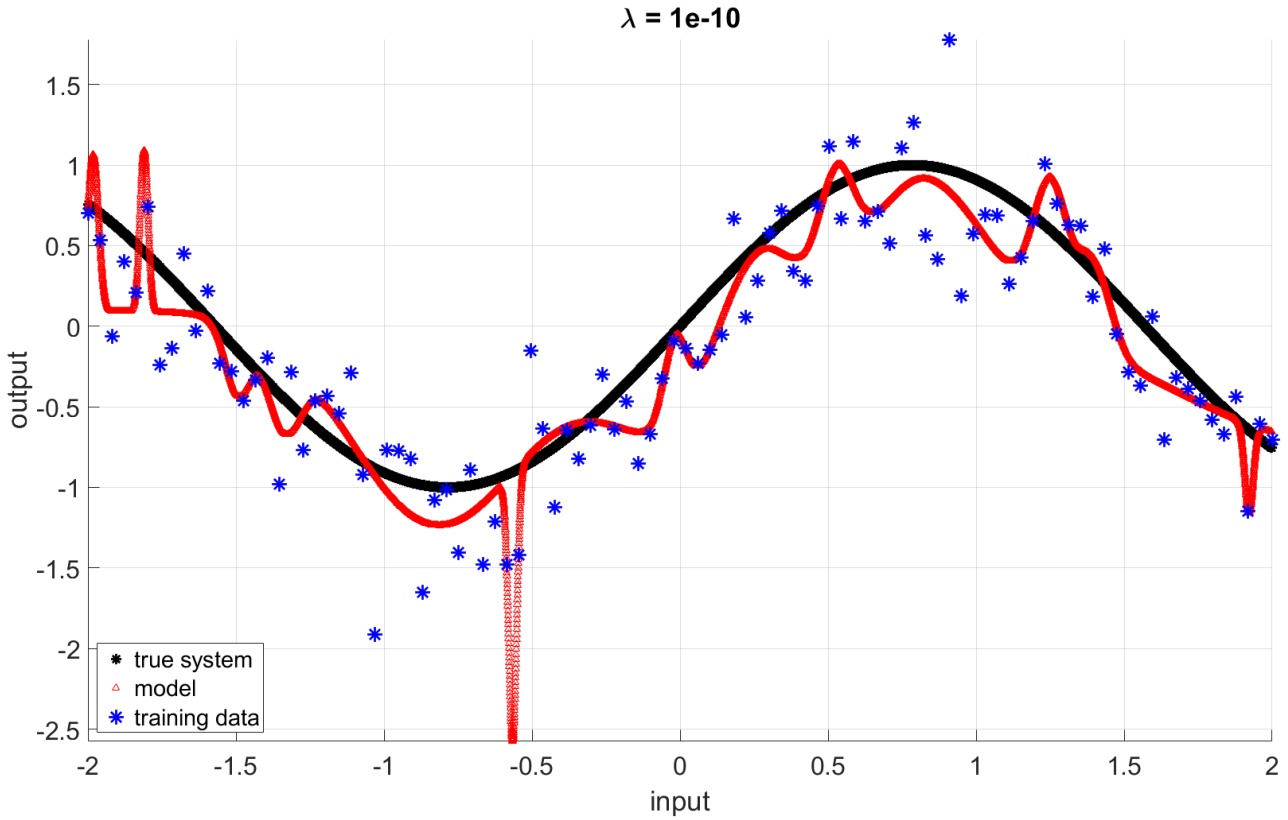
neurons = 15



$$V_N(\theta) = (1-\lambda)\frac{1}{N}e^\top e + \lambda\theta^T\theta$$

Parameter Norm Penalization

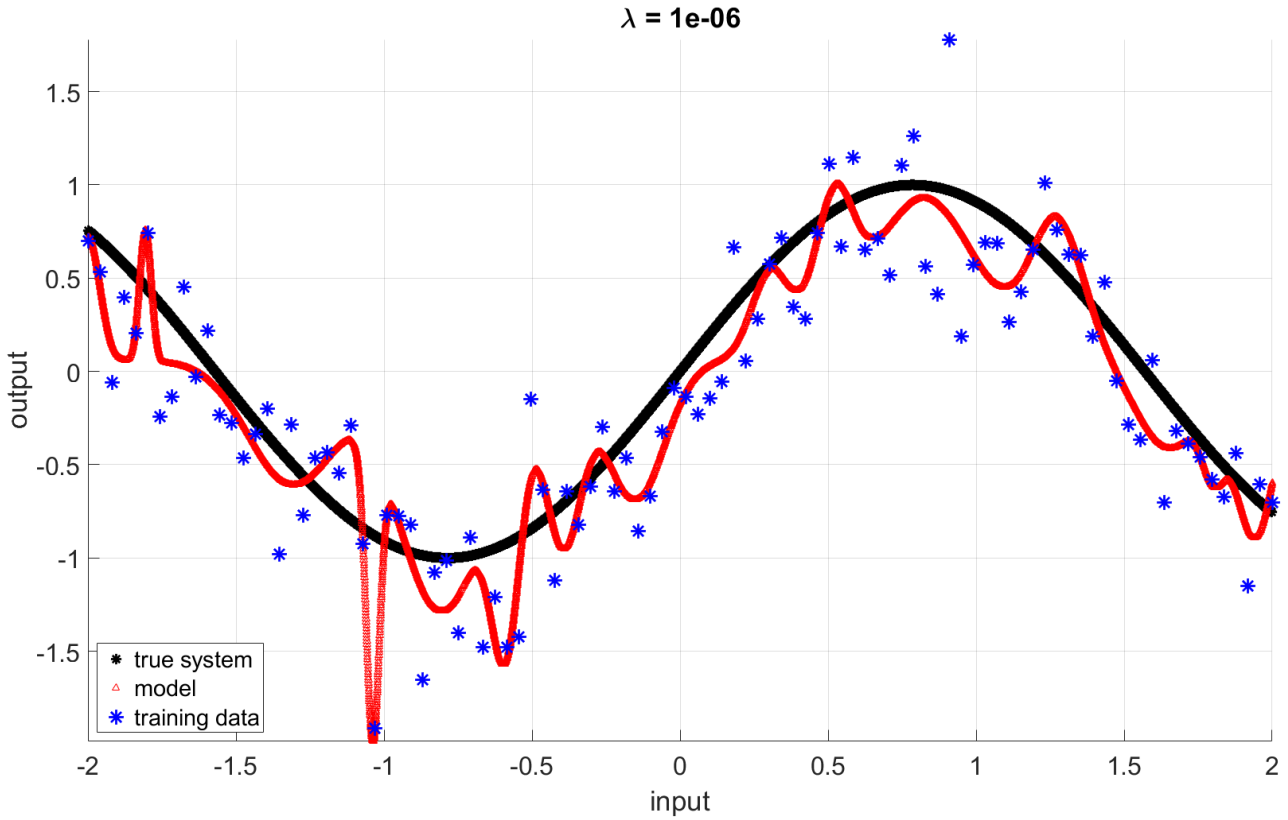
neurons = 15



$$V_N(\theta) = (1-\lambda)\frac{1}{N}e^\top e + \lambda\theta^T\theta$$

Parameter Norm Penalization

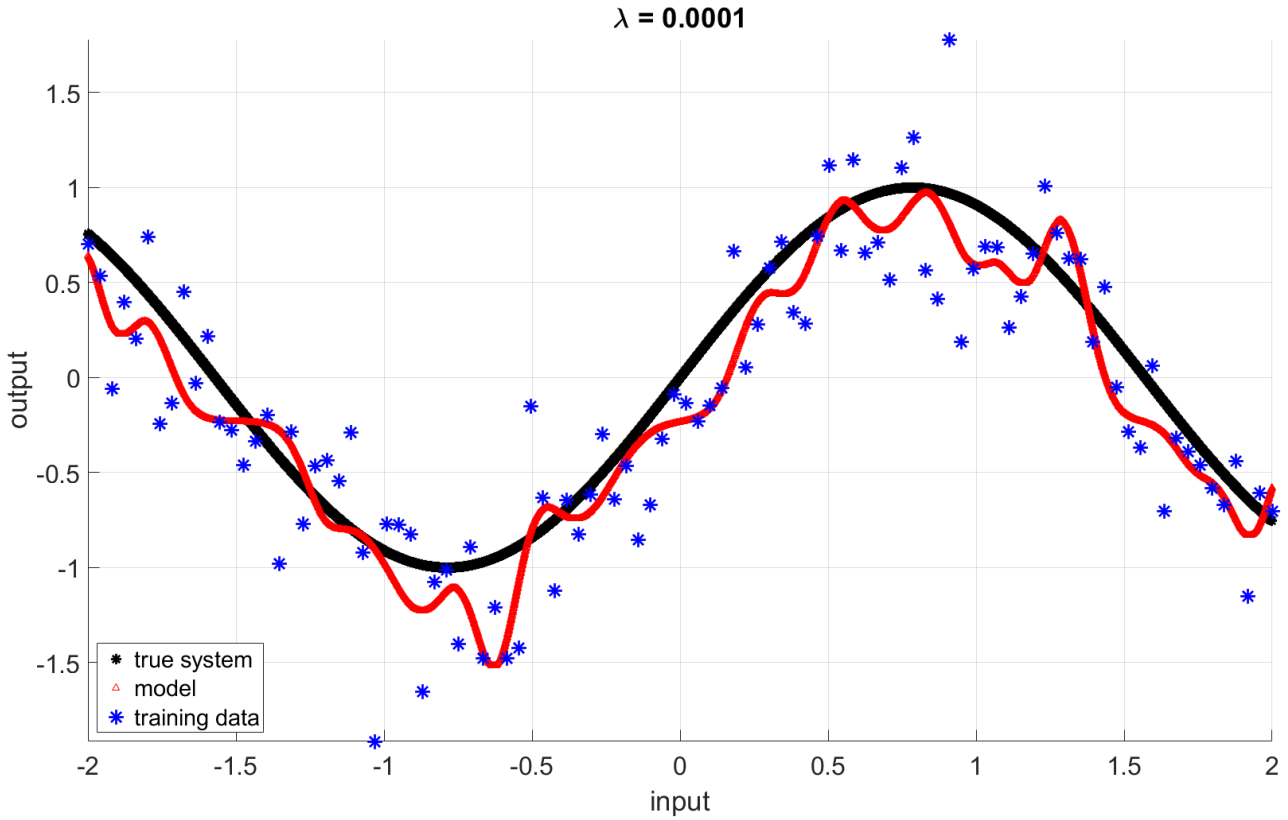
neurons = 15



$$V_N(\theta) = (1-\lambda)\frac{1}{N}e^\top e + \lambda\theta^T\theta$$

Parameter Norm Penalization

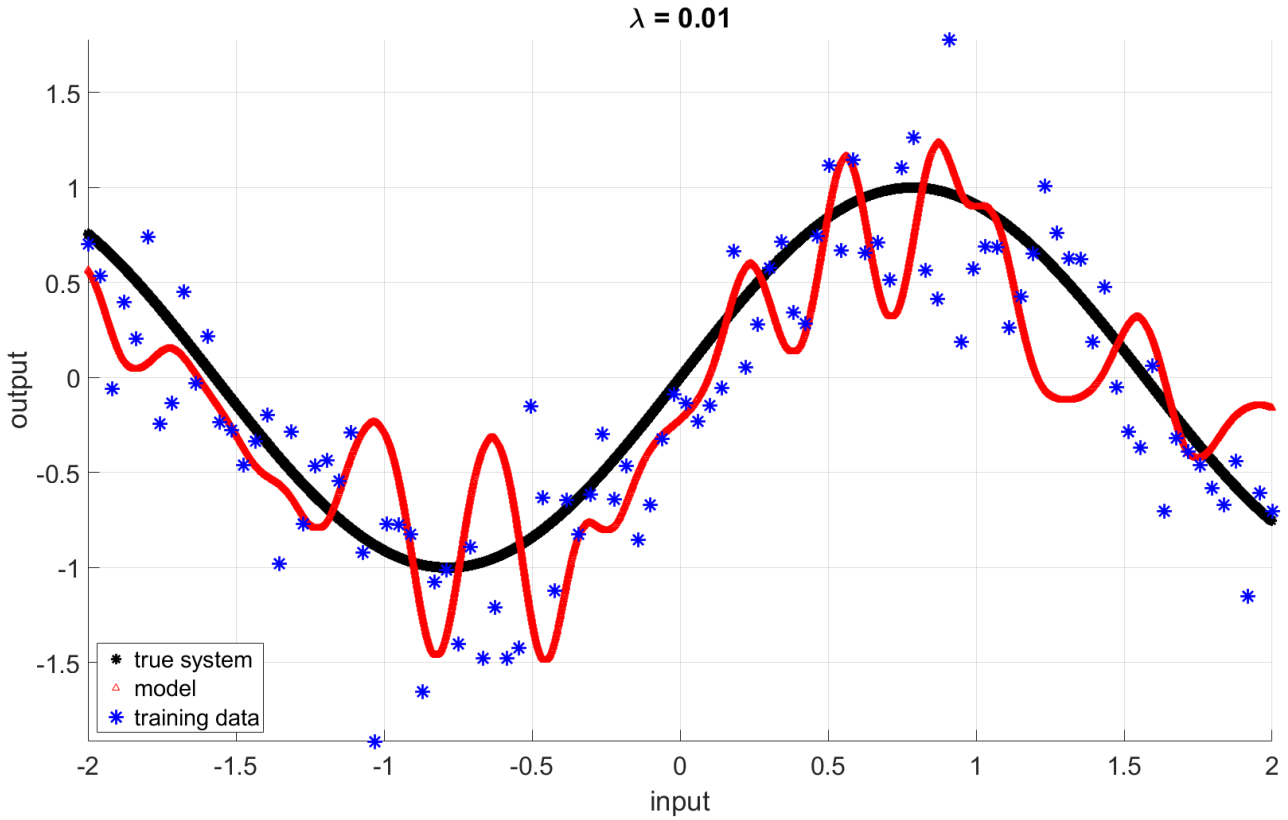
neurons = 15



$$V_N(\theta) = (1-\lambda)\frac{1}{N}e^\top e + \lambda\theta^T\theta$$

Parameter Norm Penalization

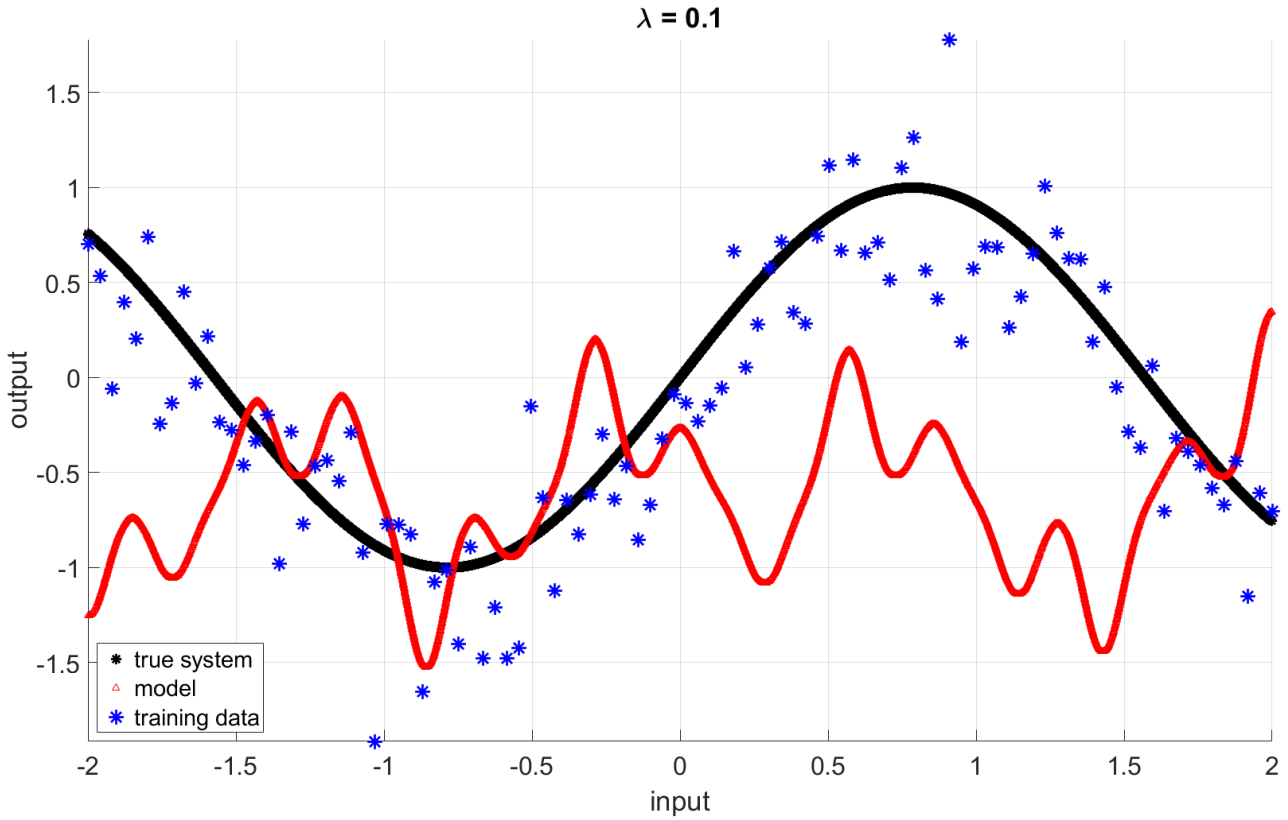
neurons = 15



$$V_N(\theta) = (1-\lambda)\frac{1}{N}e^\top e + \lambda\theta^T\theta$$

Parameter Norm Penalization

neurons = 15



$$V_N(\theta) = (1-\lambda)\frac{1}{N}e^\top e + \lambda\theta^T\theta$$

Parameter Norm Penalization

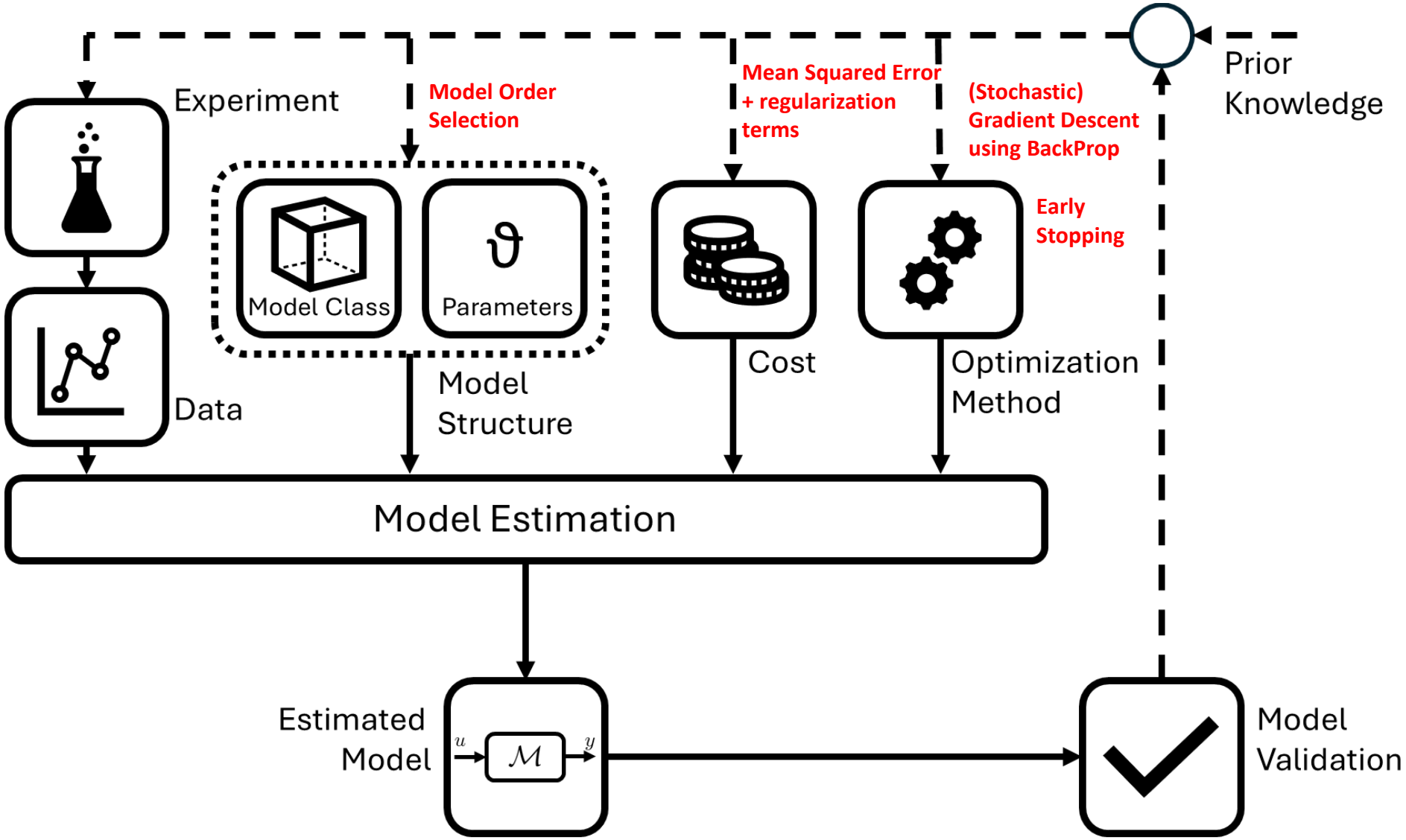
l^2 Regularization, ridge regression or Thikonov regularization:

$$V_N(\theta) = (1-\lambda)\frac{1}{N}e^\top e + \lambda\theta^\top\theta$$

Select hyperparameter through validation

(for other approaches see GP lecture)

Overfitting: Data-Driven Modelling Process



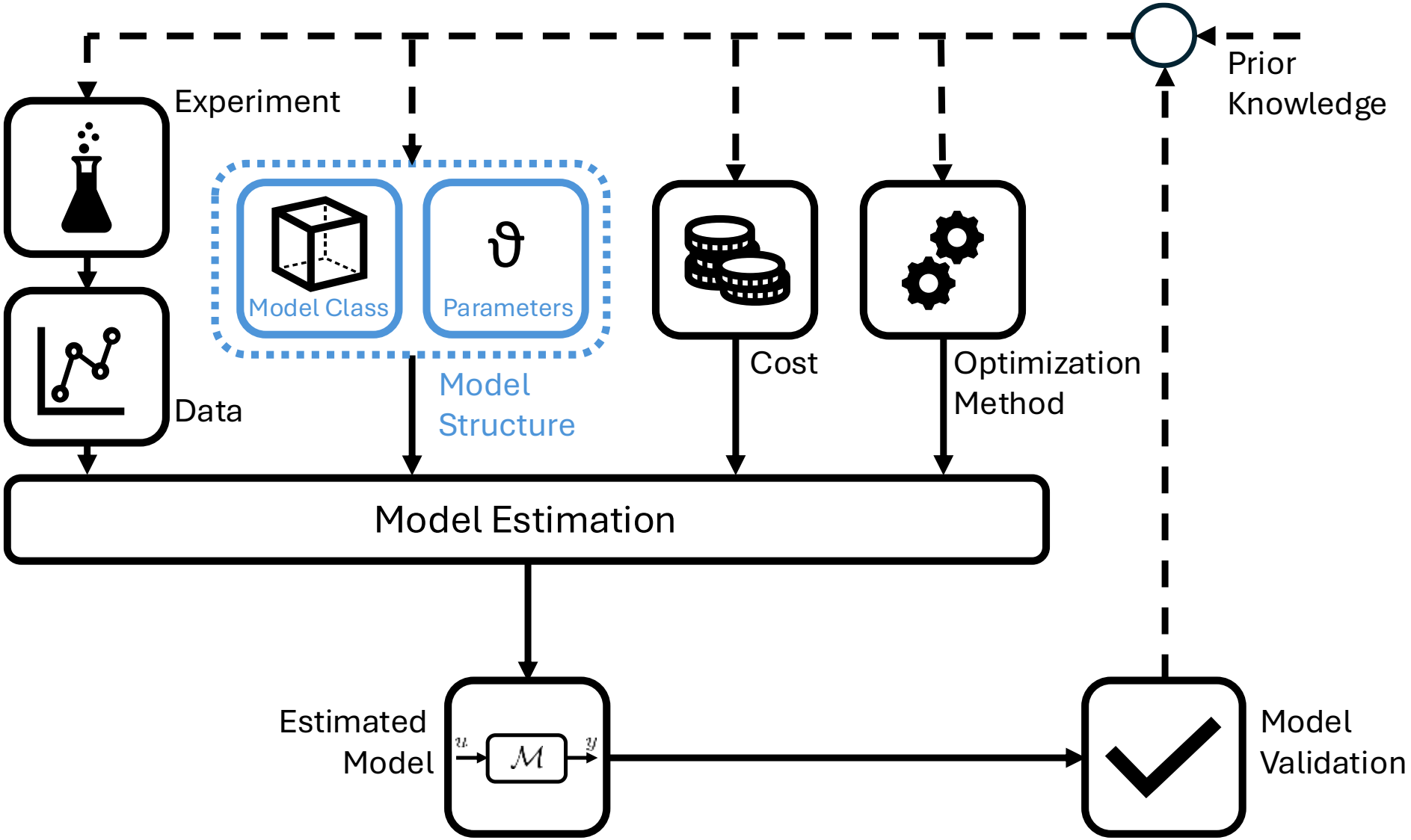
Artificial Neural Networks

Training, Validation and Test (or Training a Neural Network Cont'd)

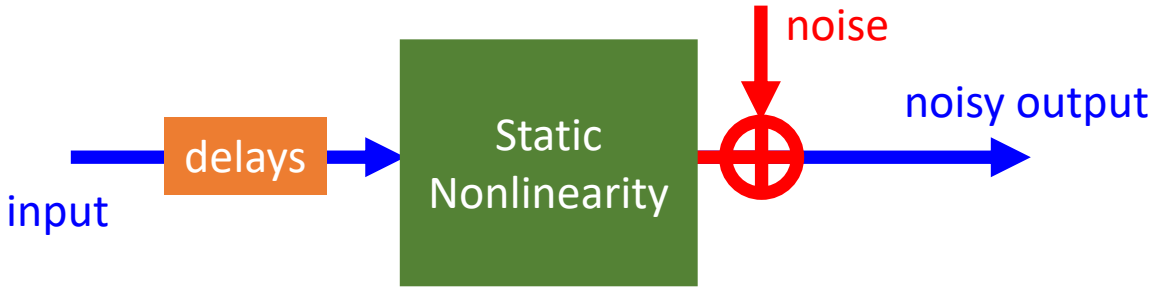
Overfitting and Regularization

Simple Neural Networks to Model Dynamics

Overfitting: Data-Driven Modelling Process



Nonlinear Finite Impulse Response (NFIR)

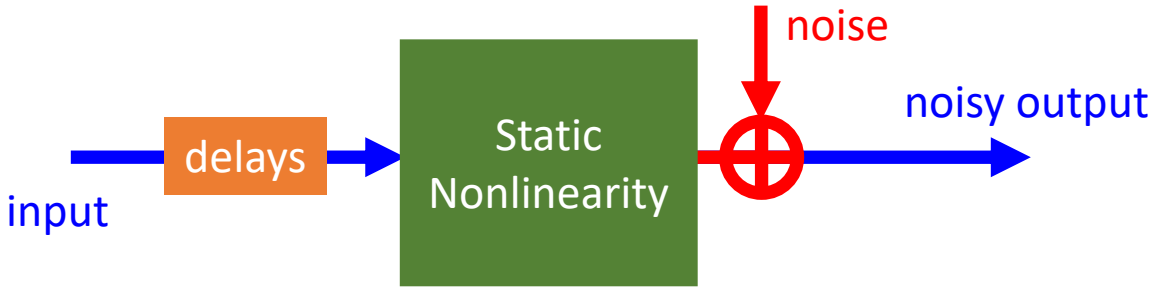


$$y_k = f\left(u_k, u_{k-1}, \ldots, u_{k-n_b}\right) + e_k$$

The nonlinear function can be represented by a simple feedforward network with 1 or more hidden layer

Network inputs: delayed system inputs

Nonlinear Finite Impulse Response (NFIR)



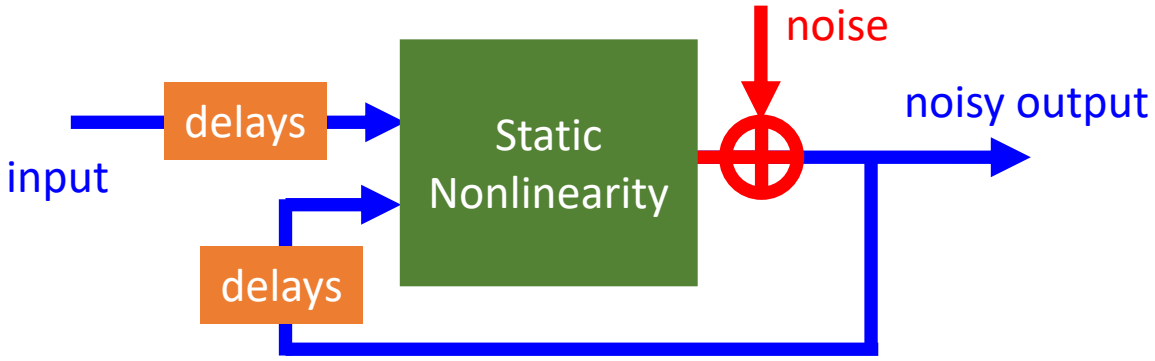
$$y_k = f(u_k, u_{k-1}, \ldots, u_{k-n_b}) + e_k$$

To be estimated known unknown

The nonlinear function can be represented by a simple feedforward network with 1 or more hidden layer

Network inputs: delayed system inputs

Nonlinear Autoregressive with eXogenous Input (NARX)

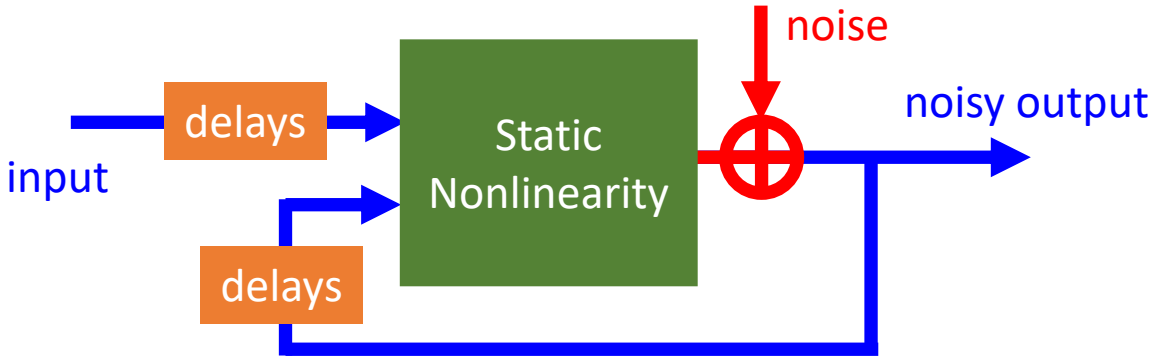


$$y_k = f\left(u_k, u_{k-1}, \ldots, u_{k-n_b}, y_{k-1}, \ldots, y_{k-n_a}\right) + e_k$$

The nonlinear function can be represented by a simple feedforward network with 1 or more hidden layer

Network inputs: delayed system inputs and outputs

Nonlinear Autoregressive with eXogenous Input (NARX)



$$y_k = f(u_k, u_{k-1}, \ldots, u_{k-n_b}, y_{k-1}, \ldots, y_{k-n_a}) + e_k$$

To be estimated known unknown

The nonlinear function can be represented by a simple feedforward network with 1 or more hidden layer

Network inputs: delayed system inputs and outputs

Nonlinear State Space (Measured States)

known states

known input

$$x_{k+1} = f\left(x_k, u_k\right) + w_k$$

process noise

$$y_k = h\left(x_k, u_k\right) + e_k$$

output noise

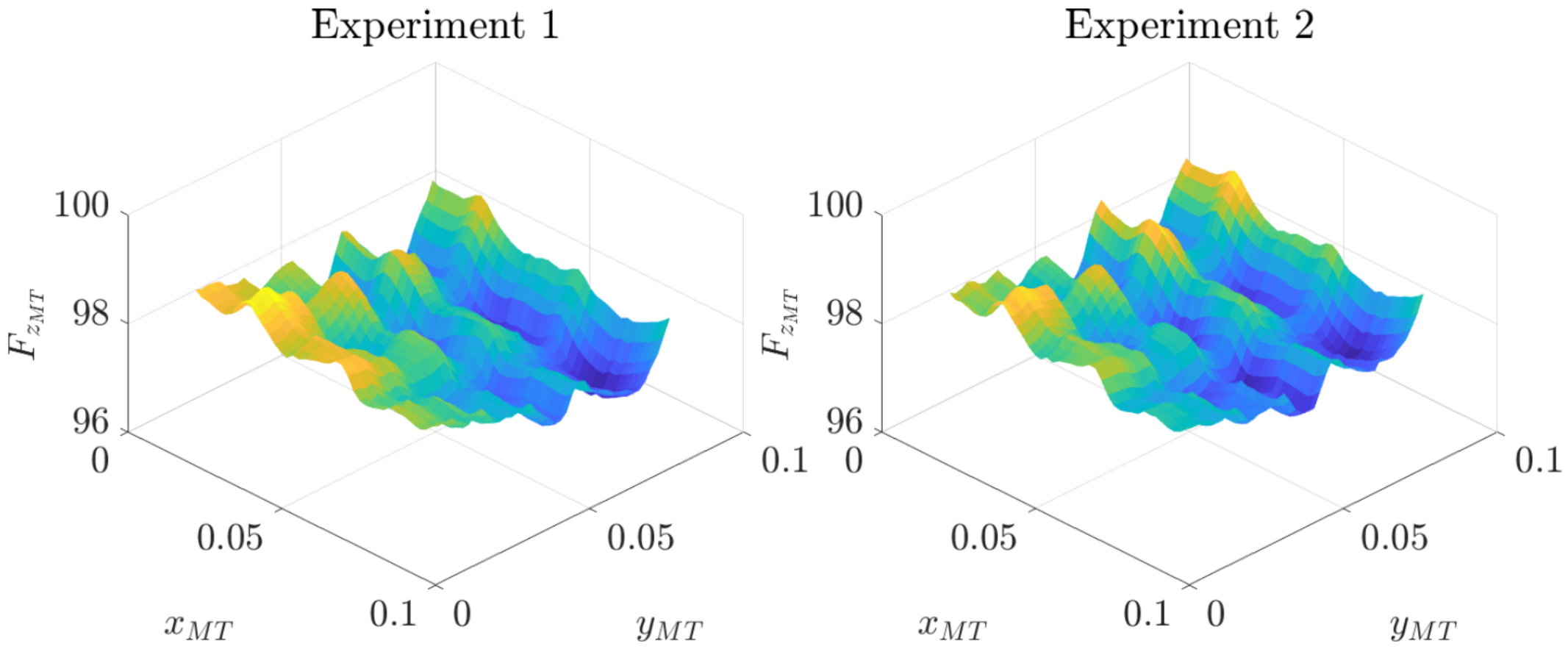
known output

multivariate static nonlinearities

The nonlinear function can be represented by a simple feedforward network with 1 or more hidden layer

Network inputs: system inputs and states

Capturing Residual Dynamics



Conclusion

What is an artificial neural network?

Simple feedforward networks

Approximation properties

Training an artificial neural network

Training, Validation and Test (or Training a Neural Network Cont'd)

What is Missing?

What is deep learning and what are deep neural networks?

How to train deep neural networks?

Deep neural networks for dynamical systems?